Appunti di Information Retrieval

Appunti del Corso

Anno Accademico 2026/2027

Indice

0 Introduzione e Architettura dell'Information Retrieval **1**
- 0.1 Definizione e Motivazioni . . . 1
- 0.2 I Due Macro-Obiettivi: Efficacia ed Efficienza . . . 3
- 0.3 Architettura di un Sistema di Information Retrieval . . . 3
- 0.4 Proprietà del Linguaggio e Modelli di Reperimento . . . 4
- 0.5 L'Inverted Index (Indice Invertito) . . . 5
- 0.6 Tecniche di Compressione dell'Indice . . . 6
- 0.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità 7
- 0.8 Evoluzione verso il Neural Information Retrieval . . . 7

1 Valutazione dei Sistemi di Information Retrieval **9**
- 1.1 Architettura di un Motore di Ricerca . . . 9
- 1.2 Come Valutare la Qualità di un Motore di Ricerca . . . 11
- 1.3 Misurare la Soddisfazione dell'Utente . . . 12
- 1.4 Il Paradigma di Cranfield e le Collezioni Benchmark . . . 15
- 1.5 Bisogno Informativo vs. Query . . . 19
- 1.6 Metriche di Valutazione Non Ordinate (*Unranked Measures*) . 19
- 1.7 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria) . . . 22
- 1.8 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG . . . 28

2 Efficienza Algoritmica sulle Architetture CPU Moderne **33**
- 2.1 Il Ruolo dell'Efficienza nell'Information Retrieval . . . 33
- 2.2 La Gerarchia di Memoria e i Principi di Località . . . 34
- 2.3 Caso di Studio: Scansione Sequenziale contro *Pointer Chasing* 37
- 2.4 Forme di Parallelismo e Architettura della Pipeline . . . 40
- 2.5 Esecuzione Superscalare, Interleaving e Conflitti Strutturali . . 41
- 2.6 Data Hazard e Dipendenze Seriali . . . 44
- 2.7 Control Hazard, Esecuzione Speculativa e Branch Prediction . 45
- 2.8 Sintesi: dai Vincoli Hardware alle Scelte Algoritmiche . . . 48

3 Elaborazione del Linguaggio Naturale e Preprocessing del Testo **50**
3.1 Recap: Metriche di Valutazione per Rilevanza Graduata . . . 50
3.2 Proprietà Fondamentali del Linguaggio Naturale 52
3.3 Acquisizione e Preprocessing del Testo 56
3.4 Modelli di Rappresentazione: Bag of Words e N-Grammi . . . 58

4 Modelli di Retrieval, Scoring e Ponderazione TF-IDF **61**
4.1 Evoluzione dei Paradigmi di Information Retrieval 61
4.2 Boolean Retrieval vs. Ranked Retrieval 62
4.3 Misure di Similarità Set-Based: Il Coefficiente di Jaccard . . . 63
4.4 Dalle Matrici di Incidenza alle Matrici di Conteggio 65
4.5 Ponderazione della Frequenza: Log-Frequency Weighting . . . 65
4.6 Document Frequency (df) e Inverse Document Frequency (idf) 67
4.7 Lo Schema di Ponderazione TF-IDF Completo 69

5 Il Modello a Spazio Vettoriale (VSM), la Notazione SMART e Okapi BM25 **72**
5.1 Fondamenti del Vector Space Model (VSM) 72
5.2 Perché la Distanza Euclidea Fallisce: L'Esperimento di Duplicazione 73
5.3 Dagli Angoli alla Cosine Similarity e Normalizzazione L2 . . . 74
5.4 Esercizio Pratico Completo: Confronto Stilistico tra Tre Romanzi 76
5.5 Varianti di Ponderazione e la Notazione SMART 78
5.6 Domande di Verifica e Concetti Chiave 80
5.7 Best Match 25 (Okapi BM25) 81
5.8 Mappa Concettuale Riassuntiva 83

6 Architettura dei Motori di Ricerca e Indicizzazione **85**
6.1 Architettura di Sistema e Paradigma a Due Stadi 85
6.2 Modelli di Scoring: Prodotto Scalare e Connessione con il Dense Retrieval 87
6.3 Strutture Dati ad Alte Prestazioni: Forward Index vs. Indice Invertito 88
6.4 Anatomia, Layout di Memoria e Compressione dell'Indice Invertito 90
6.5 Tipologie di Query, Indici Posizionali e Riscrittura del Testo . 92

7 Query Processing: Algoritmi di Scansione ed Efficienza di Salto **95**

7.1 Anatomia Statistica dell'Indice Inverso: Dizionario e Posting Lists . . . 95
7.2 Paradigmi di Scansione: TAAT vs. DAAT . . . 98
7.3 Elaborazione Term-at-a-Time (TAAT) in Dettaglio . . . 103
7.4 DAAT per Query Disgiuntive (OR) e Congiuntive (AND Base) 105
7.5 La Primitiva di Salto 'nextGEQ' e Strutture di Skipping . . . 107
7.6 Algoritmi di Intersezione Avanzati con 'nextGEQ' . . . 110
7.7 Rilevanza nei Motori Vettoriali e Sistemi Moderni (Vector Search) 114
7.8 Quadro Sinottico Comparativo degli Algoritmi . . . 114

8 Elaborazione Efficiente di Query Top-k: WAND, MaxScore e Varianti a Blocchi 116
8.1 Architettura di Retrieval a Due Stadi e Motivazione del Top-k 116
8.2 Exact Top-k Retrieval e l'Algoritmo Esaustivo con Min-Heap . 118
8.3 L'Algoritmo WAND (Weak AND) e il Dynamic Pruning . . . 122
8.4 Analisi Critica e Casi Limite nel Dynamic Pruning . . . 128
8.5 L'Algoritmo MaxScore: Partizionamento in Liste Essenziali e Non-Essenziali . . . 130
8.6 Valutazione Sperimentale e Varianti a Blocchi: BMW e VBMW 135
8.7 Sistemi Industriali: L'Ecosistema Apache Lucene ed Elasticsearch 139
8.8 Quadro Sinottico Comparativo degli Algoritmi Top-k . . . 141

Bibliografia 142

Capitolo 0

Introduzione e Architettura dell'Information Retrieval

0.1 Definizione e Motivazioni

L'**Information Retrieval** (IR, *Reperimento delle Informazioni*) è la disciplina scientifica e ingegneristica che studia la rappresentazione, la memorizzazione, l'organizzazione e l'accesso a collezioni di dati prevalentemente non strutturati o semi-strutturati, al fine di soddisfare un **bisogno informativo** (*information need*) espresso da un utente [1, 2].

Definizione 0.1.1 (Information Retrieval (Manning et al.))**.** L'Information Retrieval consiste nel reperire materiale di natura prevalentemente non strutturata (solitamente documenti testuali) che soddisfa un bisogno informativo all'interno di collezioni di grandi dimensioni memorizzate su calcolatori elettronici.

L'IR non si limita ai soli motori di ricerca Web generalisti (Google, Bing), ma costituisce l'infrastruttura abilitante di e-commerce e marketplace, enterprise/desktop search, sistemi **RAG** (*Retrieval-Augmented Generation*, che alimentano i Large Language Model con documenti esterni contestualmente rilevanti) e motori di raccomandazione.

0.1.1 Dai Motori Web all'Agentic Search

L'avvento dei modelli linguistici conversazionali (LLM) non ha reso obsoleto l'Information Retrieval, ma ne ha ampliato l'impatto: un LLM moderno si appoggia costantemente ai principi cardine dell'IR (comprensione semantica della query, matching e ranking, reperimento contestuale) per superare il

limite del *knowledge cutoff*. Nel paradigma **RAG**, il sistema intercetta la richiesta dell'utente, interroga un motore IR, inietta i documenti estratti nel contesto dell'LLM e genera una risposta supportata da citazioni verificabili.

Definizione 0.1.2 (Agentic Search)**.** L'**Agentic Search** è un processo di reperimento dinamico in cui un agente intelligente determina in modo autonomo quali ricerche successive condurre, basandosi sulle evidenze parziali raccolte: ispeziona le fonti, individua lacune conoscitive (*information gaps*), formula nuove query di raffinamento e reitera il ciclo fino alla convergenza informativa.

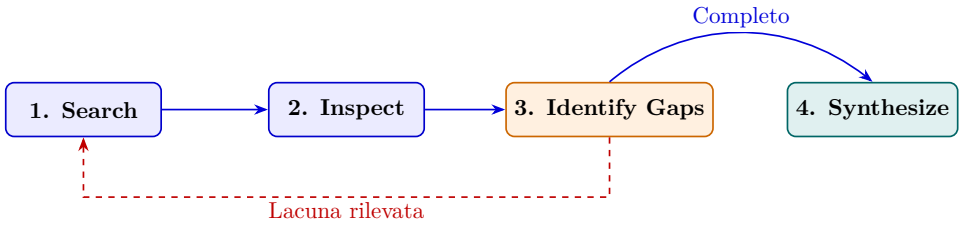


Figura 1: Ciclo a retroazione adattivo (*Adaptive Feedback Loop*) dell'Agentic Search.

0.1.2 Information Retrieval vs. Data Retrieval (Basi di Dati)

Dimensione	**Data Retrieval (Database SQL)**	**Information Retrieval**
Struttura dati	Rigidamente schematizzata (tabelle, tipi, vincoli)	Non strutturata o semi-strutturata (testo libero, HTML)
Linguaggio di query	Formale e deterministico (SQL)	Naturale o a parole chiave, spesso ambiguo
Matching	Booleano esatto	Parziale e probabilistico (gradi di rilevanza)
Ordinamento	Nessuno se non esplicito (`ORDER BY`)	Fondamentale, basato sulla stima di rilevanza
Tolleranza all'errore	Nulla	Elevata (refusi, sinonimia, polisemia)

Tabella 1: Confronto tra Data Retrieval tradizionale e Information Retrieval.

La Rilevanza: Il Cardine dell'IR

Mentre in una base di dati un record soddisfa o viola in modo booleano un predicato logico, nell'Information Retrieval il fulcro è la stima del grado di **rilevanza** rispetto al bisogno informativo dell'utente: soggettiva, situata nel contesto operativo ed evolutiva nel tempo.

0.2 I Due Macro-Obiettivi: Efficacia ed Efficienza

Un sistema di reperimento su larga scala persegue simultaneamente due obiettivi spesso in tensione:

1. **Efficacia (*Effectiveness*):** quanto accuratamente il ranking soddisfa il bisogno informativo dell'utente.
2. **Efficienza (*Efficiency*):** costo computazionale, throughput e latenza di risposta, tipicamente nell'ordine dei 10–50 millisecondi per query su archivi di miliardi di documenti.

L'efficienza di un motore di ricerca moderno dipende dall'interazione con l'architettura hardware sottostante: pipelining e processori superscalari, gerarchia di memoria e località di accesso, vettorizzazione SIMD e calcolo parallelo/distribuito su cluster (*sharding*).

0.3 Architettura di un Sistema di Information Retrieval

Un sistema IR adotta una separazione netta tra la **Pipeline Offline** (indicizzazione e addestramento dei modelli di ranking) e la **Pipeline Online** (interrogazione a bassa latenza), come illustrato in Figura 2.

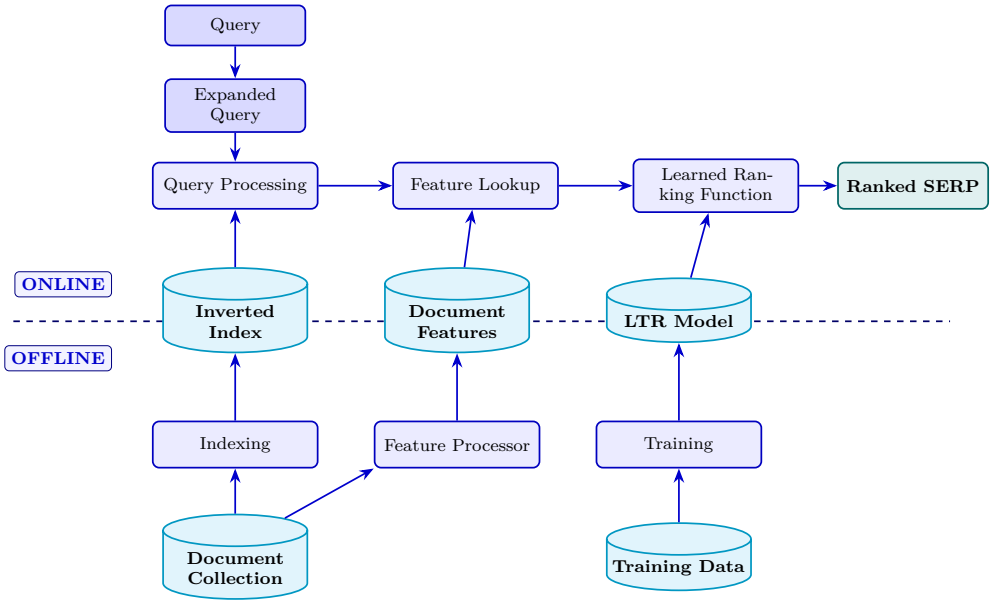


Figura 2: Architettura a griglia di un moderno motore di ricerca (pipeline Online vs. Offline).

0.3.1 Pipeline di Text Processing

Prima dell'indicizzazione, ogni flusso testuale attraversa: **tokenizzazione** (segmentazione in token lessicali), **normalizzazione/case folding** (minuscolo, diacritici), **filtraggio delle stop word** (rimozione di vocaboli ad alta frequenza ma privi di valore discriminante) e **stemming/lemmatizzazione** (riduzione di una forma flessa alla radice o al lemma canonico, es. `"searching"` $\rightarrow$ `"search"`).

0.4 Proprietà del Linguaggio e Modelli di Reperimento

- **Legge di Zipf:** la frequenza $f(r)$ di un termine è inversamente proporzionale al suo rango r nella graduatoria di frequenza, $f(r) \propto 1/r$: poche parole frequentissime occupano la maggioranza delle occorrenze, mentre la maggior parte dei termini compare pochissime volte (*long tail*).
- **Legge di Heaps:** la dimensione del vocabolario cresce come legge di potenza rispetto al numero totale di parole N della collezione, $|V| = k \cdot N^{\beta}$ (con $0.4 \leq \beta \leq 0.6$): il vocabolario non si satura mai completamente.

I principali modelli di reperimento sono il **Modello Booleano** (match esatto tramite operatori logici, privo di punteggio di rilevanza), il **Vector Space Model** (query e documenti come vettori pesati con TF-IDF, rilevanza tramite similarità del coseno) e il **Modello Probabilistico BM25**, standard de facto del reperimento lessicale:

$$\text{score}_{\text{BM25}}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t,d) \cdot (k_1+1)}{tf(t,d) + k_1 \cdot \left(1 - b + b \cdot \dfrac{|d|}{\text{avgdl}}\right)} \tag{1}$$

dove $\text{IDF}(t) = \ln\left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1\right)$ pesa la specificità del termine, k_1 (tipicamente 1.2–2.0) controlla la saturazione del term frequency e $b \in [0,1]$ (tipicamente ≈ 0.75) normalizza la lunghezza del documento $|d|$ rispetto alla lunghezza media avgdl.

0.5 L'Inverted Index (Indice Invertito)

L'**Inverted Index** inverte l'orientamento logico della memorizzazione documentale, realizzando una mappatura $t \mapsto \mathcal{P}_t$, dove $\mathcal{P}_t$ è la lista ordinata dei documenti contenenti il termine t [1].

Definizione 0.5.1 (Inverted Index)**.** Un Inverted Index è costituito da un **dizionario** (l'insieme ordinato dei termini unici, con frequenza documentale df_t e puntatore alla posting list) e, per ogni termine, da una **posting list**: un array ordinato per DocID crescente, i cui elementi possono includere metadati come frequenza locale (tf) e offset posizionali.

Esempio 0.5.1 (Collezione Giocattolo di Riferimento)**.** Si consideri una piccola collezione di tre documenti sull'indicizzazione e la ricerca testuale. La Figura 3 illustra l'Inverted Index generato per alcuni termini chiave: il termine `a` compare nei documenti $1, 2, 3$; `an` solo nel documento 1; `index` e `inverted` nei documenti $1, 2$; `query` solo nel documento 3.

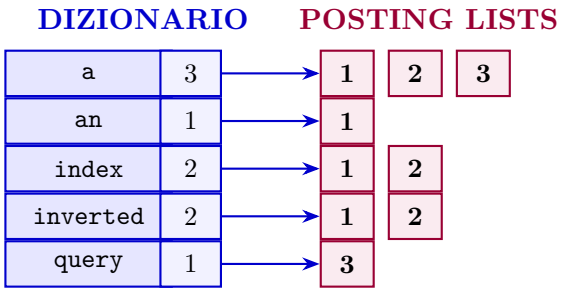


Figura 3: Struttura dell'Inverted Index: dizionario e posting list.

I costrutti booleani (AND, OR, NOT) e la ricerca per frase trovano diretta corrispondenza nell'interfaccia di ricerca avanzata dei motori commerciali (Tabella 2).

Operatore Logico IR	**Campo Google Advanced Search**	**Sintassi**
Congiunzione (AND)	*all these words*	`tri-colour rat terrier`
Ricerca per frase	*this exact word or phrase*	`"rat terrier"`
Disgiunzione (OR)	*any of these words*	`miniature OR standard`
Negazione (NOT)	*none of these words*	`-rodent -"Jack Russell"`
Intervallo numerico	*numbers ranging from...to*	`10..35 kg`

Tabella 2: Mappatura tra operatori booleani IR e Ricerca Avanzata di Google.

0.6 Tecniche di Compressione dell'Indice

La compressione dell'Inverted Index riduce drasticamente l'occupazione di memoria (sia su RAM che su disco), convertendo un collo di bottiglia di I/O in un carico di decompressione CPU, generalmente molto più veloce.

0.6.1 Binary Interpolative Coding

Il **Binary Interpolative Coding** (Moffat e Stuiver [3]) comprime sequenze strettamente crescenti di interi (i DocID delle posting list) partizionandole ricorsivamente: data una sottolista di n interi nota per essere compresa in $[l, h]$, si codifica l'elemento centrale x_m (con $m = \lfloor (n+1)/2 \rfloor$) usando $\lceil \log_2 R \rceil$ bit, dove $R = h - l - n + 1$ è il numero di valori ammissibili, e si ricorre poi separatamente sulle due metà. Quando i DocID sono densamente clusterizzati, R tende a 1 e molti elementi vengono codificati a costo pressoché nullo.

0.6.2 Compressione Basata su Dizionario: LZ77

Per testi generali, l'algoritmo **Lempel-Ziv '77** (LZ77) comprime tramite una finestra scorrevole, sostituendo ripetizioni lessicali con triple costituite da $(\text{offset}, \text{lunghezza}, \text{carattere})$ riferite a occorrenze precedenti nella finestra. Questa famiglia di algoritmi è alla base di strumenti diffusi come `gzip`, `7-Zip` e `LZ4`.

0.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità

I candidati selezionati dall'Inverted Index vengono inoltrati a un modello di Machine Learning che li riordina (**Learning-to-Rank**, LTR) [4]. Il dataset di addestramento associa a ciascuna query una lista di documenti con giudizio di rilevanza graduato $y \in \{0, \ldots, 4\}$ (da irrilevante a perfettamente rilevante); un modello calcola uno score $s_i = f_\theta(\vec{x}_{q,d_i})$ per ciascuna coppia query-documento, e la lista finale viene ottenuta ordinando per punteggio decrescente.

Il Cardine dell'LTR

The goal is to learn the ranking, not the label: l'obiettivo non è stimare accuratamente il singolo valore y, ma l'ordinamento relativo ottimale, massimizzando metriche cumulative come **NDCG** o **MAP**.

Poiché l'operazione di ordinamento è discreta e a gradini, il gradiente delle metriche di ranking rispetto ai parametri del modello è nullo quasi ovunque: non è possibile applicare direttamente la discesa del gradiente. La letteratura ha sviluppato tre famiglie di soluzioni:

1. **Pointwise:** tratta ogni documento come un'istanza indipendente (regressione o classificazione su y), ignorando la natura comparativa del ranking.

2. **Pairwise** (es. RankNet): confronta coppie di documenti, modellando con una sigmoide differenziabile la probabilità che d_i preceda d_j.

3. **Listwise** (es. LambdaMART): ottimizza l'intera lista tramite gradienti surrogati λ_{ij}, pesati con la variazione effettiva di NDCG derivante dallo scambio di d_i e d_j; combinato con alberi di decisione potenziati, è stato per oltre un decennio lo standard nei motori di ricerca commerciali.

0.8 Evoluzione verso il Neural Information Retrieval

Dal 2018, i modelli Transformer (BERT) hanno inaugurato l'era del Neural Information Retrieval [5, 6], pre-addestrati tramite **Masked Language Modeling**. Gli approcci neurali per il reperimento si dividono in due famiglie:

- **Interazione (Cross-Encoder):** query e documento sono concatenati ed elaborati congiuntamente da un Transformer, catturando interazioni

profonde a costo computazionale elevato ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$); usati solo come re-ranker su un piccolo insieme di candidati.

- **Rappresentazione (Bi-Encoder):** query e documento sono proiettati indipendentemente in uno spazio vettoriale da due encoder distinti; i vettori dei documenti sono pre-calcolabili offline, riducendo il matching online a un prodotto scalare o a una ricerca di vicini approssimati.

Tra i modelli a rappresentazione si distinguono tre famiglie di embedding: **dense single-vector** (un unico vettore denso per documento/query, interrogato con indici k-ANN su grafi come HNSW), **dense multi-vector** (es. ColBERT [5], un vettore per token con aggregazione *Late Interaction*/MaxSim, $S(q,d) = \sum_{t_q \in q} \max_{t_d \in d}(\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T)$) e **sparse single-vector** (es. SPLADE [6], interpretabile per design e più efficiente in spazio/tempo rispetto al multi-vector).

k-Approximate Nearest Neighbor Search

Su collezioni di milioni di documenti rappresentati come vettori densi ad alta dimensionalità, il recupero esatto dei k più simili alla query è computazionalmente proibitivo: si ricorre quindi a indici di **ricerca approssimata dei vicini** (k-ANN), che scambiano una minima perdita di accuratezza per un guadagno di ordini di grandezza in velocità. **TusKANNy** (`https://tuskanny.com/`), sviluppato da ISTI-CNR e Università di Pisa, ne è un esempio open-source ad alte prestazioni scritto in Rust.

Capitolo 1

Valutazione dei Sistemi di Information Retrieval

1.1 Architettura di un Motore di Ricerca

Prima di affrontare la valutazione, è utile inquadrare l'architettura complessiva di un moderno motore di ricerca, che può essere schematizzata nel seguente diagramma a blocchi:

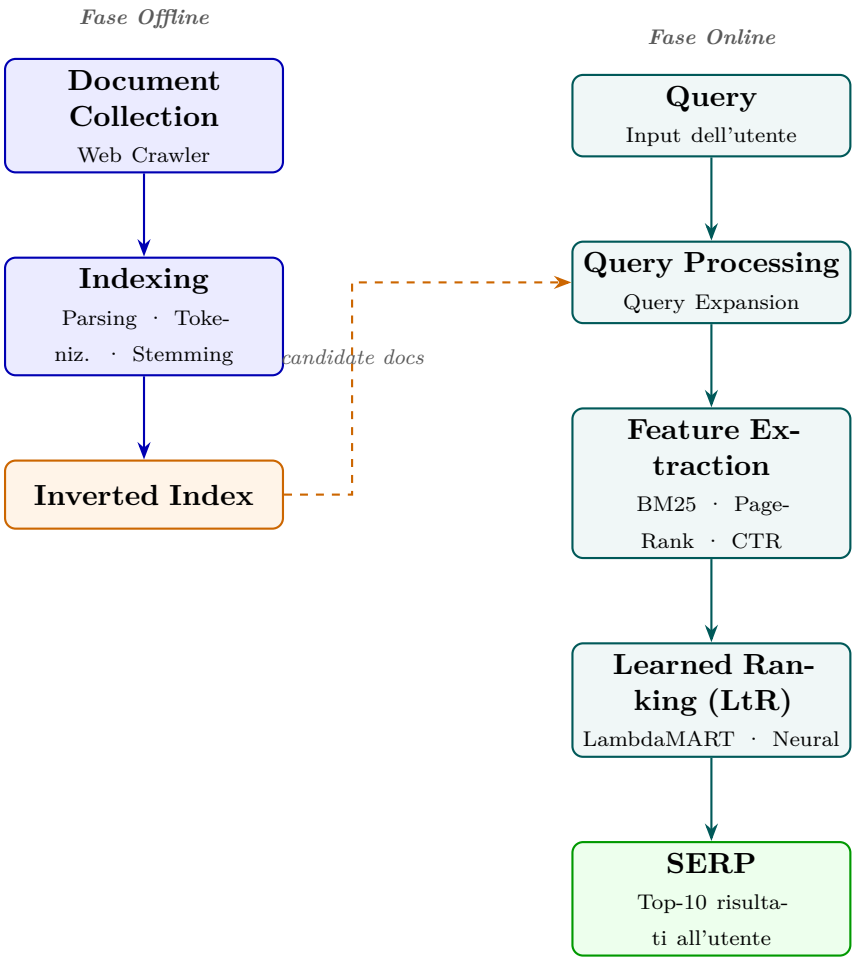


Figura 1.1: Diagramma a blocchi dell'architettura di un motore di ricerca moderno: pipeline offline (indicizzazione) e pipeline online (elaborazione query e ranking).

I componenti principali sono:

- **Web Crawler (Document Collection):** Un agente automatico che percorre il Web seguendo i collegamenti ipertestuali e scarica i documenti da indicizzare. La collezione risultante è la base di lavoro del sistema.
- **Indexing:** La pipeline di pre-elaborazione che trasforma i documenti grezzi in una rappresentazione indicizzabile: parsing HTML, tokenizzazione, normalizzazione, rimozione delle stopword e stemming/lemmatizzazione.

- **Inverted Index:** La struttura dati centrale del motore di ricerca: per ogni termine mappa la lista dei documenti che lo contengono (posting list), consentendo un recupero efficiente.
- **Query Processing e Query Expansion:** Il modulo che analizza la query dell'utente, la normalizza e — facoltativamente — la espande con sinonimi o termini correlati per aumentare il richiamo.
- **Feature Extraction:** Estrazione di segnali numerici per ciascuna coppia query-documento (TF-IDF, BM25, PageRank, freschezza, segnali di click, ecc.).
- **Learned Ranking Function (Learning to Rank):** Un modello statistico o di apprendimento automatico (ad es. LambdaMART, LambdaRank) addestrato a ordinare i documenti massimizzando una metrica di qualità, tipicamente NDCG.
- **SERP (Search Engine Results Page):** La pagina dei risultati presentata all'utente, comprensiva di titolo, snippet, URL e, nei motori moderni, risposte generate (Featured Snippets, AI Overviews).

1.2 Come Valutare la Qualità di un Motore di Ricerca

Un sistema di Information Retrieval è un artefatto complesso che integra decine di componenti eterogenei: come si valuta qualcosa di così articolato? La risposta della comunità scientifica distingue nettamente due dimensioni complementari:

1. **Metriche di Efficienza:**
    - *Velocità di indicizzazione:* Quanti documenti per ora il sistema è in grado di processare e inserire nell'indice.
    - *Indicizzazione incrementale:* Capacità di aggiornare l'indice con nuovi flussi documentali (ad es. +10 000 prodotti al giorno in un e-commerce) senza interruzioni di servizio.
    - *Velocità di ricerca (Latenza e Throughput):* Tempo di risposta (latenza al 95° o 99° percentile) e consumo di CPU/memoria nell'elaborazione di migliaia di query al secondo su collezioni di svariati milioni o miliardi di documenti.

2. **Metriche di Efficacia e Qualità dei Servizi Accessori:**

    - Qualità delle raccomandazioni correlate, accuratezza della correzione ortografica delle query (*spell checking*) e chiarezza dei riassunti testuali visualizzati (*snippet generation*).
    - **Efficacia intrinseca della graduatoria:** Misura la capacità fondamentale del motore di ricerca di soddisfare l'utente, ordinando i documenti pertinenti nelle prime posizioni della pagina dei risultati (*Search Engine Results Page*, SERP).

1.3 Misurare la Soddisfazione dell'Utente

Nel contesto operativo dei motori di ricerca commerciali, l'obiettivo ultimo è massimizzare la soddisfazione dell'utente (*User Happiness*). Poiché la felicità è uno stato cognitivo non osservabile direttamente, i sistemi di produzione ricorrono a proxy comportamentali online:

- **Clic e Click-Through Rate (CTR):** Il numero e la proporzione di clic ricevuti dai risultati restituiti.
    - *Criticità:* Titoli sensazionalistici o riassunti ingannevoli (*clickbait*) inducono clic elevati senza che il documento soddisfi realmente l'utente.
    - *La ricerca a zero clic (Zero-Click Search):* L'assenza di clic non è necessariamente un segnale negativo. Quando un utente cerca informazioni fattuali (ad es. il meteo, l'orario di un volo, una conversione di valuta o una definizione enciclopedica), la risposta visualizzata direttamente tramite *Featured Snippet* o *Knowledge Graph* soddisfa istantaneamente il bisogno senza richiedere alcun clic.
- **Conversioni e Transazioni Economiche:** Negli e-commerce, la quota di sessioni di ricerca che sfociano nell'acquisto di un prodotto e l'ammontare monetario speso dall'utente costituiscono metriche primarie di business.
- **Fidelizzazione e Visite Ripetute:** La percentuale di utenti che tornano ad adoperare il motore di ricerca entro un intervallo temporale predefinito (una settimana, un mese).

- **Dwell Time (Tempo di Permanenza):** L'intervallo di tempo che intercorre tra il momento in cui l'utente clicca su un risultato della SERP e l'istante in cui fa ritorno alla pagina dei risultati (*bouncing back*). Un Dwell Time prolungato indica generalmente che il documento ha catturato l'attenzione ed è risultato informativo, mentre un ritorno immediato alla SERP denota insoddisfazione.

La Valutazione nell'Era dei Motori Generativi e LLM

L'emergere di motori di ricerca generativi e assistenti basati su Large Language Models (LLM) ridefinisce radicalmente metriche consolidate come Dwell Time e CTR: l'utente non naviga più necessariamente attraverso una lista di link esterni, ma consuma sintesi testuali generate direttamente dal modello. In tale scenario, l'accuratezza fattuale, l'assenza di allucinazioni e la corretta attribuzione delle fonti rimpiazzano il conteggio dei clic tradizionali.
Il dibattito metodologico su come valutare questi sistemi solleva interrogativi cruciali:

- **Numero di turni conversazionali come proxy di insoddisfazione:** In un sistema conversazionale, un elevato numero di turni necessari per soddisfare un bisogno informativo può segnalare che il sistema non ha compreso l'intento iniziale dell'utente, costringendolo a continue riformulazioni. Il Dwell Time perde quindi centralità, sostituito dall'analisi dell'efficienza e della lunghezza della sessione di dialogo.

- **Varianza e instabilità del prompting:** A differenza delle query lessicali dell'IR classico, lievi variazioni nella formulazione del prompt possono indurre forti discrepanze qualitative nella risposta dell'LLM. Questa elevata sensibilità rende complessa la standardizzazione e la riproducibilità sperimentale dei benchmark.

- **Differenza ontologica tra recupero e sintesi:** Un motore tradizionale si limita a *recuperare* e ordinare documenti preesistenti da un corpus noto; un modello generativo *sintetizza* invece un testo originale probabilistico. Mentre nel primo caso la rilevanza è misurabile rispetto a un insieme oggettivo di documenti rilevanti (*ground truth*), nel secondo la correttezza fattuale della sintesi richiede verifiche semantiche inedite.

- **Il paradigma RAG (Retrieval-Augmented Generation):** L'architettura RAG vincola la generazione alla consultazione preventiva di documenti estratti da un indice IR tradizionale. In questo contesto, la valutazione diventa composita: occorre misurare separatamente e congiuntamente l'efficacia del recupero documentale e la fedeltà informativa del testo generato.

1.4 Il Paradigma di Cranfield e le Collezioni Benchmark

Sebbene i segnali online forniscano un feedback prezioso, essi risultano rumorosi, soggetti a forti bias di posizionamento (*Position Bias*) e non riproducibili sperimentalmente prima del rilascio in produzione. Per consentire valutazioni comparative controllate, la ricerca in IR adotta il **Paradigma di Cranfield**, sviluppato originariamente da Cyril Cleverdon negli anni '60 [7].

È importante contestualizzare il momento storico: Cleverdon lavorava su **biblioteche fisiche**, con schedari cartacei e cataloghi aeronautici. Il suo esperimento — condotto prima dell'avvento dei calcolatori elettronici diffusi — consisteva nel valutare manualmente quanto bene diversi sistemi di classificazione documentale riuscissero a soddisfare bisogni informativi reali. Eppure, il framework metodologico che ne derivò è rimasto il fondamento della valutazione in IR per oltre sessant'anni.

Definizione 1.4.1 (Collezione di Test (Test Collection))**.** Una collezione di test standard per l'Information Retrieval è una tripletta formale $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ costituita da:

1. Una collezione documentale benchmark $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$;
2. Una suite benchmark di bisogni informativi e query $\mathcal{Q} = \{q_1, q_2, \ldots, q_M\}$;
3. Un insieme di giudizi di rilevanza (*Relevance Judgments* o *qrels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, che associa a ciascuna coppia query-documento un valore di rilevanza (binario o graduato).

1.4.1 La Sfida della Scalabilità Combinatoria

La realizzazione manuale esaustiva dei giudizi di rilevanza è materialmente impossibile per archivi realistici. Si consideri uno scenario benchmark con:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documenti}, \quad |\mathcal{Q}| = 50\,000 \text{ query}$$

Lo spazio complessivo delle coppie ammonta a:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 5 \times 10^4 = 2.5 \times 10^{11} \text{ coppie query-documento}$$

Ipotizzando che un annotatore umano impieghi in media 2.5 secondi per valutare la rilevanza di un documento:

$$\begin{aligned} \text{Tempo Totale} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ secondi} \\ &\approx 173\,611\,111 \text{ ore/persona} \end{aligned}$$

Un impegno titanico che richiederebbe decine di migliaia di anni di lavoro umano ininterrotto! Per ovviare a ciò, i benchmark limitano i giudizi a un sottoinsieme accuratamente campionato di documenti.

1.4.2 Origini delle Query di Test

Una questione metodologica fondamentale riguarda la provenienza delle query che compongono il benchmark $\mathcal{Q}$:

- **Motori Web commerciali — Query Logs:** I grandi motori di ricerca (Google, Microsoft Bing) campionano le query di test direttamente dai propri *query log*, cioè dai log di ricerca reali degli utenti. Dataset pubblici storici come l'AOL Query Log (2006) e i dataset Microsoft ORCAS/MIMICS contengono milioni di query reali. Questo approccio garantisce che le query riflettano bisogni informativi autentici con la distribuzione di frequenza reale (incluse le query rare della coda lunga).
- **Scenario classico — Esperti a tavolino:** Nelle prime collezioni (Cranfield, TREC), esperti di dominio stilavano manualmente liste di *information needs* plausibili, simulando i bisogni di un ipotetico utente. Il processo era deliberato e supervisionato, con attenzione a coprire diversi tipi di bisogno informativo (noto, esplorativo, transazionale).
- **Perché non generare query casuali dai documenti:** Una strategia apparentemente ovvia — campionare termini o frasi dai documenti della collezione per costruire query sintetiche — è metodologicamente scorretta. Le query così generate non rifletterebbero gli intenti reali degli utenti, ma solo il vocabolario dei documenti, producendo una valutazione distorta e priva di validità ecologica.

1.4.3 Crowdsourcing: Opportunità e Limiti

Una strategia economica consiste nel delegare la valutazione a lavoratori retribuiti su piattaforme online di crowdsourcing (come Amazon Mechanical Turk). Tuttavia, la letteratura scientifica evidenzia come tali giudizi siano affetti da:

- Elevatissima varianza statistica;
- Minore competenza e presenza di spammer;
- Necessità di filtri rigorosi di consenso (ad es. votazione di maggioranza o calcolo dell'accordo inter-annotatore tramite coefficiente κ di Cohen).

Una strategia più recente e sempre più diffusa è l'impiego di **LLM come Valutatori (*LLM-as-a-Judge*)**:

- Modelli come GPT-4 vengono utilizzati con prompt strutturati per valutare la rilevanza di coppie query-documento, in sostituzione o affiancamento agli annotatori umani. La scalabilità economica è enormemente superiore al crowdsourcing tradizionale: il costo per giudizio scende di uno o due ordini di grandezza.
- Tuttavia, questa metodologia presenta **bias intrinseci** documentati: preferenza per testi più lunghi (indipendentemente dalla qualità), *self-preference bias* (i modelli tendono a valutare meglio testi simili ai propri output di addestramento) e sensibilità all'ordine di presentazione dei candidati. È quindi necessario progettare con cura i prompt di valutazione e condurre studi di calibrazione rispetto ai giudizi umani.

1.4.4 L'Infrastruttura TREC (*Text REtrieval Conference*)

Promossa dal National Institute of Standards and Technology (NIST), la conferenza **TREC** rappresenta il punto di riferimento internazionale per la valutazione delle tecnologie di recupero dell'informazione. TREC è strutturata in tracce tematiche (*Tracks*), ciascuna incentrata su specifici casi d'uso (ricerca Web, ricerca clinica, domande-risposte, deep retrieval). Il risultato prodotto da un sistema software che esegue un compito su una collezione di test prende il nome di **Run**.

La Tabella 1.1 documenta l'evoluzione dimensionale delle collezioni benchmark storiche utilizzate nella ricerca in Information Retrieval [1], evidenziando il salto di scala che ha reso indispensabile la nascita di TREC.

Collezione	**Documenti (N)**	**Query**	**Dimensione (MB)**	**Termini/Doc**	**Giudizi (Q-D)**
ADI	82	35	—	—	—
AIT	2 109	14	2	400.0	$> 10\,000$
CACM	3 204	64	2	24.5	—
CISI	1 460	112	2	46.5	—
Cranfield	1 400	225	2	53.1	Completi
LISA	5 872	35	3	—	—
Medline	1 033	30	1	—	—
NPL	11 429	93	3	—	—
OSHMED	348 566	106	400	250.0	16 140
Reuters	21 578	672	28	131.0	—
TREC (Tipica)	**740 000**	**200**	**2 000 (2 GB)**	**89–3 543**	$> 100\,000$

Tabella 1.1: Panoramica comparativa dei principali corpora benchmark storici di valutazione.

Come si evince dai dati, nelle prime collezioni degli anni '60 e '70 (come Cranfield e Medline), le ridotte dimensioni (poche migliaia di record e 1–2 MB) consentivano un'annotazione esaustiva manuale. Con l'avvento di collezioni realistiche da centinaia di migliaia di documenti e gigabyte di testo come TREC, lo spazio combinatorio delle valutazioni è esploso, imponendo l'adozione di metodologie di campionamento selettivo.

1.4.5 La Tecnica del Pooling

Per campionare i documenti da sottoporre ad annotazione umana senza dover valutare l'intera collezione, nei benchmark TREC si adotta la tecnica del **Pooling** (Figura 1.2):

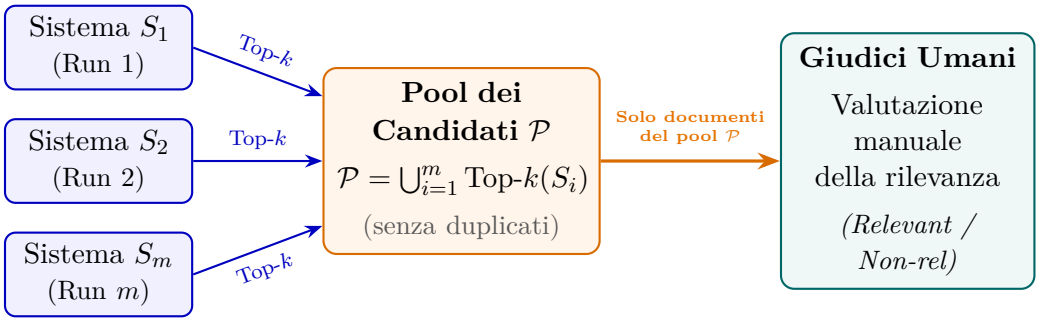


Figura 1.2: Processo di Pooling per la selezione controllata dei documenti da annotare.

1. Si raccolgono le risposte fornite da m sistemi di recupero eterogenei per una determinata query.
2. Si estraggono i primi k documenti restituiti da ciascun sistema (tipicamente $k = 50$ o $k = 100$).
3. Si calcola l'**unione** di tali insiemi eliminando i duplicati, formando il pool $\mathcal{P}$.
4. Gli annotatori umani valutano la rilevanza unicamente per i documenti appartenenti a $\mathcal{P}$.

L'Assunzione Fondamentale del Pooling

Tutti i documenti dell'archivio che non compaiono nel pool (*unpooled*) vengono assunti convenzionalmente come **non rilevanti** ($r = 0$). Sebbene tale assunzione introduca un potenziale bias a sfavore di algoritmi futuri con paradigmi radicalmente differenti, studi empirici condotti da NIST confermano che, a fronte di un pool formato da sistemi sufficientemente diversificati, le conclusioni comparative tra motori di ricerca rimangono stabili e affidabili.

1.5 Bisogno Informativo vs. Query

Una distinzione concettuale decisiva riguarda il divario tra bisogno informativo e formulazione della query:

- **Bisogno Informativo (*Information Need*):** Rappresenta lo stato cognitivo e l'obiettivo reale dell'utente. Ad esempio: *"Il fondo della mia piscina sta diventando nero e necessita di essere ripulito"*.
- **Query:** L'espressione testuale sintetica digitata nel box di ricerca. Ad esempio: `"pulitore piscina"`.

Proprietà 1.5.1 (Principio di Valutazione della Rilevanza)**.** *La rilevanza di un documento deve essere sempre stimata rispetto all'**Information Need** sottostante, mai rispetto alla mera sequenza di caratteri che compone la query.*

1.6 Metriche di Valutazione Non Ordinate (*Unranked Measures*)

Nel contesto del reperimento non ordinato (come nel modello booleano classico), il sistema restituisce un insieme non graduato di documenti. La valutazione si fonda sulla suddivisione dell'universo documentale mediante la matrice di contingenza (Tabella 1.2):

	Documento Rilevante	**Documento Non Rilevante**
Documento Reperito	True Positive (TP)	False Positive (FP)
Documento Non Reperito	False Negative (FN)	True Negative (TN)

Tabella 1.2: Matrice di contingenza per la classificazione binaria query-documento.

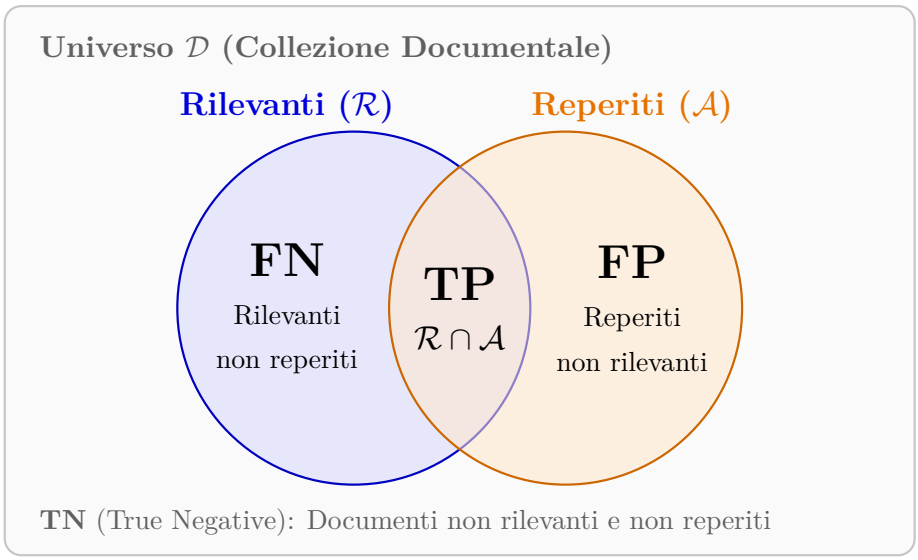


Figura 1.3: Rappresentazione insiemistica del reperimento documentale e delle regioni della matrice di confusione.

1.6.1 Precisione, Richiamo e loro Trade-off

Definizione 1.6.1 (Precisione (*Precision*, P))**.** La frazione dei documenti restituiti dal sistema che risultano effettivamente rilevanti:

$$P = \frac{TP}{TP + FP} = P(\text{rilevante} \mid \text{reperito}) \tag{1.1}$$

Definizione 1.6.2 (Richiamo (*Recall*, R))**.** La frazione dei documenti rilevanti complessivamente presenti nella collezione che il sistema è riuscito a reperire:

$$R = \frac{TP}{TP + FN} = P(\text{reperito} \mid \text{rilevante}) \tag{1.2}$$

Esiste un intrinseco **trade-off** tra Precisione e Richiamo:

- Un sistema che restituisce l'intera collezione documentale (Retrieved = $\mathcal{D}$) ottiene banalmente un richiamo perfetto $R = 100\%$, ma la sua precisione crolla a livelli minimi ($P \approx 0$).
- Viceversa, un sistema conservativo che restituisce unicamente il singolo documento più sicuro può conseguire $P = 100\%$, pagando il prezzo di un richiamo trascurabile.

1.6.2 La Media Armonica: F-Measure (F_1-Score)

Per sintetizzare Precisione e Richiamo in un unico valore scalare che premi l'equilibrio tra le due grandezze, si adotta la **F-Measure** (o F_1-score), definita come la **media armonica** di P ed R:

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{1.3}$$

Proprietà 1.6.1 (Proprietà della Media Armonica)**.** *La media armonica gode di proprietà matematiche fondamentali per la valutazione:*

1. *È sempre minore o uguale alla media geometrica e alla media aritmetica:*

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a+b}{2}$$

2. *Quando i due argomenti differiscono notevolmente, la media armonica collassa verso il* ***valore minimo*** *anziché posizionarsi a metà strada.*

Esempio 1.6.1 (Comportamento Limite dell'F-Measure)**.** Si consideri un sistema banale che restituisce l'intera collezione: $R = 1.0$ e $P = 0.0001$.

- La media aritmetica assegna un punteggio fuorviante pari a:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- La media armonica (F_1) demolisce giustamente la valutazione:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

Ciò dimostra per quale motivo la media aritmetica sia inadeguata e l'F-measure costituisca la scelta corretta.

1.6.3 La Misura F_β Pesata

Qualora si desideri attribuire pesi differenziati a Precisione e Richiamo, si introduce un parametro di ponderazione $\alpha \in [0, 1]$:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \tag{1.4}$$

Esprimendo α in funzione di un parametro β tale che $\alpha = \frac{1}{1+\beta^2}$, si ricava la formulazione classica di F_β:

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \tag{1.5}$$

- Valori di $\beta < 1$ (ad es. $\beta = 0.5$) enfatizzano la **Precisione** rispetto al Richiamo (utile nelle ricerche su Web generico).
- Valori di $\beta > 1$ (ad es. $\beta = 2.0$) enfatizzano il **Richiamo** rispetto alla Precisione (fondamentale nelle ricerche brevettuali, legali e mediche, dove omettere un documento rilevante comporta danni gravi).

1.7 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria)

Nei motori di ricerca moderni i documenti non vengono restituiti come insiemi informi, ma ordinati in base a uno score di rilevanza decrescente. Inoltre, gli utenti Web raramente oltrepassano la prima pagina dei risultati (i primi 10 o 20 elementi).

1.7.1 Precision@k ($P@k$) e Recall@k ($R@k$)

Definizione 1.7.1 (Precision@k)**.** Fissata una soglia di rango $k \in \mathbb{N}^+$, la Precision@k corrisponde alla proporzione di documenti rilevanti presenti nei primi k posti della graduatoria:

$$P@k = \frac{1}{k} \sum_{i=1}^{k} \text{rel}_i \tag{1.6}$$

dove $\text{rel}_i \in \{0, 1\}$ denota la rilevanza binaria del documento al rango i.

Esempio 1.7.1 (Calcolo di Precision@k)**.** Si consideri una lista ordinata in cui la sequenza di rilevanza binaria dei primi 5 documenti è data da $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (si noti che $P@4 < P@3$)
- $P@5 = \frac{3}{5} = 0.60$

La Distorsione di $P@k$ nella Media tra Query

Sebbene $P@k$ sia intuitivo per stimare la qualità della prima pagina, presenta un difetto metodologico critico quando viene mediato su insiemi di query eterogenee:

- Se per una query specifica q_A esistono solo $R_A = 2$ documenti rilevanti nell'intera collezione, il valore massimo teoricamente raggiungibile per $P@10(q_A)$ è pari a $\frac{2}{10} = 0.20$, anche qualora il motore collochi perfettamente entrambi i documenti ai primi due ranghi!
- Calcolando la media aritmetica di $P@10$ tra query, il motore viene ingiustamente penalizzato dalla rarità intrinseca dei documenti rilevanti di alcune query, e non da un errore dell'algoritmo di ranking.

In modo analogo, la metrica complementare *Recall@k* ($R@k$) calcola la frazione dei documenti rilevanti complessivi reperiti entro il rango k:

$$Recall@k = \frac{1}{R} \sum_{i=1}^{k} \text{rel}_i \tag{1.7}$$

1.7.2 Average Precision (AP)

L'**Average Precision** (AP) sintetizza l'intera curva Precision-Recall in un singolo indicatore scalare per una specifica query, valutando la precisione istantanea in corrispondenza di ogni posizione in cui il richiamo aumenta.

Definizione 1.7.2 (Average Precision)**.** Dato un ordinamento di N documenti per una query con R documenti rilevanti totali presenti nella collezione, l'AP è definita come:

$$AP = \frac{1}{R} \sum_{k=1}^{N} P@k \cdot \text{rel}_k \tag{1.8}$$

Proprietà 1.7.1 (Interpretazione Geometrica e Ruolo di $1/R$)**.** *L'Average Precision possiede una chiara interpretazione analitica:*

1. *Ogni volta che si incontra un documento rilevante al rango k, il richiamo compie un gradino discreto di ampiezza $\Delta Recall = \frac{1}{R}$. L'AP calcola la somma delle aree dei rettangoli $P@k \cdot \Delta Recall$, rappresentando l'**approssimazione numerica dell'Area Sotto la Curva Precision-Recall** (AUC-PR).*

2. *La normalizzazione per R (il numero totale di documenti rilevanti nella collezione) garantisce che i documenti rilevanti non recuperati entro i primi N ranghi contribuiscano implicitamente con precisione pari a 0, penalizzando severamente i sistemi affetti da scarso richiamo.*

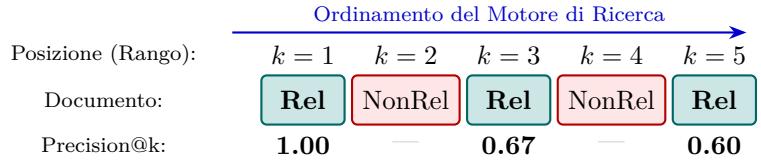


Figura 1.4: Rappresentazione schematica del calcolo di Average Precision su 5 documenti con $R = 3$ rilevanti totali.

Esempio 1.7.2 (Calcolo di Base con $R = 3$ Rilevanti)**.** Si consideri lo scenario di Figura 1.4 con $R = 3$ documenti rilevanti totali, posizionati ai ranghi $k_1 = 1$, $k_2 = 3$ e $k_3 = 5$:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

L'Average Precision risulta:

$$AP = \frac{1}{3}\left(1.00 + \frac{2}{3} + \frac{3}{5}\right) = \frac{1}{3}\,(2.267) \approx 0.756 \quad (75.6\%) \tag{1.9}$$

Esempio 1.7.3 (Confronto Comparativo tra Due Ordinamenti)**.** Si consideri una query per la quale esistono $R = 6$ documenti rilevanti nella collezione. Due motori di ricerca alternativi restituiscono i seguenti primi 10 documenti:

- **Ranking #1:** Rilevanti concentrati nelle prime posizioni:

  $$\langle \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{NonRel}, \text{Rel} \rangle$$

- **Ranking #2:** Rilevanti dispersi e ritardati:

  $$\langle \text{NonRel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel} \rangle$$

La Tabella 1.3 traccia l'evoluzione rango per rango di Precisione e Richiamo per entrambi i sistemi.

	Ranking #1			**Ranking #2**		
Rango (k)	**Esito**	**Recall**	**Precision**	**Esito**	**Recall**	**Precision**
1	**Rel**	0.17	**1.00**	NonRel	0.00	—
2	NonRel	0.17	—	**Rel**	0.17	**0.50**
3	**Rel**	0.33	**0.67**	NonRel	0.17	—
4	**Rel**	0.50	**0.75**	NonRel	0.17	—
5	**Rel**	0.67	**0.80**	**Rel**	0.33	**0.40**
6	**Rel**	0.83	**0.83**	**Rel**	0.50	**0.50**
7	NonRel	0.83	—	**Rel**	0.67	**0.57**
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	**Rel**	0.83	**0.56**
10	**Rel**	1.00	**0.60**	**Rel**	1.00	**0.60**
Average Precision (AP)			**0.78**			**0.52**

Tabella 1.3: Calcolo comparativo dettagliato di Average Precision tra Ranking #1 e Ranking #2.

I rispettivi valori di AP risultano:

$$AP(\text{Ranking \#1}) = \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78}$$

$$AP(\text{Ranking \#2}) = \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52}$$

Sebbene entrambi i motori raggiungano un richiamo finale identico ($R = 100\%$ sui 10 documenti), Ranking #1 ottiene un punteggio nettamente superiore (0.78 vs 0.52) poiché premia la densità di documenti rilevanti nella parte alta della graduatoria.

1.7.3 Mean Average Precision (MAP)

Quando si valuta un motore di ricerca su un banco di prova comprendente $|\mathcal{Q}|$ bisogni informativi distinti, si calcola la **Mean Average Precision** (MAP) come media aritmetica dei valori di AP:

Definizione 1.7.3 (Mean Average Precision)**.**

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} AP(q) \tag{1.10}$$

Proprietà 1.7.2 (Macro-Averaging vs. Micro-Averaging)**.** *Il MAP adotta formalmente una procedura di* ***Macro-Averaging****: ciascuna query possiede un peso identico pari a* $1/|\mathcal{Q}|$*.*

- *Se si adottasse il Micro-Averaging (ovvero sommare tutte le precisioni istantanee e dividere per il numero totale di documenti rilevanti dell'intera collezione), la misura verrebbe totalmente monopolizzata da pochissime query iper-generiche (ad es.* `"film"` *o* `"notizie"`*, aventi decine di migliaia di documenti rilevanti), mascherando il fallimento del motore sulle query rare e specifiche (long-tail).*
- *Il Macro-Averaging garantisce che ogni singola esigenza informativa dell'utente riceva pari dignità nella valutazione finale.*

Esempio 1.7.4 (Calcolo di MAP su Due Query)**.** Si consideri la valutazione congiunta di due query q_1 e q_2:

- **Query 1 (**$R_1 = 5$ **rilevanti totali):** Il sistema restituisce 5 rilevanti ai ranghi $k \in \{1, 3, 6, 9, 10\}$ con precisioni istantanee pari a 1.00, 0.67, 0.50, 0.44 e 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62}$$

- **Query 2 (**$R_2 = 3$ **rilevanti totali):** Il sistema restituisce i 3 rilevanti ai ranghi $k \in \{2, 5, 7\}$ con precisioni istantanee pari a 0.50, 0.40 e 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44}$$

Il valore di Mean Average Precision del motore sui due compiti ammonta a:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \tag{1.11}$$

1.7.4 Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k

Nei moderni motori di ricerca su Web e nei sistemi di raccomandazione, calcolare l'AP sull'intera collezione risulta impraticabile (perché il numero totale R di rilevanti non è noto a priori) e poco rilevante per l'esperienza utente. Si adotta pertanto la variante troncata a soglia fissa K (**MAP@K**):

Definizione 1.7.4 (AP@K e MAP@K)**.** Fissata la soglia di taglio K (tipicamente $K = 10$ o $K = 20$), l'Average Precision at K per la query q_i è definita come:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^{K} P@k(q_i) \cdot \text{rel}(q_i, k) \tag{1.12}$$

dove il fattore di normalizzazione K_i è dato dal numero di documenti rilevanti presenti nei primi K posti:

$$K_i = \sum_{k=1}^{K} \text{rel}(q_i, k) \quad (\text{oppure } K_i = \min(R_i, K)) \tag{1.13}$$

La metrica globale **MAP@K** su un insieme di query $\mathcal{Q}$ corrisponde alla media:

$$MAP@K(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} AP@K(q_i) \tag{1.14}$$

Questa formulazione risolve brillantemente il limite di $P@K$: normalizzando per K_i anziché rigidamente per K, una query con soli 2 rilevanti che li colloca ai primi 2 posti ottiene $AP@10 = 1.0$ anziché essere artificialmente limitata a 0.20.

1.7.5 Mean Reciprocal Rank (MRR)

In scenari quali le ricerche di elementi noti (*Known-Item Search*), le query navigazionali (ad es. cercare l'home page di un ateneo) o il reperimento di risposte fattuali, l'utente è interessato unicamente a individuare la **prima risposta corretta**. Lo sforzo cognitivo e il tempo speso dall'utente sono inversamente proporzionali alla posizione occupata dal primo risultato rilevante:

Definizione 1.7.5 (Reciprocal Rank e MRR)**.** Sia $\text{rank}_1(q)$ la posizione del primo documento rilevante restituito per la query q. Il **Reciprocal Rank** (RR) è dato da:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \tag{1.15}$$

Il **Mean Reciprocal Rank** (MRR) è la media aritmetica su tutte le query:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \tag{1.16}$$

Se per una query il primo documento rilevante si trova al rango 1, il punteggio è 1.0; se scende al rango 2 diviene 0.5; se scende al rango 10 crolla a 0.1.

1.8 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG

Nelle applicazioni Web reali, l'ipotesi che un documento sia puramente rilevante (1) o non rilevante (0) risulta insufficiente. Si consideri la seguente query reale:

```
Query:  "information retrieval unipi"
```

Una tipica SERP commerciale restituisce risultati con livelli qualitativi marcatamente differenti:

1. **DidaWiki del Corso (Docente, CFU, Programma, Materiale didattico):** Rappresenta la pagina principale che soddisfa pienamente l'intento dell'utente $\implies$ **Altamente Rilevante** $(r = 3)$.

2. **Portale Ufficiale UniPi Valutami (Scheda d'esame e requisiti istituzionali):** Pagina autorevole ma meno ricca di dispense didattiche $\implies$ **Rilevante** $(r = 2)$.

3. **Brochure PDF del Corso di Laurea Magistrale:** Contiene solo una breve menzione del corso $\implies$ **Marginalmente Rilevante** $(r = 1)$.

4. **Articoli di cronaca o pagine non correlate:** $\implies$ **Irrilevante** $(r = 0)$.

Tutti i primi tre documenti sono in qualche misura attinenti, ma un utente sarà soddisfatto solo se la pagina più informativa compare in cima. Per catturare questa granularità, la letteratura adotta scale a più livelli (tipicamente $r \in \{0, 1, 2, 3\}$ o $\{0, \ldots, 4\}$).

L'approccio del **Discounted Cumulative Gain** (DCG), introdotto da Järvelin e Kekäläinen [8], poggia su due assunzioni fondamentali:

1. I documenti altamente rilevanti apportano un'utilità (*gain*) di gran lunga superiore rispetto a documenti solo marginalmente rilevanti.

2. Minore è la posizione ordinale (rango) di un documento nella graduatoria, minore è la probabilità che l'utente lo esamini; pertanto, il guadagno deve essere progressivamente scontato (*discounted*) verso il basso.

1.8.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Definizione 1.8.1 (Cumulative Gain a Rango p)**.** Dato il vettore delle rilevanze graduate $\mathbf{r} = [r_1, r_2, \ldots, r_p]$, il Cumulative Gain è la somma lineare dei guadagni:

$$CG_p = \sum_{i=1}^{p} r_i \tag{1.17}$$

Il CG_p è insensibile alla posizione: scambiare un documento altamente rilevante al rango 1 con uno al rango 10 non altera il valore di CG_p.

Definizione 1.8.2 (Discounted Cumulative Gain (Formulazione Standard))**.** Il DCG applica uno sconto logaritmico basato sul rango della posizione i:

$$DCG_p = r_1 + \sum_{i=2}^{p} \frac{r_i}{\log_2(i)} \tag{1.18}$$

Significato Comportamentale dello Sconto Logaritmico

Lo sconto $\frac{1}{\log_2(i)}$ riflette il decadimento esponenziale dell'attenzione visiva dell'utente secondo il modello di lettura a cascata:

- Al rango $i = 4$, il fattore di sconto è $\frac{1}{\log_2(4)} = \frac{1}{2}$: il valore informativo percepito si dimezza rispetto alla prima posizione.
- Al rango $i = 8$, il fattore è $\frac{1}{\log_2(8)} = \frac{1}{3}$.
- Al rango $i = 16$, il fattore scende a $\frac{1}{\log_2(16)} = \frac{1}{4}$.

Nei motori di ricerca commerciali (Google, Bing) si impiega frequentemente la **formulazione esponenziale**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^{p} \frac{2^{r_i} - 1}{\log_2(i+1)} \tag{1.19}$$

Chiarimento sulla Notazione degli Indici

Si noti la differenza nell'argomento del logaritmo:

- Nell'Equazione (1.18) il primo termine r_1 viene estratto per evitare $\log_2(1) = 0$ al denominatore, mentre la sommatoria parte da $i = 2$ con $\log_2(i)$.
- Nell'Equazione (1.19) la sommatoria include direttamente il rango $i = 1$ adottando $\log_2(i + 1)$ al denominatore ($\log_2(1 + 1) = \log_2(2) = 1$).
- Inoltre, il numeratore $2^{r_i} - 1$ enfatizza in modo non lineare i documenti di massima rilevanza ($r = 3 \implies 2^3 - 1 = 7$, mentre $r = 1 \implies 2^1 - 1 = 1$), penalizzando aspramente il loro mancato posizionamento nei primi ranghi.

1.8.2 Normalized Discounted Cumulative Gain (NDCG)

Poiché query differenti presentano un numero variabile di documenti rilevanti, il valore grezzo di DCG_p non è confrontabile tra query diverse. Si ricorre quindi all'**Ideal DCG** ($IDCG_p$), ottenuto calcolando il DCG della graduatoria ideale che ordina tutti i documenti per rilevanza decrescente:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \quad \in [0, 1] \tag{1.20}$$

1.8.3 Esempio Numerico Completo: Calcolo Passo-Passo

Si consideri la seguente graduatoria di 10 documenti, valutati su scala 0–3:

$$\mathbf{r} = [3,\ 2,\ 3,\ 0,\ 0,\ 1,\ 2,\ 2,\ 3,\ 0]$$

La sequenza ideale ottenuta ordinando per rilevanza non crescente è:

$$\mathbf{r}^* = [3,\ 3,\ 3,\ 2,\ 2,\ 2,\ 1,\ 0,\ 0,\ 0]$$

La Tabella 1.4 riporta l'evoluzione dettagliata di ogni singolo termine posizionale.

i	r_i	Sconto	Guadagno Scontato	$DCG@i$	r_i^*	Guadagno Ideale	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	**3.00**	3	3.00	**3.00**	**1.00**
2	2	$\log_2(2) = 1.000$	2.00	**5.00**	3	3.00	**6.00**	**0.83**
3	3	$\log_2(3) = 1.585$	1.89	**6.89**	3	1.89	**7.89**	**0.87**
4	0	$\log_2(4) = 2.000$	0.00	**6.89**	2	1.00	**8.89**	**0.76**
5	0	$\log_2(5) = 2.322$	0.00	**6.89**	2	0.86	**9.75**	**0.71**
6	1	$\log_2(6) = 2.585$	0.39	**7.28**	2	0.77	**10.52**	**0.69**
7	2	$\log_2(7) = 2.807$	0.71	**7.99**	1	0.36	**10.88**	**0.73**
8	2	$\log_2(8) = 3.000$	0.67	**8.66**	0	0.00	**10.88**	**0.80**
9	3	$\log_2(9) = 3.170$	0.95	**9.61**	0	0.00	**10.88**	**0.88**
10	0	$\log_2(10) = 3.322$	0.00	**9.61**	0	0.00	**10.88**	**0.88**

Tabella 1.4: Tracciamento dettagliato del calcolo di $DCG@i$, $IDCG@i$ e $NDCG@i$ per $i = 1, \ldots, 10$.

1.8.4 Confronto tra Funzioni di Ranking Alternative

Si consideri il caso studio di 4 documenti $\{d_1, d_2, d_3, d_4\}$ con gradi di rilevanza:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordinamento $\langle d_4, d_3, d_2, d_1 \rangle$ con vettori di rilevanza $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$
$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordinamento $\langle d_3, d_4, d_2, d_1 \rangle$ con rilevanze $[2, 2, 1, 0]$. Poiché i primi due documenti possiedono il medesimo grado ($r = 2$), l'inversione non produce alcuna penalità posizionale:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$
$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordinamento $\langle d_3, d_2, d_4, d_1 \rangle$ con rilevanze $[2, 1, 2, 0]$. In questo caso, il documento d_2 ($r = 1$) scavalca erroneamente d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + \frac{2}{1.585} = 4.2619$$
$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

Questo calcolo dimostra con precisione matematica come la metrica NDCG penalizzi le inversioni di rango proporzionalmente alla gravità dello scambio di rilevanza.

Capitolo 2

Efficienza Algoritmica sulle Architetture CPU Moderne

2.1 Il Ruolo dell'Efficienza nell'Information Retrieval

Lo sviluppo di sistemi di elaborazione delle informazioni si articola lungo due dimensioni complementari e spesso in tensione tra loro:

1. **Efficacia (*Effectiveness*):** massimizza la soddisfazione dell'utente finale (*user satisfaction*), concentrandosi sulla qualità semantica dei risultati, sul ranking e sull'elaborazione del linguaggio naturale, tipicamente con strumenti ad alto livello di astrazione (Python, framework di Machine Learning).

2. **Efficienza (*Efficiency*):** minimizza i costi operativi infrastrutturali, progettando algoritmi ottimizzati fino all'ultimo ciclo di clock della CPU e all'ultimo bit di memoria occupato.

Non è richiesto che una singola figura professionale padroneggi entrambi gli estremi; tuttavia chi sviluppa ad alto livello deve comprendere i vincoli fisici dell'hardware sottostante per non progettare soluzioni impraticabili, mentre chi ottimizza a basso livello deve preservare la struttura e la semantica dei dati per non comprometterne l'efficacia.

2.1.1 I Tre Pilastri dell'Efficienza nei Sistemi IR

Esperienza utente. Nei sistemi web su scala planetaria, la risposta a una query deve essere erogata entro una frazione infinitesima di secondo. L'utente

percepisce qualsiasi ritardo superiore a pochi decimi di secondo: se i risultati tardano o risultano irrilevanti, abbandona il servizio a favore della concorrenza. L'efficienza algoritmica non serve solo a rispondere più rapidamente, ma consente di eseguire algoritmi di ranking e filtraggio più sofisticati ed efficaci *entro il medesimo budget temporale prestabilito* (tipicamente un tetto massimo di 100–200 ms).

Scalabilità operativa. I sistemi moderni gestiscono miliardi di documenti, elementi multimediali, query e log generati ogni secondo. Il numero di macchine necessarie è approssimativamente:

$$\text{Numero di macchine} = \left\lceil \frac{\text{Volume totale dei dati}}{\text{Capacità operativa del singolo nodo}} \right\rceil .$$

Se il volume dei dati raddoppia, un sistema ben progettato richiede al più il doppio delle macchine (scalabilità lineare); un algoritmo inefficiente può invece richiedere una crescita superlineare dei costi (ad es. quadruplicare i server), insostenibile su larga scala. Per rispettare le latenze richieste, i dati attivi devono risiedere in RAM ed evitare gli accessi a disco: ciò richiede algoritmi efficienti non solo in tempo ma anche in spazio, tramite tecniche di compressione. Inoltre, poiché i dati vengono replicati su più macchine per tolleranza ai guasti (*fault tolerance*), ogni inefficienza di memoria viene amplificata dal fattore di replica dell'infrastruttura.

Sostenibilità ambientale ed economica. La riduzione del numero di macchine fisiche abbatte direttamente il consumo energetico dei data center. I server ad alta densità dissipano ingenti quantità di calore e gran parte dell'energia è consumata dagli impianti di raffreddamento; per questo molti data center su larga scala vengono collocati in regioni fredde o vicino a corsi d'acqua (*free cooling*). Un risparmio computazionale marginale, anche solo del 3%, genera risparmi economici rilevanti e riduce l'impronta di carbonio.

2.2 La Gerarchia di Memoria e i Principi di Località

Nei moderni sistemi di reperimento dell'informazione, le prestazioni complessive sono dominate dai tempi di accesso alla memoria piuttosto che dalla mera velocità di calcolo dell'ALU. Questa sezione analizza la struttura gerarchica della memoria, il divario di latenza tra processore e DRAM, e i principi architetturali di località e prefetching che guidano la progettazione di algoritmi e indici ad alte prestazioni.

2.2.1 Il Divario di Latenza tra CPU e Memoria

Nel modello teorico computazionale classico (macchina RAM astratta) l'accesso a qualunque locazione di memoria ha costo unitario costante, $\mathcal{O}(1)$. Nelle macchine fisiche reali le unità aritmetiche della CPU operano a velocità enormemente superiori rispetto alla memoria principale (DRAM), ed esiste una gerarchia di livelli intermedi (le *cache*) che approssima l'accesso rapido per i dati usati di frequente.

Livello hardware	**Latenza tipica**	**Cicli CPU**	**Capacità tipica**
Istruzione CPU (ALU)	0.5–5 ns	1–2	registri interni
L1I / L1D Cache	0.5 ns	≈ 1	32 KB per core
L2 Cache	1.5 ns	≈ 3–4	256 KB (fino a 2 MB)
L3 Cache (LLC)	5.0 ns	≈ 10–15	16 MB (fino a 64 MB+)
RAM principale (DRAM)	80.0 ns	≈ 160–200+	64 GB+
Disco / Storage	500 000 ns (0.5 ms)	milioni	8 TB+

Tabella 2.1: Confronto quantitativo di latenza e capacità tra i livelli della gerarchia di memoria.

Definizione 2.2.1 (Algoritmo *Memory-Bound*)**.** Un algoritmo che accede alla RAM centrale in modo disordinato paga ≈ 80 ns per ogni accesso, intervallo in cui la CPU potrebbe eseguire centinaia di istruzioni. L'algoritmo si dice **memory-bound**: il tempo di esecuzione è dominato dall'attesa del trasferimento dati dalla DRAM e la CPU trascorre la maggior parte del tempo in stallo, rendendo irrilevante l'ottimizzazione del numero di operazioni aritmetiche.

2.2.2 Architettura Fisica sul Chip

La cache L1 è integrata a ridosso dell'unità di decodifica e delle unità di esecuzione; per evitare conflitti di bus è fisicamente suddivisa in **L1I** (*Instruction Cache*, contiene il codice macchina prelevato per l'esecuzione) e **L1D** (*Data Cache*, contiene gli operandi delle istruzioni di `load`/`store`). La cache L2 è integrata nel singolo core e funge da cuscinetto a bassa latenza; la cache L3 è esterna ai singoli core ma interna al package ed è condivisa tra tutti i core della macchina.

La Cache Line come Unità Minima di Trasferimento

La RAM e la gerarchia di cache non trasferiscono mai singoli byte o singole variabili isolate: l'unità minima di trasferimento hardware è la **cache line**, tipicamente $B = 64$ byte. Se un programma necessita di un intero a 64 bit (8 byte) situato all'indirizzo A, l'hardware carica forzatamente l'intero blocco allineato

$$[\lfloor A/64 \rfloor \times 64,\ \lfloor A/64 \rfloor \times 64 + 63]\,.$$

Se il programma legge quell'unico valore e poi salta a un indirizzo casuale, i restanti 56 byte (87.5% del traffico di memoria) vengono sprecati.

2.2.3 Località Temporale e Spaziale

Un programma sfrutta efficacemente la gerarchia di memoria solo se rispetta almeno una delle due forme di località:

- **Località temporale:** quando una data locazione di memoria viene acceduta, è altamente probabile che la medesima locazione venga acceduta nuovamente nell'immediato futuro. Le politiche di rimpiazzo delle cache (varianti di **LRU**, *Least Recently Used*) scartano i dati meno recentemente utilizzati per preservare le locazioni accedute di frequente.
- **Località spaziale:** quando una data locazione di memoria viene acceduta, è altamente probabile che locazioni contigue o vicine vengano accedute nell'immediato futuro. Poiché l'hardware trasferisce l'intera cache line, l'accesso a elementi contigui ammortizza il costo di caricamento su tutti gli elementi contenuti al suo interno.

2.2.4 Hardware Prefetcher

I processori integrano circuiti dedicati, gli **hardware prefetcher**, il cui compito è anticipare le richieste di memoria dell'applicazione:

- **Stream prefetcher:** se un programma scansiona linearmente la memoria da sinistra verso destra $(A, A+64, A+128, \dots)$, il circuito rileva il pattern sequenziale e ordina il caricamento preventivo delle cache line successive in L1/L2 prima ancora che il programma emetta l'istruzione di lettura, mascherando così la latenza.

- **Stride prefetcher:** intercetta accessi a passo costante (*stride*), ad esempio scansioni che campionano la memoria a intervalli fissi; riconosciuto il gradiente costante, carica selettivamente le cache line corrispondenti al passo di salto.
- **Pattern casuali:** di fronte a salti casuali e non deterministici (*pointer chasing*), nessun hardware prefetcher è in grado di anticipare gli accessi, causando lo stallo completo della pipeline a ogni lettura.

2.3 Caso di Studio: Scansione Sequenziale contro *Pointer Chasing*

Per dimostrare sperimentalmente l'impatto della località e del prefetching si confrontano due funzioni Rust operanti su un vettore `v` di dimensione N. Il vettore di ingresso rappresenta una **permutazione casuale con un singolo ciclo** $\pi \in S_N$.

Definizione 2.3.1 (Permutazione con un singolo ciclo)**.** Una permutazione π dell'insieme $\{0, 1, \ldots, N-1\}$ contiene un singolo ciclo se e solo se, partendo dall'indice 0, la catena $k_{t+1} = \pi(k_t)$ visita ogni posizione dell'array esattamente una volta prima di tornare a 0:

$$0 \to \pi(0) \to \pi(\pi(0)) \to \cdots \to \pi^{N-1}(0) \to 0,$$

senza cicli isolati o sottocicli disgiunti.

Entrambe le funzioni calcolano la sommatoria di tutti gli elementi del vettore, $\sum_{i=0}^{N-1} v[i] = \frac{(N-1)N}{2}$.

Listing 2.1: Scansione sequenziale lineare.

```
pub fn sum(v: &[usize]) -> usize {
    let mut sum = 0;
    for &value in v.iter() {
        sum += value;
    }
    sum
}
```

Listing 2.2: Scansione per salti casuali (*pointer chasing*).

```
pub fn sum_by_jumping(v: &[usize]) -> usize {
    let mut sum = 0;
```

```
    let mut k = 0;
    for _ in 0..v.len() {
        sum += v[k];
        k = v[k]; // il valore letto diventa l'indice
           della prossima iterazione
    }
    sum
}
```

Nel modello teorico RAM entrambe le funzioni eseguono N iterazioni con un'operazione di somma per passo: hanno cioè identica complessità asintotica $\mathcal{O}(N)$. Sulla macchina fisica, misurando il tempo medio per singolo elemento al variare della dimensione dell'array, emergono però divergenze marcate.

			Tempo per elem.		
N	**Dim.**	**Livello cache**	`sum()`	`jumping()`	**Rallentamento**
256	2 KiB	L1 (32 KB)	0.1 ns	1.3 ns	13×
8 192	64 KiB	L2 (256 KB)	0.1 ns	2.5 ns	25×
262 144	2 MiB	L3 (16 MB)	0.2 ns	13.4 ns	67×
16 777 216	128 MiB	RAM (DRAM)	0.4 ns	84.9 ns	212×

Tabella 2.2: Tempo medio per elemento di `sum()` e `sum_by_jumping()` al variare della dimensione dell'array.

2.3.1 Analisi delle Cause Fisiche

Perché `sum()` varia solo da 0.1 a 0.4 ns? Con interi `usize` a 64 bit (8 byte), in una cache line da 64 byte alloggiano $64/8 = 8$ elementi contigui: si ha piena località spaziale. L'accesso a una nuova linea comporta un miss iniziale, ma i successivi 7 accessi avvengono a costo pressoché nullo da cache, e lo stream prefetcher carica le linee successive in anticipo rispetto all'esecuzione. Indicando con T_{comp} il tempo di calcolo e con T_{load} il tempo di trasferimento ammortizzato, il costo osservato per elemento segue

$$\text{Latenza osservata} \approx \max(T_{\text{comp}},\ T_{\text{load_ammortizzato}})\,,$$

quasi interamente mascherato dal calcolo; l'aumento da 0.1 a 0.4 ns su 128 MiB è dovuto unicamente alla saturazione della banda del bus verso la RAM.

Perché `sum_by_jumping()` arriva a 84.9 ns? Poiché la permutazione è casuale, l'indirizzo $k_{t+1} = v[k_t]$ punta a una locazione arbitraria: la probabilità che cada nella stessa cache line è trascurabile, quindi per ogni elemento letto

da 8 byte i restanti 56 byte trasferiti vengono scartati (assenza totale di località spaziale). Non essendovi passo lineare o costante, il prefetching è inibito. Inoltre la CPU deve conoscere l'indice della lettura successiva prima di poterla emettere: si tratta di una **dipendenza seriale** che impedisce ogni sovrapposizione. Quando la dimensione dell'array supera la capienza della L3 (128 MiB $\gg$ 16 MB), ogni accesso diventa un cache miss globale verso la DRAM:

Tempo per elemento $\approx$ latenza di accesso fisico alla DRAM $\approx$ 80–85 ns,

valore che coincide esattamente con i 84.9 ns misurati.

Esempio 2.3.1 (Ammortizzare il costo del *pointer chasing* sfruttando la cache line)**.** Se il salto avviene tra blocchi di 64 byte (8 interi a 64 bit), ma tutti gli elementi del blocco vengono sommati prima del salto successivo, il costo di un cache miss verso la RAM ($\approx$ 85 ns) viene sostenuto una sola volta ogni 8 operazioni di somma:

```
pub fn sum_by_jumping_cache_lines(v: &[usize]) -> usize {
    let mut sum = 0;
    let mut k = 0;
    let num_blocks = v.len() / 8;
    for _ in 0..num_blocks {
        let base = k * 8;
        // sfrutta appieno la cache line gia' caricata in memoria
        sum += v[base] + v[base + 1] + v[base + 2] + v[base + 3]
            + v[base + 4] + v[base + 5] + v[base + 6];
        k = v[base + 7] % num_blocks; // puntatore al prossimo salto
    }
    sum
}
```

La latenza viene ammortizzata su 8 elementi, portando il tempo medio atteso per valore a

$$\text{Tempo medio per valore} \approx \frac{85\,\text{ns}}{8} \approx 10.6\,\text{ns},$$

un risultato intermedio tra la scansione lineare e il salto casuale elemento per elemento.

Nota 2.3.1 (Eliminare il rallentamento residuo di `sum()`)**.** Il lieve rallentamento da 0.1 a 0.4 ns osservato in `sum()` si verifica perché $T_{\text{comp}} \approx 0.1$ ns è

inferiore al tempo necessario ad alimentare il flusso dati dalla RAM su vettori molto grandi (regime memory-bound). Aumentando la complessità computazionale del corpo del ciclo (ad es. moltiplicazioni o funzioni non banali su ciascun elemento) si ottiene $T_{\text{comp}} \gg T_{\text{load_RAM}}$: la relazione $\max(T_{\text{comp}}, T_{\text{load}})$ diventa costantemente pari a T_{comp}, il regime diventa *CPU-bound* e la curva dei tempi risulta perfettamente piatta al variare di N.

2.4 Forme di Parallelismo e Architettura della Pipeline

Le moderne CPU sfruttano molteplici forme di parallelismo: il **multi-threading/multi-core** (più core fisici indipendenti sullo stesso processore), il **pipelining** (suddivisione dell'esecuzione dell'istruzione in stadi operanti simultaneamente su istruzioni contigue) e l'esecuzione **superscalare** (unità di calcolo multiple nello stesso core per elaborare più istruzioni indipendenti per ciclo).

Parallelismo Triviale nei Motori di Ricerca

Nei motori di ricerca non è necessario sincronizzare più core per rispondere a una singola query: si adotta il **parallelismo triviale** (*embarrassingly parallel*), in cui, data la presenza di migliaia di utenti simultanei, ogni thread su un singolo core serve in modo completamente isolato una specifica query. Non essendovi condivisione di stato, si azzerano le contese sui lock e i problemi di coerenza della cache.

2.4.1 Anatomia del Pipelining a Cinque Stadi

Il pipelining scompone l'esecuzione di un'istruzione in stadi sequenziali indipendenti che, in un'architettura standard, sono cinque:

1. **IF** (*Instruction Fetch*): prelievo dell'istruzione in codice macchina dalla memoria (L1I) tramite il *program counter*.
2. **ID** (*Instruction Decode*): decodifica dell'istruzione in micro-operazioni (μ-ops) e lettura dei registri sorgente.
3. **EX** (*Execute*): esecuzione del calcolo aritmetico-logico nelle ALU o calcolo dell'indirizzo effettivo di memoria.
4. **MEM**: accesso al sottosistema di memoria (L1D) per istruzioni di caricamento o scrittura (`LOAD`/`STORE`).

5. **WB** (*Write-Back*): scrittura del risultato nei registri architetturali e ritiro dell'istruzione.

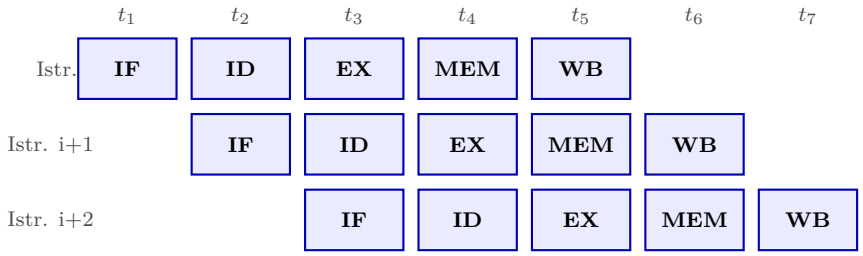


Figura 2.1: Sovrapposizione degli stadi della pipeline a 5 stadi su istruzioni consecutive.

2.4.2 Latenza, Throughput e Profondità della Pipeline

Sia k il numero di stadi della pipeline e τ il periodo del ciclo di clock. La **latenza** L è il tempo totale richiesto da una singola istruzione per attraversare l'intera pipeline, $L = k \cdot \tau$: nelle CPU moderne la latenza per un'operazione semplice varia tipicamente tra 10 e 20 cicli di clock. Il **throughput** W è il numero di istruzioni completate per unità di tempo e, a regime, $W = 1/\tau$ istruzioni al secondo, poiché a ogni ciclo un'istruzione esce dalla pipeline. Il periodo τ è imposto dal ritardo dello stadio più lento della catena circuitale,

$$\tau \geq \max_{1 \leq i \leq k} \left(t_{\text{stadio}_i} \right).$$

Per innalzare la frequenza di clock (ridurre τ) i progettisti aumentano la profondità k della pipeline (fino a 15–20 o più stadi nelle CPU moderne), implementando ciascuno stadio con circuiteria più semplice e veloce; di contro, l'aumento di k dilata la penalità in caso di svuotamento forzato della pipeline.

2.5 Esecuzione Superscalare, Interleaving e Conflitti Strutturali

Un processore si definisce **superscalare** quando possiede l'abilità architetturale di avviare e completare più di un'istruzione per ciclo di clock ($IPC > 1$, *Instructions Per Cycle*) all'interno dello stesso core, integrando unità logico-aritmetiche multiple (ALU per interi, moltiplicatori, unità di shift, unità di load/store) servite da porte di emissione parallele (*execution ports*). Architetture reali come Intel Haswell dispongono ad esempio di una coda di decodifica

a 56 voci, un *reorder buffer* a 192 voci, una stazione di prenotazione unificata a 60 voci e 8 porte di esecuzione parallele, ciascuna specializzata per classi di operazioni (ALU intere, load/store, unità vettoriali, branch, ecc.).

2.5.1 L'Esperimento dell'*Interleaving* Aritmetico

Si sottopongono a benchmark quattro funzioni Rust che operano su un vettore di interi a 64 bit `&[u64]`, accumulando un numero crescente di operazioni indipendenti per iterazione.

Listing 2.3: Da un'unica riduzione a quattro riduzioni indipendenti per iterazione.

```
pub fn do_stuff(v: &[u64]) -> u64 {
    let mut prod = 1;
    for &value in v.iter() {
        prod *= value;
    }
    prod
}

pub fn do_more_stuff(v: &[u64]) -> (u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    for &value in v.iter() {
        prod *= value;
        sum += value;
    }
    (prod, sum)
}

pub fn do_more_stuff_2(v: &[u64]) -> (u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    for &value in v.iter() {
        prod *= value;
        sum += value;
        xor ^= value;
    }
    (prod, sum, xor)
}

#[inline]
```

```rust
pub fn do_more_stuff_3(v: &[u64]) -> (u64, u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    let mut or = 0;
    for &value in v.iter() {
        prod *= value;
        sum += value;
        xor ^= value;
        or |= value;
    }
    (prod, sum, xor, or)
}
```

Funzione	Tempo per iterazione
`do_stuff()` (1 operazione)	0.4 ns
`do_more_stuff()` (2 operazioni)	0.4 ns
`do_more_stuff_2()` (3 operazioni)	0.4 ns
`do_more_stuff_3()` (4 operazioni)	0.5 ns

Tabella 2.3: Tempo per iterazione al crescere del numero di riduzioni indipendenti accumulate.

2.5.2 Sovrapposizione delle Istruzioni e Saturazione delle Porte

Da una a tre operazioni il tempo resta invariato a 0.4 ns perché le variabili di accumulo non presentano dipendenze reciproche: $\text{prod}_{t+1} = \text{prod}_t \cdot v[i]$, $\text{sum}_{t+1} = \text{sum}_t + v[i]$, $\text{xor}_{t+1} = \text{xor}_t \oplus v[i]$. Il core superscalare invia contemporaneamente la moltiplicazione, l'addizione e lo XOR a tre porte di esecuzione distinte: le tre operazioni vengono completate nel medesimo ciclo di clock, e il lavoro aggiuntivo è svolto a costo nullo.

Con la quarta operazione (`or`) il tempo sale a 0.5 ns anziché raddoppiare a 0.8 ns: si tratta di uno **structural hazard**, poiché le unità fisiche disponibili per calcoli logico-aritmetici in parallelo in un singolo ciclo sono sature (solo tre istruzioni di quel tipo possono essere emesse simultaneamente), e una delle quattro operazioni deve attendere. Tuttavia la CPU *out-of-order* non processa le iterazioni come compartimenti stagni: srotola implicitamente la finestra di istruzioni raggruppandole a blocchi di tre, assorbendo l'operazione in eccesso nei cicli successivi e saturando le capacità residue delle porte. L'overhead

reale corrisponde quindi a una frazione marginale di ciclo ($\Delta t = +0.1\,\text{ns}$) anziché a un ciclo intero aggiuntivo.

2.6 Data Hazard e Dipendenze Seriali

Definizione 2.6.1 (Data Hazard)**.** Si verifica un **data hazard** quando un'istruzione necessita di un operando non ancora disponibile poiché in fase di calcolo da parte di un'istruzione precedente. Corrisponde a una dipendenza sui dati di tipo **RAW** (*Read-After-Write*):

$$\text{Inst}_A:\ R_1 \leftarrow R_2 + R_3 \qquad \Longrightarrow \qquad \text{Inst}_B:\ R_4 \leftarrow R_1 \times R_5,$$

dove Inst_B deve rimanere in stallo nella pipeline finché Inst_A non conclude il calcolo.

Si consideri un codice che esegue lo stesso numero di operazioni per iterazione della precedente `do_more_stuff_3`, ma concatenando i dati in una catena stretta:

Listing 2.4: Catena di dipendenza seriale RAW tra quattro operazioni.

```
pub fn do_more_stuff_4(v: &[u64]) -> (u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    for &value in v.iter() {
        sum += value;  // 1. calcola sum
        prod *= sum;   // 2. dipende strettamente da sum
        xor ^= prod;   // 3. dipende strettamente da
           prod
        sum += xor;    // 4. dipende da xor e
           sovrascrive sum per l'iterazione successiva!
    }
    (prod, sum, xor)
}
```

A causa della catena RAW, le unità di esecuzione non possono avviare alcun calcolo in parallelo:

$$\text{Latenza iterazione} = t_{\text{add}} + t_{\text{mul}} + t_{\text{xor}} + t_{\text{add_feedback}}.$$

Poiché l'ultima operazione aggiorna `sum` e la prima istruzione dell'iterazione successiva richiede il nuovo valore di `sum`, la CPU *out-of-order* non può avviare l'iterazione $t + 1$ in anticipo: le unità superscalari restano inutilizzate.

Sperimentalmente, contro i 0.5 ns di `do_more_stuff_3` (operazioni indipendenti), `do_more_stuff_4` misura 1.7 ns per iterazione ($\approx 3.5\times$ più lento), pur eseguendo lo stesso numero di operazioni aritmetiche.

2.6.1 Soluzione Algoritmica: *Interleaving* di Vettori Indipendenti

Quando occorre eseguire tale calcolo su due distinti vettori `V` e `West`, l'approccio sequenziale (chiamando `do_more_stuff_4` su `V` e poi su `West`) impiega un tempo complessivo pari a $2T$. Se invece si scrive un'unica funzione che elabora entrambi i vettori all'interno della stessa iterazione, alternando le due catene indipendenti:

Listing 2.5: Fusione (interleaving) di due catene RAW indipendenti tra loro.

```
for i in 0..n {
    // catena per V (occupa le porte al ciclo k)
    sum_v += v[i];
    prod_v *= sum_v;
    xor_v ^= prod_v;
    sum_v += xor_v;
    // catena per West, completamente indipendente da V!
    sum_w += west[i];
    prod_w *= sum_w;
    xor_w ^= prod_w;
    sum_w += xor_w;
}
```

il tempo di esecuzione misurato è identico a quello del solo vettore `V` ($\approx T$, anziché $2T$). Le operazioni su `West` non dipendono da quelle su `V`: mentre la catena di `V` è in attesa dei risultati delle ALU, le unità di calcolo libere processano la catena di `West`, saturando a costo nullo i tempi morti imposti dalle dipendenze sui dati.

2.7 Control Hazard, Esecuzione Speculativa e Branch Prediction

Definizione 2.7.1 (Control Hazard)**.** Si verifica un **control hazard** quando la CPU non può determinare quale sia la successiva istruzione da prelevare dalla memoria (*instruction fetch*) a causa della presenza di un salto condizionato (`if`, `branch`).

La CPU non può fermarsi a ogni condizione: adotta pertanto l'**esecuzione speculativa**, ipotizzando tramite un *branch predictor* quale ramo verrà imboccato e prelevando speculativamente le istruzioni successive per alimentare la pipeline. Se l'esito della condizione si rivela discorde rispetto alla predizione, tutto il lavoro speculativo completato negli stadi intermedi deve essere annullato, con uno svuotamento completo della pipeline (*pipeline flush*) e il ricaricamento dal ramo corretto:

$$\text{Penalità per mispredizione} = 15\text{–}20 \text{ cicli di clock.}$$

2.7.1 Il Paradosso della Condizione Casuale

Si introduce un filtro condizionale in un codice con catena di dipendenza, operando su un array di interi casuali uniformemente distribuiti in $[0, 99]$:

Listing 2.6: Ramo condizionale su valori casuali: la CPU non può predire l'esito.

```
pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    for &value in v.iter() {
        if value > 50 { // vera con p = 0.5 su valori
           casuali in [0, 99]
            sum += value;
            prod *= sum;
            xor ^= prod;
            sum += xor;
        }
    }
    (prod, sum, xor)
}
```

Poiché la condizione è vera in media il 50% delle volte, ci si aspetterebbe intuitivamente un tempo pressoché dimezzato rispetto a `do_more_stuff_4` ($1.7/2 \approx 0.85$ ns). Il tempo misurato sale invece a 3.5 ns, più del doppio più lento della versione priva di condizione.

Il Costo Reale non è il Calcolo ma la Mispredizione

Con valori casuali il branch predictor ha accuratezza pari al lancio di una moneta, $P(\text{hit}) = P(\text{miss}) = 0.50$. Il costo medio per iterazione è allora

$$\begin{aligned}\text{Costo medio} &\approx 0.5 \times T_{\text{corpo}} + P(\text{miss}) \times \text{Penalità flush} \\ &\approx 0.5 \times 0.85\,\text{ns} + 0.5 \times 5.0\,\text{ns} \approx 2.9\text{–}3.5\,\text{ns},\end{aligned}$$

avendo assunto una penalità di 15–20 cicli ($\approx$ 4–5 ns su una CPU a 4 GHz). Il costo dell'algoritmo non è più dettato dalle operazioni aritmetiche, ma è dominato dal tempo perso a svuotare e ricaricare la pipeline.

2.7.2 Verifica Sperimentale: Ordinamento dei Dati e Pattern Periodici

Per dimostrare che il rallentamento deriva esclusivamente dal comportamento del branch predictor, si esegue lo stesso identico codice modificando solo la disposizione dei dati nell'array.

Configurazione	**Tempo / iter.**	**Comportamento del predittore**
`do_more_stuff_4()` (nessun `if`)	1.7 ns	—
`do_more_stuff_5()` (dati casuali)	3.5 ns	Ramo impredicibile, miss rate $\approx 50\%$
`do_more_stuff_5()` (dati ordinati)	1.2 ns	Predizione quasi perfetta, un solo miss
`do_more_stuff_5()` (dati alternati)	1.1 ns	Sequenza periodica, miss rate $\approx 0\%$

Tabella 2.4: Tempo per iterazione di `do_more_stuff_5()` al variare della disposizione dei dati.

Con **dati ordinati** (tutti gli elementi ≤ 50 all'inizio dell'array e tutti quelli > 50 alla fine) il predittore assume che il salto non sia preso per l'intera prima metà e che sia preso per la seconda: si verifica un unico misprediction al punto di transizione, e rimosso il costo dei flush il tempo scende a 1.2 ns, inferiore ai 1.7 ns della versione senza condizione poiché metà delle iterazioni non esegue il corpo dell'`if`. Con **dati alternati** secondo un pattern periodico (*falso, vero, falso, vero, ...*), i predittori integrano tabelle storiche dei pattern

(*branch history shift register*): identificata la periodicità, il predittore anticipa l'esito a ogni passo senza errori, annullando gli stalli.

2.7.3 Tecniche *Branchless* e il Salto Implicito dei Cicli `for`

Se i dati non sono ordinabili né predicibili, l'unica soluzione consiste nel rimuovere fisicamente il salto condizionale (*branchless code*), sostituendolo con espressioni algebriche o istruzioni condizionali assembly (`CMOV`). Ad esempio, invece di

```
if value > 50 {
    sum += value;
}
```

si mappa la guardia logica su un valore binario scalare (0 o 1), eseguendo sempre l'operazione aritmetica:

```
let condition = (value > 50) as u64; // vale 1 se vero, 0 se falso
sum += value * condition;            // somma 'value' se vero, somma '0' se falso
```

Il programma compie sempre il calcolo matematico, ma il flusso di istruzioni è lineare: non vi sono salti, e le mispredizioni sono azzerate.

Nota 2.7.1 (Perché il salto del ciclo `for` non costituisce un problema)**.** Ogni ciclo `for` si conclude a livello macchina con un salto condizionato all'inizio del blocco. Questo branch non degrada le prestazioni perché è altamente asimmetrico: per N iterazioni la condizione di permanenza nel ciclo risulta vera per $N-1$ volte consecutive ed è falsa solo all'uscita. Il branch predictor impara immediatamente il pattern ("salto preso") e commette un unico errore di predizione al termine del ciclo, con un impatto del tutto trascurabile sul tempo complessivo.

2.8 Sintesi: dai Vincoli Hardware alle Scelte Algoritmiche

I fenomeni analizzati in questo capitolo – gerarchia di memoria e località, pipelining, esecuzione superscalare, *data hazard* e *control hazard* – non sono dettagli implementativi trascurabili, ma vincoli fisici che determinano il reale comportamento prestazionale degli algoritmi di Information Retrieval su larga scala. In particolare:

- le strutture dati (posting list, indici invertiti) vanno progettate privilegiando l'accesso sequenziale e contiguo, per sfruttare la località spaziale e l'hardware prefetching;
- quando possibile, conviene esprimere più riduzioni indipendenti all'interno dello stesso ciclo per saturare le porte di esecuzione superscalari a costo marginale;
- le catene di dipendenza strette (RAW) tra variabili di accumulo vanno identificate e, se possibile, disaccoppiate o intrecciate (*interleaving*) con calcoli indipendenti;
- i rami condizionali imprevedibili sui dati vanno eliminati con tecniche *branchless* o mitigati riordinando i dati per favorire la predizione.

La trattazione della vettorizzazione SIMD e degli strumenti di analisi del codice compilato (*Compiler Explorer*) è rimandata a un capitolo successivo.

Capitolo 3

Elaborazione del Linguaggio Naturale e Preprocessing del Testo

3.1 Recap: Metriche di Valutazione per Rilevanza Graduata (CG, DCG, NDCG)

Nei sistemi reali di Information Retrieval, l'assunzione di rilevanza puramente binaria (documento pertinente oppure non pertinente) risulta insufficiente a cogliere le sfumature qualitative dei risultati restituiti da un motore di ricerca. I giudizi di rilevanza vengono tipicamente formulati su scale graduate discrete (ad esempio su una scala Likert da 0 a 3 o da 0 a 4, indicando livelli crescenti di pertinenza).

La valutazione di una graduatoria ordinata (*ranking*) alla posizione di taglio p si fonda su due principi cardine:

1. **Guadagno (*Gain*):** posizionare documenti a elevata rilevanza nelle primissime posizioni della pagina dei risultati (*SERP - Search Engine Results Page*) incrementa sensibilmente l'utilità percepita dall'utente.

2. **Sconto Posizionale (*Discounting*):** documenti pertinenti collocati in posizioni basse della graduatoria offrono un'utilità marginale progressivamente inferiore, poiché l'attenzione visiva dell'utente decresce monotonicamente scorrendo la lista.

3.1.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Dato un vettore di rilevanze graduate $\mathbf{r} = [r_1, r_2, \ldots, r_p]$ per i primi p documenti restituiti per una query q:

Definizione 3.1.1 (Cumulative Gain a Rango p)**.** Il **Cumulative Gain** (CG_p) somma linearmente i valori di rilevanza fino alla soglia p:

$$CG_p = \sum_{i=1}^{p} r_i. \tag{3.1}$$

Limite intrinseco: il CG_p è del tutto insensibile all'ordinamento posizionale: scambiare un documento altamente rilevante ($r = 3$) tra la prima e la decima posizione non altera il valore complessivo di CG.

Per modellare il decadimento dell'attenzione dell'utente, si introduce una funzione di sconto logaritmico rispetto alla posizione di rango i:

Definizione 3.1.2 (Discounted Cumulative Gain a Rango p)**.** Nelle formulazioni standard della letteratura e industriale:

$$\mathrm{DCG}_p = r_1 + \sum_{i=2}^{p} \frac{r_i}{\log_2(i)}, \tag{3.2}$$

$$\mathrm{DCG}_p^{\exp} = \sum_{i=1}^{p} \frac{2^{r_i} - 1}{\log_2(i+1)}. \tag{3.3}$$

La variante esponenziale (3.3) assegna un premio sproporzionato ai documenti con pertinenza massima nelle primissime posizioni, riflettendo le metriche adottate dai motori di ricerca commerciali.

3.1.2 Normalized Discounted Cumulative Gain (NDCG) e la Necessità della Normalizzazione

Il valore grezzo di DCG presenta un vincolo fondamentale: è una metrica strettamente **query-dependent**. Se per una query q_A esistono nel corpus 100 documenti a massima rilevanza, il valore teorico raggiungibile di DCG sarà elevatissimo; se per una query q_B esiste un solo documento rilevante, il DCG massimo sarà necessariamente modesto. Non è pertanto corretto mediare direttamente i valori grezzi di DCG tra query eterogenee.

Per garantire la comparabilità globale (*macro-averaging*), si calcola l'**Ideal DCG** (IDCG_p), definito come il DCG ottenuto ordinando tutti i documenti

della collezione per rilevanza strettamente decrescente fino al rango p:

$$\mathrm{NDCG}_p = \frac{\mathrm{DCG}_p}{\mathrm{IDCG}_p} \quad \in [0, 1]. \tag{3.4}$$

Un punteggio di $\mathrm{NDCG}_p = 1.0$ certifica che l'algoritmo ha generato l'ordinamento teoricamente ottimo possibile fino alla profondità p.

3.2 Proprietà Fondamentali del Linguaggio Naturale

Prima di definire i modelli matematici di indicizzazione e reperimento, è indispensabile esaminare la natura intrinsecamente complessa dei dati testuali. Questa sezione illustra le caratteristiche distintive del linguaggio umano, la distribuzione a legge di potenza delle parole (Legge di Zipf) e i vincoli statistici che ne condizionano l'elaborazione automatica.

3.2.1 Caratteristiche Distintive del Linguaggio Umano

L'elaborazione automatica del linguaggio naturale pone ostacoli strutturali dovuti a proprietà intrinseche del linguaggio umano:

- **Migliaia di simboli discreti:** i vocabolari contengono centinaia di migliaia di parole uniche e radici morfologiche, arricchite costantemente da neologismi e linguaggi settoriali.
- **Sintassi complessa e ricorsiva:** le frasi non sono semplici sequenze di caratteri, ma alberi di dipendenze annidati con relazioni grammaticali a lungo raggio.
- **Semantica per lo più composizionale:** il significato di un enunciato deriva dalla combinazione ordinata dei suoi costituenti elementari. Vi sono tuttavia importanti eccezioni:
  - *Collocazioni:* associazioni lessicali convenzionali frequenti (ad es. *"make the bed"*, *"saltare la lezione"*).
  - *Espressioni idiomatiche:* locuzioni figurate il cui senso globale non è deducibile dai singoli vocaboli (ad es. *"break a leg"*, *"dalla padella nella brace"*).
- **Ambiguità poliedrica:**

  - *Part-of-Speech (POS) Ambiguity:* la medesima voce grammaticale assume ruoli diversi a seconda del contesto (es. *"beat"*: sostantivo in *"Listen to this cool beat"*, verbo in *"I'll beat you at checkers"*).
  - *Word Sense Ambiguity (Polisemia e Omonimia):* una stessa forma superficiale possiede accezioni concettuali differenti (es. *"interest"*: inclinazione intellettuale vs tasso di interesse bancario).
  - *Ambiguità sintattica:* frasi compatibili con molteplici alberi sintattici distinti. Celebre l'esempio di Groucho Marx: *"I shot an elephant in my pajamas"*, in cui il complemento di luogo/stato può legarsi sintatticamente al soggetto (*io ero in pigiama*) oppure all'oggetto (*l'elefante indossava il pigiama*).

- **Dipendenza dalla conoscenza del mondo (*Shared World Knowledge*):** la comprensione corretta presuppone nozioni esterne al testo stesso. La frase *"I'm faster than Lewis Hamilton"* richiede la nozione ontologica che Hamilton sia un pilota di Formula 1; *"I was on a bike with a helmet"* implica che il casco sia indossato sul capo del ciclista, mentre *"I was on a bike with an electric engine"* implica che il motore sia parte integrante della bicicletta.

- **Evoluzione temporale continua:** slittamenti semantici (es. *"in the cloud"*) e creazione rapida di neologismi tecnologici e sociali (es. *"to google"*, *"crowdfunded"*, *"super spreader"*).

3.2.2 La Legge di Zipf e il Principio del Minimo Sforzo

La frequenza di occorrenza delle parole in un corpus naturale non segue un andamento gaussiano uniforme, bensì una distribuzione empirica a legge di potenza (*power law*) formulata dal linguista George Kingsley Zipf:

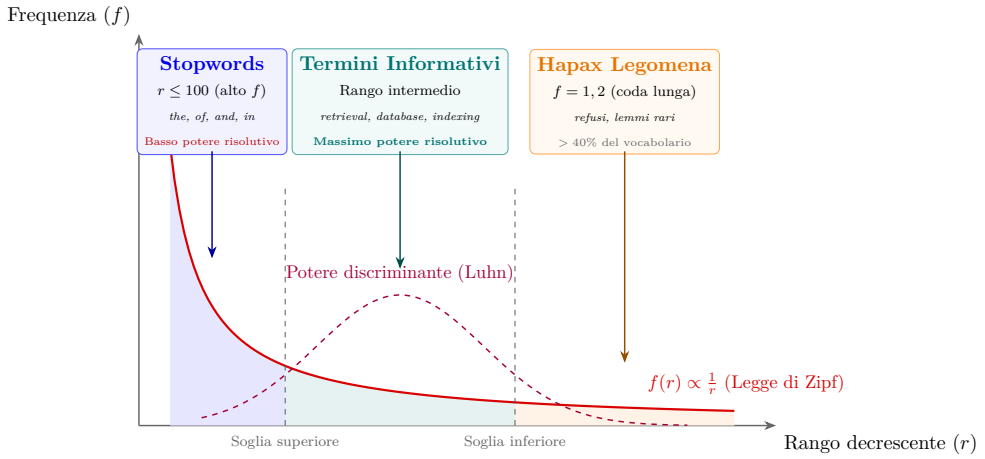


Figura 3.1: Distribuzione iperbolica di Zipf e potere discriminante del lessico (ipotesi di Luhn): suddivisione tra stopwords ad altissima frequenza ($r \leq 100$), termini a medio rango ottimali per l'indicizzazione, e la coda degli hapax legomena.

Definizione 3.2.1 (Legge di Zipf)**.** Ordinando i termini di un corpus per frequenza decrescente, il termine a rango r ha frequenza empirica f inversamente proporzionale a r:

$$f \propto \frac{1}{r} \implies f \cdot r \approx C, \tag{3.5}$$

dove C è una costante caratteristica del corpus testuale.

Nella generalizzazione di Mandelbrot (*Zipf-Mandelbrot law*):

$$f(r) \propto \frac{1}{(r+b)^a}, \tag{3.6}$$

con parametri empirici stimati tipicamente a $a \approx 1$ e $b \approx 2.7$.

Principio del Minimo Sforzo (*Principle of Least Effort*)

Zipf postulò che questa legge emerga dall'equilibrio economico tra due tendenze cognitive in conflitto:

1. **Sforzo del Parlante (*Speaker*):** mira a minimizzare il dispendio articolatorio impiegando un vocabolario ristretto, con parole brevi, ad altissima frequenza e fortemente polisemiche.

2. **Sforzo dell'Ascoltatore (*Hearer*):** predilige un lessico ampio e diversificato, formato da termini specifici e rari, così da annullare l'ambiguità e il costo di disambiguazione concettuale.

Il compromesso evolutivo tra queste esigenze produce l'andamento iperbolico osservato.

Dalla formulazione di Zipf discendono due proprietà verificabili empiricamente:

- Nel corpus giornalistico standard *Reuters 21578* (circa 500 000 occorrenze totali), appena **313 parole uniche** coprono oltre il 50% di tutte le occorrenze testuali della collezione.

- Il numero di significati lessicali distinti m di una parola è inversamente proporzionale alla radice della sua frequenza: $m \propto 1/\sqrt{f}$.

3.2.3 Parole Rare e Stopwords

Dalla distribuzione di Zipf si ricava una polarizzazione del vocabolario:

1. **Stopwords:** parole a frequenza altissima e rango minimo ($r \leq 100$), costituite da particelle grammaticali, articoli, preposizioni e congiunzioni (*the*, *of*, *a*, *in*, *to*). Presentano un contenuto informativo quasi nullo per identificare il tema di un documento. Tuttavia, la loro rimozione drastica può danneggiare compiti di ricerca esatta (es. titoli o espressioni idiomatiche come *"to be or not to be"*) e analisi del sentimento.

2. **Parole Rare (Hapax Legomena):** vocaboli che compaiono una sola volta (o pochissime volte) nel corpus. Costituiscono tipicamente il 40%–50% dell'intero vocabolario e comprendono errori tipografici (*typos*), slug tecnici, codici alfanumerici, o termini rari con altissimo potere discriminante.

3.3 Acquisizione e Preprocessing del Testo

La trasformazione di una collezione di documenti eterogenei non strutturati in una rappresentazione computabile richiede una sequenza rigorosa di operazioni di normalizzazione e pulizia. Questa sezione descrive le fasi della pipeline di preprocessing: dal recupero e filtraggio del markup HTML, alla tokenizzazione e alla riduzione morfologica tramite stemming e lemmatizzazione.

3.3.1 Estrazione del Testo dal Web

Il flusso di indicizzazione ha inizio dal recupero dei dati non strutturati tramite protocolli di rete:

1. **Recupero del documento (I/O di rete):** la libreria `urllib.request` consente di inviare richieste HTTP e scaricare il markup grezzo:

Listing 3.1: Scaricamento di pagine HTML con urllib.

```
from urllib import request

url = "http://example.org/page.html"
response = request.urlopen(url)
html = response.read().decode('utf8')
```

2. **HTML Stripping e Isolamento del Contenuto:** la presenza di tag HTML di formattazione, metadati e script introdurrebbe rumore non informativo. L'impiego di parser dedicati quali `BeautifulSoup` consente di estrarre selettivamente il testo:

Listing 3.2: Estrazione del testo pulito tramite BeautifulSoup.

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html, "html5lib")
# Estrazione globale del testo grezzo non
   strutturato
raw_text = soup.get_text()

# Estrazione selettiva e controllata dai blocchi
   paragrafo
paragraphs = [' '.join(s.findAll(text=True)) for s
   in soup.findAll('p')]
```

3.3.2 Segmentazione e Tokenizzazione

Sentence Splitting (Divisione in Frasi). Identificare i confini di frase (*sentence boundaries*) non coincide banalmente con la ricerca del carattere punto (`.`). Il punto è impiegato negli acronimi (*U.S.A.*), nelle abbreviazioni di titoli e toponimi (*Dr.*, *St.*), nei numeri decimali (*8.30*) e nei punti di sospensione. L'algoritmo di NLTK `sent_tokenize()` ricorre a un classificatore addestrato a distinguere i ruoli contestuali della punteggiatura.

Word Tokenization (Tokenizzazione dei Termini). Consiste nella scomposizione del flusso continuo di caratteri in unità lessicali minime (*tokens*):

Listing 3.3: Tokenizzazione standard con NLTK.

```
1 from nltk import word_tokenize
2
3 tokens = word_tokenize("The President of the United
     States of America")
4 # Output: ['The', 'President', 'of', 'the', 'United', '
     States', 'of', 'America']
```

In casi specifici si impiegano tokenizer ad hoc (es. `TweetTokenizer` per emoji, menzioni e hashtag, o `RegexpTokenizer` per pattern ad espressione regolare).

3.3.3 Normalizzazione Morfologica: Stemming vs. Lemmatizzazione

Lo scopo primario della normalizzazione morfologica è ricondurre le varianti flessive e derivazionali dei termini verso una rappresentazione canonica comune, contraendo la dimensionalità dello spazio lessicale.

Stemming

Procedura puramente algoritmica basata su euristiche di manipolazione di stringhe (taglio o rimpiazzo di prefissi e suffissi):

- **Caratteristiche:** rapidissimo nell'esecuzione, agisce su singole parole senza richiedere dizionari completi o analisi contestuale.
- **Criticità:** produce frequentemente pseudo-radici inesistenti nel vocabolario reale (*overstemming* e *understemming*).
- **Esempio (Porter Stemmer):**

  - `"cars"` → `"car"` (rimozione corretta del plurale).
  - `"was"` → `"wa"` (l'algoritmo interpreta erroneamente la lettera finale `s` come desinenza del plurale).

Lemmatizzazione

Processo morfologico rigoroso finalizzato a individuare il lemma base canonico (la voce di dizionario) del vocabolo:

- **Caratteristiche:** sfrutta la disambiguazione grammaticale (*Part-of-Speech Tagging*) e si appoggia a basi lessicali complete (come *WordNet*).
- **Esempio (`WordNetLemmatizer`):**
  - Senza specificare il POS (default sostantivo): `lemmatize("was")` restituisce `"wa"`.
  - Indicando il POS verbale: `lemmatize("was", pos='v')` restituisce la forma base canonica `"be"`.

3.4 Modelli di Rappresentazione: Bag of Words e N-Grammi

Completata la fase di normalizzazione dei termini, il testo deve essere convertito in strutture dati formali adatte alla quantificazione e al calcolo algebrico. Questa sezione analizza il classico modello *Bag of Words* (BoW), evidenziandone i limiti strutturali derivanti dalla perdita dell'ordine delle parole, e introduce le rappresentazioni basate su N-grammi di parole e di caratteri.

3.4.1 Il Modello Bag of Words (BoW)

Il modello Bag of Words astrae il testo rimuovendo interamente l'ordine sintattico originario e le relazioni grammaticali, trattando il documento come un multiset (o insieme) di token atomici indipendenti:

Listing 3.4: Rappresentazione insiemistica di base.

```
from nltk import word_tokenize

text = 'the president of the united states of america'
tokens = word_tokenize(text)  # Lunghezza: 8 token
bow_set = set(tokens)         # Cardinalita: 6 elementi
   unici
```

```
6 # bow_set: {'america', 'of', 'president', 'states', 'the
    ', 'united'}
```

Limiti Strutturali del Bag of Words

1. **Perdita totale dell'ordine sintattico e composizionale:** Si considerino due frasi con significato diametralmente opposto:

   - t_1: *"I won, and thus you lose."* (affermazione di vittoria)
   - t_2: *"I lose, and thus you won."* (ammissione di sconfitta)

   Sotto la rappresentazione insiemistica del modello BoW:

   $$\text{set}(t_1) \equiv \text{set}(t_2) = \{\text{'and'}, \text{'lose'}, \text{'thus'}, \text{'won'}, \text{'you'}\}. \tag{3.7}$$

   Per il modello di calcolo, i due enunciati risultano indistinguibili, provocando un vero e proprio **collasso semantico**.

2. **Perdita della molteplicità (nella variante insiemistica binaria):** non tiene traccia dell'intensità con cui una parola ricorre nel testo.

3.4.2 I Modelli N-grammi

Per mitigare la perdita dell'ordinamento senza sostenere i costi computazionali del parsing sintattico profondo, si estende il vocabolario con gli **n-grammi** (sequenze contigue di n elementi).

Word N-grams (N-grammi di Parole)

Estrae tuple di n parole consecutive ($n = 2$ per bigrammi, $n = 3$ per trigrammi), preservando la sintassi locale:

- Applicato all'esempio precedente ($n = 2$):

  $$\text{ngrams}(t_1, 2) = \{\text{'I won'}, \text{'won and'}, \text{'and thus'}, \text{'thus you'}, \text{'you lose'}\}, \tag{3.8}$$

  $$\text{ngrams}(t_2, 2) = \{\text{'I lose'}, \text{'lose and'}, \text{'and thus'}, \text{'thus you'}, \text{'you won'}\}. \tag{3.9}$$

  I due insiemi differiscono radicalmente ($\text{ngrams}(t_1, 2) \neq \text{ngrams}(t_2, 2)$), consentendo al modello di distinguere il vincitore dal perdente.

Character N-grams (N-grammi di Caratteri)

Estrae sottostringhe di n caratteri consecutivi all'interno dei singoli lemmi (tipicamente $n = 3$):

- **Funzione nell'IR:** conferisce tolleranza agli errori ortografici di digitazione (*typo-tolerance*) e rileva affinità morfologiche.
- **Esempio di trigrammi su vocaboli affetti da refuso** (`"rainbow"` vs. `"rainbaw"`):

$$C_3(\text{"rainbow"}) = \{\text{'r-a-i', 'a-i-n', 'i-n-b', 'n-b-o', 'b-o-w'}\}, \tag{3.10}$$
$$C_3(\text{"rainbaw"}) = \{\text{'r-a-i', 'a-i-n', 'i-n-b', 'n-b-a', 'b-a-w'}\}. \tag{3.11}$$

  L'intersezione è non vuota:

$$C_3(\text{"rainbow"}) \cap C_3(\text{"rainbaw"}) = \{\text{'r-a-i', 'a-i-n', 'i-n-b'}\}. \tag{3.12}$$

  Poiché condividono 3 trigrammi su 5 totali (60%), il motore può identificare la somiglianza e suggerire correzioni automatiche (*"Forse cercavi: rainbow"*).

Capitolo 4

Modelli di Retrieval, Scoring e Ponderazione TF-IDF

4.1 Evoluzione dei Paradigmi di Information Retrieval

La storia dell'Information Retrieval si articola attraverso quattro grandi ere architetturali:

1. **Era Statistica (Anni '60 – Metà anni '90):** i motori computavano la rilevanza poggiando unicamente sulla frequenza statistica dei termini all'interno dei documenti (tf) e del corpus (idf). Questa impostazione soffriva di estrema permeabilità al *keyword stuffing*: era sufficiente ripetere artificialmente parole chiave nel corpo della pagina web per scalarne indebitamente la graduatoria.

2. **Era Strutturale e PageRank (1998):** Sergey Brin e Larry Page affrontano il collasso del ranking puramente statistico sfruttando la topologia del Web Graph. Un collegamento ipertestuale in ingresso (*incoming link*) funge da voto di autorevolezza conferito dalla pagina sorgente. Modificare la struttura del grafo globale è esponenzialmente più costoso rispetto all'alterazione del testo locale.

3. **Era del Machine Learning e Learning-to-Rank (Anni 2000):** la rilevanza viene stimata aggregando centinaia di segnali disomogenei tramite modelli supervisionati: salienza testuale, reputazione topologica, e metriche comportamentali dell'utente (CTR, frequenza di riformulazione, Dwell Time).

4. **Era Neurale e dei Modelli Densi (2018 – Oggi):** l'avvento dell'architettura Transformer consente la cattura contestuale profonda della semantica. I documenti vengono proiettati in spazi di incorporamento continuo (*dense embeddings*) o tramite rappresentazioni sparse apprese (*learned sparse representations*).

4.2 Boolean Retrieval vs. Ranked Retrieval

Il confronto sistematico tra il modello logico tradizionale e il paradigma moderno a punteggio graduato evidenzia differenze profonde:

Proprietà	**Boolean Retrieval**	**Ranked Retrieval**
Criterio di Matching	Esatto, proposizionale e binario ($d \models q \implies \{0,1\}$).	Continuo e scalare su numeri reali: $\text{Score}(q,d) \in \mathbb{R}$.
Formulazione Query	Espressioni logiche vincolanti (`AND`, `OR`, `NOT`).	Linguaggio naturale libero (*free-text query*).
Output fornito	Insieme non ordinato di documenti conformi.	Lista ordinata in modo decrescente per score stimato.
Esperienza Utente	Sindrome di *Feast or Famine* (Banchetto o Carestia).	Restituzione istantanea dei primi k elementi (top-k, tipicamente $k \approx 10$).
Destinatari Tipici	Utenti specialisti (documentalisti, legali, brevettisti).	Utenti generici del Web.

Tabella 4.1: Confronto analitico tra Boolean Retrieval e Ranked Retrieval.

La Patologia del "Feast or Famine"

L'interazione ordinaria con il modello booleano si rivela ostica a causa di due estremi indesiderati:

- **Carestia (*Famine*):** l'uso dell'operatore congiuntivo `AND` restringe rigidamente i vincoli. Se un solo termine della query manca nel testo (ad esempio per una lieve variazione sinonimica), il documento viene escluso, producendo frequentemente zero risultati.
- **Banchetto (*Feast*):** l'operatore disgiuntivo `OR` rilassa eccessivamente la selezione, restituendo decine di migliaia di documenti senza alcuna prioritizzazione qualitativa.

4.3 Misure di Similarità Set-Based: Il Coefficiente di Jaccard

Un primo tentativo di formalizzare un ordinamento continuo senza impiegare algebra vettoriale complessa consiste nell'impiego del coefficiente di sovrapposizione insiemistica di Jaccard:

Definizione 4.3.1 (Coefficiente di Jaccard)**.** Dati due insiemi di termini A e B, la somiglianza di Jaccard è il rapporto tra la cardinalità della loro intersezione e la cardinalità della loro unione:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}. \tag{4.1}$$

Proprietà: è confinato nell'intervallo $[0,1]$, dove $J(A,A) = 1$ e $J(A,B) = 0 \iff A \cap B = \emptyset$.

4.3.1 Esempio Guidato Svolto a Lezione e il Paradosso della Lunghezza

Si consideri la query q e due documenti candidati:

- **Query** q**:** *"ides of march"*
  $Q = \{\text{ides}, \text{of}, \text{march}\}$, $|Q| = 3$.
- **Documento** d_1**:** *"caesar died in march"*
  $D_1 = \{\text{caesar}, \text{died}, \text{in}, \text{march}\}$, $|D_1| = 4$.

- **Documento d_2:** *"the long march"*
  $D_2 = \{\text{the}, \text{long}, \text{march}\}$, $|D_2| = 3$.

Calcolo analitico:

$$Q \cap D_1 = \{\text{march}\} \implies |Q \cap D_1| = 1, \tag{4.2}$$

$$J(q, d_1) = \frac{1}{3+4-1} = \frac{1}{6} \approx 0.166. \tag{4.3}$$

$$Q \cap D_2 = \{\text{march}\} \implies |Q \cap D_2| = 1, \tag{4.4}$$

$$J(q, d_2) = \frac{1}{3+3-1} = \frac{1}{5} = 0.200. \tag{4.5}$$

Il Paradosso di Rilevanza nel Coefficiente di Jaccard

Poiché $J(q, d_2) = 0.200 > J(q, d_1) = 0.166$, l'algoritmo decreta che d_2 sia più rilevante di d_1. Tuttavia, sotto il profilo semantico e storico, d_1 tratta esattamente della morte di Cesare (avvenuta alle idi di marzo), mentre d_2 descrive un evento storico disconnesso (la "lunga marcia" cinese). Il documento d_2 riceve uno score superiore unicamente perché è più breve, subendo una minore espansione del denominatore $|A \cup B|$.

4.3.2 Limiti Strutturali del Coefficiente di Jaccard per l'IR

1. **Ignora la frequenza locale (tf):** un termine presente 50 volte in un testo viene contato nell'intersezione esattamente come un termine presente una volta sola.

2. **Ignora la rarità globale (df):** vocaboli frequentissimi come la preposizione *"of"* godono del medesimo peso di termini rari e discriminanti come *"ides"*.

3. **Penalizzazione ingiustificata dei documenti ampi:** testi esaustivi e pertinenti che contengono molti termini addizionali subiscono un rigonfiamento dell'unione, vedendo collassare artificiosamente il proprio punteggio.

4.4 Dalle Matrici di Incidenza alle Matrici di Conteggio

Per superare la rigidità insiemistica, la modellazione documentale evolve verso l'algebra lineare.

4.4.1 Matrice Binaria Termine-Documento (Incidence Matrix)

Dato un vocabolario $V = \{t_1, \ldots, t_{|V|}\}$ e un corpus $D = \{d_1, \ldots, d_N\}$, la matrice di incidenza $M \in \{0,1\}^{|V| \times N}$ mappa la presenza binaria:

$$M_{t,d} = \begin{cases} 1 & \text{se } t \in d, \\ 0 & \text{altrimenti.} \end{cases} \tag{4.6}$$

4.4.2 Matrice di Conteggio Termine-Documento (Count Matrix)

Riconoscendo che la frequenza di occorrenza riflette il grado di enfasi tematica, si passa alla matrice dei conteggi grezzi $C \in \mathbb{N}^{|V| \times N}$ in cui $C_{t,d} = \text{count}(t,d) = tf_{t,d}$.

Termine	Antony & Cleo.	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Tabella 4.2: Matrice di conteggio dei termini su una selezione di opere di Shakespeare.

4.5 Ponderazione della Frequenza dei Termini: Log-Frequency Weighting

La semplice presenza o il conteggio lineare delle occorrenze di una parola non riflettono fedelmente il valore informativo del testo. Questa sezione esamina i

limiti della frequenza grezza e formalizza la trasformazione logaritmica (wf) come meccanismo matematico per modellare il principio dell'utilità marginale decrescente.

4.5.1 Perché il Conteggio Lineare (tf) è Inadeguato

La rilevanza semantica percepita dall'utente non cresce in modo proporzionale al conteggio puro delle parole:

- Un testo in cui un termine compare 10 volte è plausibilmente più attinente di uno in cui compare 1 volta sola, ma certamente **non è** 10 **volte più rilevante**.
- Ogni occorrenza successiva dello stesso termine apporta un incremento marginale di informazione progressivamente decrescente (*legge dei rendimenti decrescenti*).

4.5.2 Formulazione Logaritmica di tf

Per smorzare l'effetto di frequenze grezze sproporzionate, si introduce la trasformazione logaritmica:

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{se } tf_{t,d} > 0, \\ 0 & \text{altrimenti.} \end{cases} \tag{4.7}$$

Progressione dei Pesi:

- $tf = 0 \implies w = 0$
- $tf = 1 \implies w = 1 + \log_{10}(1) = 1.0$
- $tf = 2 \implies w = 1 + \log_{10}(2) \approx 1.301$
- $tf = 10 \implies w = 1 + \log_{10}(10) = 2.0$
- $tf = 100 \implies w = 1 + \log_{10}(100) = 3.0$
- $tf = 1000 \implies w = 1 + \log_{10}(1000) = 4.0$

In contesti computazionali continui, si adotta frequentemente la variante continua $\log(1 + tf_{t,d})$, che evita la discontinuità in zero poiché $\log(1 + 0) = 0$.

Se il ranking viene calcolato unicamente in base alla frequenza dei termini condivisi:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} (1 + \log_{10}(tf_{t,d})) \,. \tag{4.8}$$

4.6 Document Frequency (df) e Inverse Document Frequency (idf)

Accanto alla frequenza locale del termine all'interno del singolo documento, è necessario quantificare la rarità complessiva della parola nell'intero corpus di riferimento. Questa sezione formalizza la metrica dell'*Inverse Document Frequency* (IDF), ne analizza le proprietà matematiche di neutralizzazione delle parole vuote e ne illustra l'impatto sul ranking delle query composte da più termini.

4.6.1 L'Informatività Semantica e la Rarità Globale

La salienza locale di una parola all'interno di un documento (tf) non ne misura la specificità tematica nella collezione globale:

- Un termine ubiquo presente in quasi tutti i documenti (es. *increase*, *high*, o le congiunzioni) possiede un potere discriminante pressoché nullo.
- Un termine presente in un numero ristrettissimo di testi (es. *Calpurnia*, *arachnocentric*) possiede un altissimo potere discriminante: la sua comparsa segnala una pertinenza probabilistica quasi certa.

4.6.2 Definizione Matematica dell'IDF

Sia $N = |D|$ la dimensione totale della collezione e sia df_t (*Document Frequency*) il numero di documenti contenenti almeno un'occorrenza del termine t. L'**Inverse Document Frequency (IDF)** è formalizzato come:

$$idf_t = \log_{10}\left(\frac{N}{df_t}\right). \tag{4.9}$$

Il logaritmo smorza l'ampiezza dell'argomento N/df_t, evitando che su corpus di milioni di pagine i pesi assumano valori ingestibili.

4.6.3 Esempio Completo Svolto a Lezione ($N = 1\,000\,000$)

Si consideri un corpus composto da un milione di documenti:

Termine (t)	df_t	**Rapporto** N/df_t	$idf_t = \log_{10}(N/df_t)$
Calpurnia	1	10^6	6.0
animal	100	10^4	4.0
sunday	1 000	10^3	3.0
fly	10 000	10^2	2.0
under	100 000	10^1	1.0
the	1 000 000	1	0.0

Tabella 4.3: Calcolo analitico dell'IDF al variare della Document Frequency su $N = 10^6$.

Proprietà Chiave dell'IDF

1. **Neutralizzazione automatica dei termini onnipresenti:** se una parola compare in ogni documento ($df_t = N$), si ha $idf_t = \log_{10}(1) = 0$. Il termine si auto-annulla nel calcolo del punteggio, agendo come filtro naturale per le stopwords.

2. **Precomputazione Offline Disaccoppiata:** il valore idf_t dipende unicamente dalla statistica della collezione e **non dalla query**. Di conseguenza, tutti gli idf_t vengono calcolati offline durante l'indicizzazione e memorizzati nelle tabelle del vocabolario, richiedendo solo un tempo di accesso $\mathcal{O}(1)$ a tempo di query.

4.6.4 L'Effetto dell'IDF sull'Ordinamento: Query a Termine Singolo vs. Multi-Termine

Per query monoremiche ($|q| = 1$). L'IDF **non altera l'ordinamento relativo** tra i documenti.

Dimostrazione. Sia la query $q = \{t\}$. Lo score ponderato da solo IDF per un documento d che contiene t è:

$$\text{Score}(q, d) = \log_{10}\left(\frac{N}{df_t}\right). \tag{4.10}$$

Poiché questo valore è identico per qualunque documento contenente t, l'IDF agisce come una costante moltiplicativa uniforme. L'ordinamento dipenderà unicamente dal termine $tf_{t,d}$. □

Per query multi-termine ($|q| \geq 2$). L'IDF diviene il **fattore cardine di ri-pesatura**: in una query come *"capricious person"*, il termine generico *"person"* (basso idf) viene pesato pochissimo, mentre il termine raro *"capricious"* (alto idf) domina la graduatoria, premiando i documenti mirati.

4.6.5 Collection Frequency (cf_t) vs. Document Frequency (df_t)

- **Collection Frequency (cf_t):** numero totale di occorrenze di t nel corpus: $cf_t = \sum_d tf_{t,d}$.
- **Document Frequency (df_t):** numero di documenti unici contenenti t almeno una volta.

Esempio 4.6.1 (Confronto tra "insurance" e "try")**.** Nel corpus d'esame:

- *"try"*: $cf = 10\,422$, distribuito su $df = 8\,760$ documenti.
- *"insurance"*: $cf = 10\,440$, concentrato in soli $df = 3\,997$ documenti.

Nonostante la frequenza complessiva cf sia pressoché identica, *"insurance"* è nettamente più informativa e discriminante, e riceve un idf marcatamente superiore.

4.7 Lo Schema di Ponderazione TF-IDF Completo

L'integrazione congiunta della frequenza logaritmica locale (tf) e della rarità inversa globale (idf) converge nello schema di pesatura più celebre e influente dell'Information Retrieval classico. Questa sezione presenta la formalizzazione del peso TF-IDF, ne sviluppa una simulazione analitica dettagliata e introduce la matrice termine–documento ponderata.

4.7.1 Formulazione Matematica

La combinazione tra salienza locale del termine (tf) e specificità globale nel corpus (idf) definisce il peso **tf-idf**:

$$w_{t,d} = \text{tf-idf}_{t,d} = wf_{t,d} \times idf_t, \tag{4.11}$$

dove:

$$wf_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & \text{se } tf_{t,d} > 0, \\ 0 & \text{altrimenti,} \end{cases} \qquad idf_t = \log_{10}\left(\frac{N}{df_t}\right). \tag{4.12}$$

Lo score preliminare tra una query non ponderata q e un documento d è:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf-idf}_{t,d}. \tag{4.13}$$

4.7.2 Esercizio Numerico Svolto alla Lavagna

Dati di partenza: $N = 806\,791$, Query: *"car auto insurance best"*.

Termine (t)	tf **Doc 1**	tf **Doc 2**	tf **Doc 3**	df_t	$idf_t = \log_{10}(N/df_t)$
car	27	4	24	18 165	1.65
auto	3	33	0	6 723	2.08
insurance	0	33	29	19 241	1.62
best	14	0	17	25 235	1.50

Tabella 4.4: Dati dell'esercizio alla lavagna: conteggi locali e statistiche di collezione.

Caso A: Calcolo con conteggio grezzo ($tf_{t,d} \times idf_t$). Per **Doc 1**:

- car: $27 \times 1.65 = 44.55$
- auto: $3 \times 2.08 = 6.24$
- insurance: $0 \times 1.62 = 0.0$
- best: $14 \times 1.50 = 21.0$

Per **Doc 2**:

- car: $4 \times 1.65 = 6.60$
- auto: $33 \times 2.08 = 68.64$
- insurance: $33 \times 1.62 = 53.46$
- best: $0 \times 1.50 = 0.0$

Per **Doc 3**:

- car: $24 \times 1.65 = 39.60$

- auto: $0 \times 2.08 = 0.0$
- insurance: $29 \times 1.62 = 46.98$
- best: $17 \times 1.50 = 25.50$

Caso B: Calcolo con smorzamento logaritmico $\log_{10}(1 + tf_{t,d}) \times idf_t$**.** Applicando la compressione logaritmica su **Doc 1**:

- car: $\log_{10}(1 + 27) = \log_{10}(28) \approx 1.447 \implies 1.447 \times 1.65 \approx \mathbf{2.39}$
- auto: $\log_{10}(1 + 3) = \log_{10}(4) \approx 0.602 \implies 0.602 \times 2.08 \approx \mathbf{1.25}$
- insurance: $\log_{10}(1 + 0) = 0 \implies 0 \times 1.62 = \mathbf{0.0}$
- best: $\log_{10}(1 + 14) = \log_{10}(15) \approx 1.176 \implies 1.176 \times 1.50 \approx \mathbf{1.76}$

Lo smorzamento logaritmico impedisce ai singoli conteggi estremi di sopraffare la rilevanza informativa globale veicolata dall'IDF.

4.7.3 La Matrice Termine-Documento Ponderata

Sostituendo i valori di conteggio con i pesi reali tf-idf$_{t,d}$, la collezione documentale viene proiettata in una matrice reale:

$$W \in \mathbb{R}^{|V| \times N}. \tag{4.14}$$

Ciascun documento della collezione corrisponde ora a un vettore geometrico $\vec{d} \in \mathbb{R}^{|V|}$, ponendo le fondamenta per l'analisi geometrica dello spazio vettoriale.

Capitolo 5

Il Modello a Spazio Vettoriale (VSM), la Notazione SMART e Okapi BM25

5.1 Fondamenti del Vector Space Model (VSM)

Nel **Vector Space Model** (VSM), la collezione documentale e le query dell'utente vengono modellate come punti e vettori geometrici immersi in uno spazio vettoriale reale a dimensionalità finita:

$$\text{Dimensione dello spazio} = |V|, \tag{5.1}$$

dove ogni asse ortogonale corrisponde a un termine unico del vocabolario V.

Sia la query q sia il documento d sono rappresentati come vettori multidimensionali:

$$\vec{v}(q) \in \mathbb{R}^{|V|}, \qquad \vec{v}(d) \in \mathbb{R}^{|V|}. \tag{5.2}$$

5.1.1 Sparsità Estrema dei Vettori Reali

In contesti applicativi reali, la cardinalità del vocabolario $|V|$ raggiunge facilmente l'ordine di 10^6 vocaboli. Un tipico documento web contiene da qualche centinaio a poche migliaia di parole distinte:

$$\frac{|\{i \mid v_i(d) \neq 0\}|}{|V|} \leq \frac{10^3}{10^6} = 0.001 = 0.1\%. \tag{5.3}$$

Oltre il **99.9%** delle coordinate di ciascun vettore documentale è identicamente pari a zero. Pertanto, la memorizzazione e le operazioni algebriche vengono

implementate tramite strutture dati per vettori sparsi (*sparse vectors*) e liste d'inversione (*posting lists*).

5.2 Perché la Distanza Euclidea Fallisce: L'Esperimento di Duplicazione

La misura geometrica più naturale per quantificare la prossimità tra due vettori nello spazio euclideo è la distanza euclidea (L_2):

$$\text{dist}_{L_2}(\vec{q}, \vec{d}) = \|\vec{q} - \vec{d}\|_2 = \sqrt{\sum_{i=1}^{|V|}(q_i - d_i)^2}. \tag{5.4}$$

Tuttavia, tale metrica risulta inadatta per l'Information Retrieval a causa della sensibilità alla lunghezza estrinseca del testo.

5.2.1 L'Esperimento Mentale del Documento Concatenato ($d' = d \circ d$)

Si consideri un documento d e se ne crei una replica concatenata a se stessa, denominata d', in cui ogni enunciato è ripetuto due volte:

1. **Contenuto Semantico:** d' e d sono identici sotto il profilo del contenuto informativo e della distribuzione tematica.

2. **Magnitudo del Vettore:** ogni frequenza locale raddoppia ($tf_{t,d'} = 2 \cdot tf_{t,d}$), provocando un raddoppio della norma del vettore ($\|\vec{d'}\|_2 \approx 2\|\vec{d}\|_2$).

3. **Esplosione della Distanza Euclidea:** la distanza euclidea tra la query $\vec{q}$ (tipicamente corta) e $\vec{d'}$ diviene notevolmente maggiore rispetto a quella tra $\vec{q}$ e $\vec{d}$:
$$\|\vec{q} - \vec{d'}\|_2 \gg \|\vec{q} - \vec{d}\|_2. \tag{5.5}$$
Ordinando per distanza euclidea crescente, il documento più esaustivo d' viene fortemente penalizzato.

4. **Direzione Angolare Immutata:** entrambi i vettori $\vec{d}$ e $\vec{d'}$ giacciono **esattamente sulla stessa semiretta** uscente dall'origine degli assi. L'angolo compreso θ tra di essi è rigorosamente pari a $0°$.

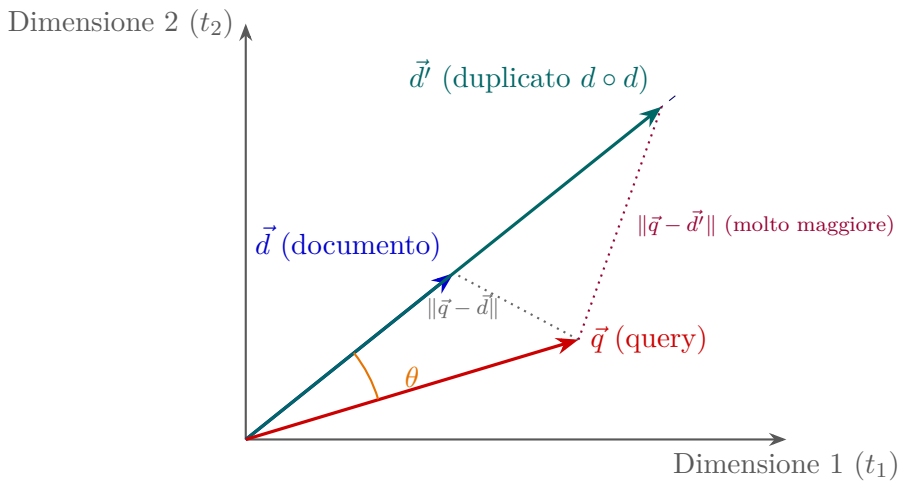


Figura 5.1: Fallimento della distanza euclidea: d e d' condividono la stessa direzione ($\theta = 0°$), ma d' ha distanza euclidea molto maggiore da q.

Direzione vs. Magnitudo

La vicinanza tematica tra un testo e un bisogno informativo deve essere invariante rispetto alla prolissità del testo. La rilevanza risiede nella **direzione angolare** dei vettori, non nella lunghezza delle frecce nello spazio.

5.3 Dagli Angoli alla Cosine Similarity e Normalizzazione L_2

Ordinare i documenti per angolo θ crescente (angolo piccolo $\implies$ alta affinità tematica) equivale a ordinarli per **coseno decrescente** di θ, poiché nell'intervallo $[0, \pi/2]$ la funzione coseno è strettamente decrescente:

$$\theta = 0° \implies \cos(0°) = 1 \quad \text{(massima affinità, vettori collineari)}, \tag{5.6}$$

$$\theta = 90° \implies \cos(90°) = 0 \quad \text{(vettori ortogonali, nessun termine comune)}. \tag{5.7}$$

Poiché i pesi TF-IDF sono non negativi, lo spazio di ricerca è interamente confinato al primo iper-ottante positivo: $\cos(\theta) \in [0, 1]$.

5.3.1 Derivazione Analitica della Cosine Similarity

Dalla definizione geometrica del prodotto scalare (*dot product*):

$$\vec{q} \cdot \vec{d} = \|\vec{q}\|_2 \|\vec{d}\|_2 \cos(\theta), \tag{5.8}$$

dove la norma euclidea (L_2) è:

$$\|\vec{x}\|_2 = \sqrt{\sum_{i=1}^{|V|} x_i^2}. \tag{5.9}$$

Esplicitando $\cos(\theta)$, si ottiene la formula cardine della **Cosine Similarity**:

$$\text{Sim}_{\cos}(\vec{q}, \vec{d}) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|_2 \|\vec{d}\|_2} = \frac{\sum_{t \in q \cap d} q_t d_t}{\sqrt{\sum_{t \in q} q_t^2} \sqrt{\sum_{t \in d} d_t^2}}. \tag{5.10}$$

5.3.2 Normalizzazione L_2 e Proiezione sull'Ipersfera Unitaria

Dividere un vettore per la propria norma euclidea equivale a proiettarlo sulla superficie dell'**ipersfera unitaria** multidimensionale:

$$\vec{v}_{\text{norm}} = \frac{\vec{v}}{\|\vec{v}\|_2} \implies \|\vec{v}_{\text{norm}}\|_2 = 1. \tag{5.11}$$

Quando tutti i vettori sono preventivamente normalizzati a lunghezza unitaria, la similarità coseno collassa in un banale prodotto scalare:

$$\cos(\vec{q}, \vec{d}) = \vec{q}_{\text{norm}} \cdot \vec{d}_{\text{norm}} = \sum_{t \in q \cap d} q_{\text{norm},t} \cdot d_{\text{norm},t}. \tag{5.12}$$

5.3.3 Architettura di Calcolo: Disaccoppiamento Offline vs. Online

- **Fase Offline (Indicizzazione):** la norma $\|\vec{d}\|_2 = \sqrt{\sum_t d_t^2}$ dipende unicamente dal documento. Viene calcolata una sola volta per ciascun documento e memorizzata in una tabella d'accesso diretto (o nel doc-index).
- **Fase Online (Query Processing):** all'arrivo della query q, si computa $\|\vec{q}\|_2$. Durante la scansione congiunta delle posting list, il motore accumula il prodotto scalare parziale sui soli termini condivisi $t \in q \cap d$ e normalizza il punteggio finale:

$$\text{Score}(q, d) = \frac{1}{\|\vec{q}\|_2} \sum_{t \in q \cap d} \frac{q_t d_t}{\|\vec{d}\|_2}. \tag{5.13}$$

5.4 Esercizio Pratico Completo: Confronto Stilistico tra Tre Romanzi

Il docente ha guidato l'analisi comparativa tra tre capolavori della letteratura inglese:

1. **SaS:** *Sense and Sensibility* (Jane Austen, 1811)
2. **PaP:** *Pride and Prejudice* (Jane Austen, 1813)
3. **WH:** *Wuthering Heights* (Emily Brontë, 1847)

L'obiettivo è verificare se la geometria del VSM riconosca una vicinanza stilistica superiore tra le due opere della medesima autrice rispetto all'opera di un'autrice differente.

5.4.1 Passo 1: Matrice dei Conteggi Grezzi (tf)

Termine	**SaS** (d_1)	**PaP** (d_2)	**WH** (d_3)
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Tabella 5.1: Frequenze grezze dei termini nei tre romanzi.

5.4.2 Passo 2: Ponderazione Log-TF ($1 + \log_{10}(tf)$)

Applicando la trasformazione sublineare $w = 1 + \log_{10}(tf)$ se $tf > 0$, e 0 altrimenti:

- **SaS:**
  - affection: $1 + \log_{10}(115) \approx 1 + 2.0607 = 3.06$
  - jealous: $1 + \log_{10}(10) = 1 + 1.0 = 2.00$
  - gossip: $1 + \log_{10}(2) \approx 1 + 0.3010 = 1.30$
  - wuthering: 0.00
- **PaP:**
  - affection: $1 + \log_{10}(58) \approx 1 + 1.7634 = 2.76$

- jealous: $1 + \log_{10}(7) \approx 1 + 0.8451 = 1.85$
- gossip: 0.00
- wuthering: 0.00

- **WH:**
  - affection: $1 + \log_{10}(20) \approx 1 + 1.3010 = 2.30$
  - jealous: $1 + \log_{10}(11) \approx 1 + 1.0414 = 2.04$
  - gossip: $1 + \log_{10}(6) \approx 1 + 0.7782 = 1.78$
  - wuthering: $1 + \log_{10}(38) \approx 1 + 1.5798 = 2.58$

5.4.3 Passo 3: Calcolo delle Norme Euclidee e Vettori Unitari

Calcolo delle norme L_2:

$$\|\vec{v}_{\text{SaS}}\|_2 = \sqrt{3.06^2 + 2.00^2 + 1.30^2 + 0^2} = \sqrt{15.0536} \approx \mathbf{3.88}, \tag{5.14}$$

$$\|\vec{v}_{\text{PaP}}\|_2 = \sqrt{2.76^2 + 1.85^2 + 0^2 + 0^2} = \sqrt{11.0401} \approx \mathbf{3.32}, \tag{5.15}$$

$$\|\vec{v}_{\text{WH}}\|_2 = \sqrt{2.30^2 + 2.04^2 + 1.78^2 + 2.58^2} = \sqrt{19.2764} \approx \mathbf{4.39}. \tag{5.16}$$

Dividendo ciascuna colonna per la relativa norma, si ottengono i vettori unitari normalizzati:

Termine	**SaS** ($\vec{v}/3.88$)	**PaP** ($\vec{v}/3.32$)	**WH** ($\vec{v}/4.39$)
affection	$3.06/3.88 \approx \mathbf{0.789}$	$2.76/3.32 \approx \mathbf{0.832}$	$2.30/4.39 \approx \mathbf{0.524}$
jealous	$2.00/3.88 \approx \mathbf{0.515}$	$1.85/3.32 \approx \mathbf{0.555}$	$2.04/4.39 \approx \mathbf{0.465}$
gossip	$1.30/3.88 \approx \mathbf{0.335}$	$0.00/3.32 = \mathbf{0.000}$	$1.78/4.39 \approx \mathbf{0.405}$
wuthering	$0.00/3.88 = \mathbf{0.000}$	$0.00/3.32 = \mathbf{0.000}$	$2.58/4.39 \approx \mathbf{0.588}$

Tabella 5.2: Vettori di documento normalizzati a norma euclidea unitaria (L_2).

5.4.4 Passo 4: Calcolo delle Similarità Coseno

$$\cos(\text{SaS}, \text{PaP}) = (0.789 \cdot 0.832) + (0.515 \cdot 0.555) \approx \mathbf{0.94}, \tag{5.17}$$

$$\cos(\text{SaS}, \text{WH}) = (0.789 \cdot 0.524) + (0.515 \cdot 0.465) + (0.335 \cdot 0.405) \approx \mathbf{0.79}, \tag{5.18}$$

$$\cos(\text{PaP}, \text{WH}) = (0.832 \cdot 0.524) + (0.555 \cdot 0.465) \approx \mathbf{0.69}. \tag{5.19}$$

Interpretazione dei Risultati

$$\cos(\text{SaS}, \text{PaP}) \approx 0.94 > \cos(\text{SaS}, \text{WH}) \approx 0.79 > \cos(\text{PaP}, \text{WH}) \approx 0.69. \tag{5.20}$$

La Cosine Similarity rileva correttamente che le due opere di Jane Austen condividono un'affinità lessicale molto elevata (94%), nettamente superiore rispetto all'opera di Emily Brontë.

5.5 Varianti di Ponderazione e la Notazione SMART

Per catalogare sistematicamente le decine di combinazioni possibili nella ponderazione vettoriale, Gerard Salton e i progettisti del sistema SMART hanno introdotto una convenzione mnemonica a due terne di caratteri:

$$\text{Schema SMART} \implies ddd.qqq, \tag{5.21}$$

in cui la prima terna *ddd* specifica la pesatura del documento e la seconda terna *qqq* quella della query.

Term Frequency (tf)		**Document Frequency (df)**		**Normalizzazione**	
`n`	tf (natural)	`n`	1 (nessun idf)	`n`	Nessuna (1)
`l`	$1 + \log_{10}(tf)$	`t`	$\log_{10}(N/df)$	`c`	Coseno $(1/\|\vec{w}\|_2)$
`a`	$0.5 + 0.5\frac{tf}{\max(tf)}$	`p`	$\max\left(0, \log\frac{N-df}{df}\right)$	`u`	Pivoted unique
`b`	1 se $tf > 0$, altr. 0			`b`	Byte size $(1/\text{len}^{\alpha})$

Tabella 5.3: Tassonomia delle componenti di ponderazione nella notazione SMART.

5.5.1 Lo Schema Industriale Standard: `lnc.ltc`

Una delle configurazioni più accreditate ed efficaci è la combinazione asimmetrica **`lnc.ltc`**:

- **Per il Documento (`lnc`):**
  - `l`: Term Frequency logaritmica $(1 + \log_{10} tf)$.
  - `n`: Nessun fattore IDF applicato al documento.

  - `c`: Normalizzazione coseno (L_2).

- **Per la Query (`ltc`):**
  - `l`: Term Frequency logaritmica ($1 + \log_{10} tf$).
  - `t`: Fattore IDF classico $\log_{10}(N/df)$.
  - `c`: Normalizzazione coseno (L_2).

Razionale Architetturale dell'Asimmetria

Non applicare l'IDF al documento (`n`) evita di alterare l'indice invertito globale con pesi che dipendono da corpus statici e previene penalizzazioni distorsive all'interno di testi ampi. Il compito di discriminare la specificità informativa dei termini cercati viene interamente delegato alla query (`t`).

5.5.2 Esercizio Numerico Completo `lnc.ltc` Svolto a Lezione

Dati di configurazione:

- **Documento:** *"car insurance auto insurance"*
- **Query:** *"best car insurance"*
- $N = 1\,000\,000$. Frequenze documentali:
  - `auto`: $df = 5\,000 \implies idf = \log_{10}(10^6/5000) = \log_{10}(200) \approx 2.3$
  - `best`: $df = 50\,000 \implies idf = \log_{10}(10^6/50000) = \log_{10}(20) \approx 1.3$
  - `car`: $df = 10\,000 \implies idf = \log_{10}(10^6/10000) = \log_{10}(100) = 2.0$
  - `insurance`: $df = 1\,000 \implies idf = \log_{10}(10^6/1000) = 3.0$

Passo 1: Elaborazione del Vettore Query (`ltc`)

Termine	tf raw	tf-wt $(1 + \log tf)$	df	idf	Peso non norm.	Peso norm. $(w/\|\vec{q}\|)$
auto	0	0.0	5 000	2.3	$0.0 \times 2.3 = 0.0$	**0.00**
best	1	1.0	50 000	1.3	$1.0 \times 1.3 = 1.3$	$1.3/3.84 \approx$ **0.34**
car	1	1.0	10 000	2.0	$1.0 \times 2.0 = 2.0$	$2.0/3.84 \approx$ **0.52**
insurance	1	1.0	1 000	3.0	$1.0 \times 3.0 = 3.0$	$3.0/3.84 \approx$ **0.78**

Tabella 5.4: Calcolo del vettore di query secondo lo schema `ltc`.

Norma euclidea del vettore query:

$$\|\vec{q}\|_2 = \sqrt{0^2 + 1.3^2 + 2.0^2 + 3.0^2} = \sqrt{0 + 1.69 + 4.0 + 9.0} = \sqrt{14.69} \approx \mathbf{3.84}. \tag{5.22}$$

Passo 2: Elaborazione del Vettore Documento (`lnc`)

Termine	tf **raw**	tf**-wt** $(1 + \log tf)$	idf **(nullo per** `n`**)**	**Peso non norm.**	**Peso norm.** $(w/\|\vec{d}\|)$
auto	1	1.0	—	1.0	$1.0/1.92 \approx \mathbf{0.52}$
best	0	0.0	—	0.0	**0.00**
car	1	1.0	—	1.0	$1.0/1.92 \approx \mathbf{0.52}$
insurance	2	$1 + \log_{10}(2) \approx 1.30$	—	1.3	$1.3/1.92 \approx \mathbf{0.68}$

Tabella 5.5: Calcolo del vettore di documento secondo lo schema `lnc`.

Norma euclidea del vettore documento:

$$\|\vec{d}\|_2 = \sqrt{1.0^2 + 0^2 + 1.0^2 + 1.30^2} = \sqrt{1.0 + 0 + 1.0 + 1.69} = \sqrt{3.69} \approx \mathbf{1.92}. \tag{5.23}$$

Passo 3: Calcolo del Punteggio Finale di Ranking

Il punteggio è il prodotto scalare tra i vettori normalizzati:

$$\begin{aligned} \text{Score}(q, d) &= \vec{q}_{\text{norm}} \cdot \vec{d}_{\text{norm}} \\ &= (0.00 \cdot 0.52) + (0.34 \cdot 0.00) + (0.52 \cdot 0.52) + (0.78 \cdot 0.68) \\ &= 0 + 0 + 0.2704 + 0.5304 \approx \mathbf{0.80}. \end{aligned} \tag{5.24}$$

5.6 Domande di Verifica e Concetti Chiave

1. **Perché il coefficiente di Jaccard assegna talvolta un punteggio superiore a documenti meno pertinenti?** Opera su insiemi puri, trascurando la frequenza locale dei termini (tf) e la rarità globale nella collezione (idf). Inoltre penalizza eccessivamente i documenti lunghi a causa dell'espansione dell'unione al denominatore.

2. **Perché non si adotta la distanza euclidea nello Spazio Vettoriale per ordinare i documenti?** La distanza euclidea risente della magnitudo: un documento duplicato con identico contenuto ha una norma doppia, risultando geometricamente lontanissimo dalla query pur avendo angolo zero ($\theta = 0°$).

3. **Che cosa rappresenta geometricamente la normalizzazione L_2?** Rappresenta la proiezione radiale dei vettori sulla superficie dell'ipersfera unitaria, neutralizzando l'effetto della lunghezza del documento e riducendo il calcolo del coseno a un semplice prodotto scalare.

4. **Qual è l'effetto dell'IDF su una query monoremica?** Nullo sul ranking relativo: il fattore IDF agisce come una costante moltiplicativa identica per tutti i documenti contenenti la parola, lasciando invariato l'ordinamento.

5.7 Best Match 25 (Okapi BM25)

Sviluppato negli anni '80 e '90 da Stephen Robertson e Karen Spärck Jones all'interno del sistema sperimentale Okapi presso la City University di Londra, **BM25** (*Best Match 25*) è una funzione di ranking non-lineare basata sul modello probabilistico di reperimento. Costituisce tuttora la baseline di riferimento più solida e performante per il reperimento lessicale primario (*first-stage retrieval*) nei motori di ricerca industriali.

5.7.1 Formulazione Matematica di BM25

Dato un documento D e una query a testo libero $Q = \{q_1, q_2, \ldots, q_n\}$, il punteggio BM25 è:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \dfrac{|D|}{\text{avgdl}}\right)}, \quad (5.25)$$

dove l'IDF probabilistico di Robertson-Spärck Jones è formulato come:

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right), \quad (5.26)$$

con le seguenti variabili operative:

- N: numero totale di documenti nell'intera collezione $\mathcal{D}$,
- $n(q_i)$: numero di documenti contenenti il termine di query q_i (equivalente a df_{q_i}),
- $f(q_i, D)$: frequenza grezza del termine q_i nel documento D (pari a $\text{tf}_{q_i,D}$),

- $|D|$: lunghezza complessiva del documento misurata in numero di token,
- avgdl: lunghezza media dei documenti nell'intera collezione (*average document length*).

5.7.2 Ruolo e Calibrazione degli Iperparametri

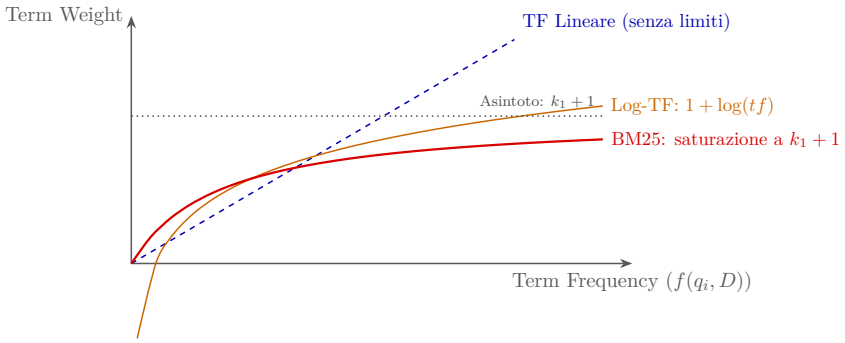


Figura 5.2: Confronto qualitativo tra crescita del TF lineare, TF logaritmico e saturazione asintotica in BM25.

1. **Parametro $k_1 \in [1.2, 2.0]$ (Saturazione della frequenza):** Controlla la rapidità con cui il contributo della Term Frequency converge asintoticamente al proprio limite superiore. Al tendere della frequenza locale all'infinito:
$$\lim_{f(q_i,D)\to\infty} \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + \dots} = k_1 + 1. \tag{5.27}$$
A differenza del modello TF-IDF classico in cui il peso continua a salire indefinitamente, in BM25 il contributo informativo di un singolo termine è rigorosamente limitato superiormente da $k_1 + 1$. *Caso limite:* se $k_1 = 0$, la componente di frequenza si annulla e il modello degenera in un ranking puramente binario basato unicamente su presenza/assenza e IDF.

2. **Parametro $b \in [0, 1]$ (Penalizzazione della lunghezza documentale):** Regola l'incidenza della lunghezza del documento rispetto alla media della collezione:
$$B = \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right). \tag{5.28}$$
   - Se $b = 1$, la normalizzazione è totale: un documento lungo il doppio rispetto alla media deve presentare il doppio delle occorrenze del termine per vedersi riconosciuto lo stesso score.

- Se $b = 0$, la normalizzazione è disattivata ($B = 1$ costante): la lunghezza del documento viene totalmente ignorata.
- Valore standard consigliato in letteratura: $b \approx 0.75$.

Valutazione di BM25 su Larga Scala

La formula di Okapi BM25 definisce la funzione analitica di scoring per ciascuna coppia query–documento. Tuttavia, nei moderni motori di ricerca con miliardi di documenti, la valutazione non può avvenire scansionando l'intera collezione. L'esecuzione ottimizzata di BM25 tramite liste invertite e gli algoritmi di query processing (*Term-at-a-Time* e *Document-at-a-Time* con tecniche di pruning dinamico come WAND) vengono trattati in dettaglio nel Capitolo 6.

5.8 Mappa Concettuale Riassuntiva

La Figura 5.3 offre una visione d'insieme del flusso completo di elaborazione: dall'acquisizione della risorsa web grezza fino alle funzioni di scoring vettoriale e probabilistico.

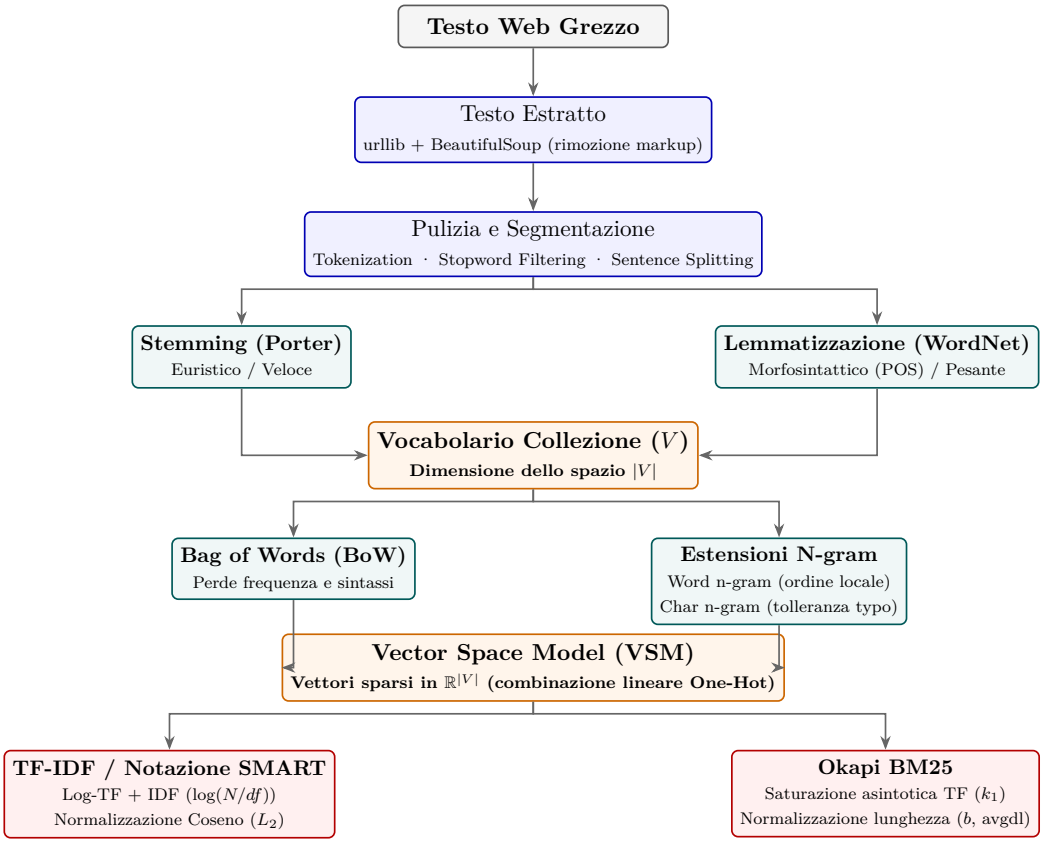


Figura 5.3: Mappa concettuale integrata della pipeline di IR: dall'acquisizione del testo ai modelli di recupero vettoriali e probabilistici.

Dallo Scoring Matematico all'Architettura di Sistema

Con la formalizzazione dello Spazio Vettoriale, degli schemi SMART e del modello probabilistico BM25 si conclude la trattazione teorica di *cosa* calcolare. Il Capitolo 6 affronta la dimensione ingegneristica di *come* eseguire tali calcoli in pochi millisecondi tramite l'architettura a due stadi, gli indici invertiti compressi e gli algoritmi di query processing TAAT e DAAT.

Capitolo 6

Architettura dei Motori di Ricerca e Indicizzazione

6.1 Architettura di Sistema e Paradigma a Due Stadi

Un motore di ricerca su scala Web deve conciliare requisiti architetturali apparentemente inconciliabili: processare collezioni testuali non strutturate di dimensioni sterminate garantendo al contempo una latenza di risposta interattiva ($<$ 50–100 ms) per ciascuna interrogazione utente. Per soddisfare tali vincoli, il flusso computazionale viene rigidamente ripartito tra processi asincroni massivi (**fase offline**) e processi sincroni a latenza vincolata (**fase online**), come schematizzato nella Sezione 6.1.

6.1.1 Pipeline Offline: Ingestione, Indicizzazione e Compressione

La pipeline offline opera sulla collezione documentale $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ attraverso tre macro-fasi:

1. **Text Processing e Parsing:** I flussi testuali grezzi raccolti tramite web crawling vengono ripuliti dal markup, tokenizzati e normalizzati secondo le pipeline analizzate nella Sezione 3.3.2.

2. **Generazione del Vocabolario e dell'Indice:** Viene estratto il lessico $\mathcal{V} = \{t_1, \ldots, t_M\}$ e costruita la struttura associativa che mappa ciascun termine alle relative occorrenze nei documenti.

3. **Compressione Numerica Lossless:** Poiché l'overhead dimensionale dell'indice grezzo può eguagliare o superare la dimensione del corpus di partenza, vengono applicati algoritmi di compressione per interi volti a massimizzare la residenza dei dati nei livelli alti della gerarchia di memoria (RAM e cache di sistema, come approfondito nel Capitolo 2).

6.1.2 Pipeline Online e Paradigma di Retrieval a Due Stadi

L'architettura moderna di risoluzione delle query adotta una pipeline a due stadi (*Two-Stage Retrieval Pipeline*), volta a bilanciare l'efficienza computazionale del recupero lessicale con l'elevata accuratezza semantica dei modelli predittivi complessi:

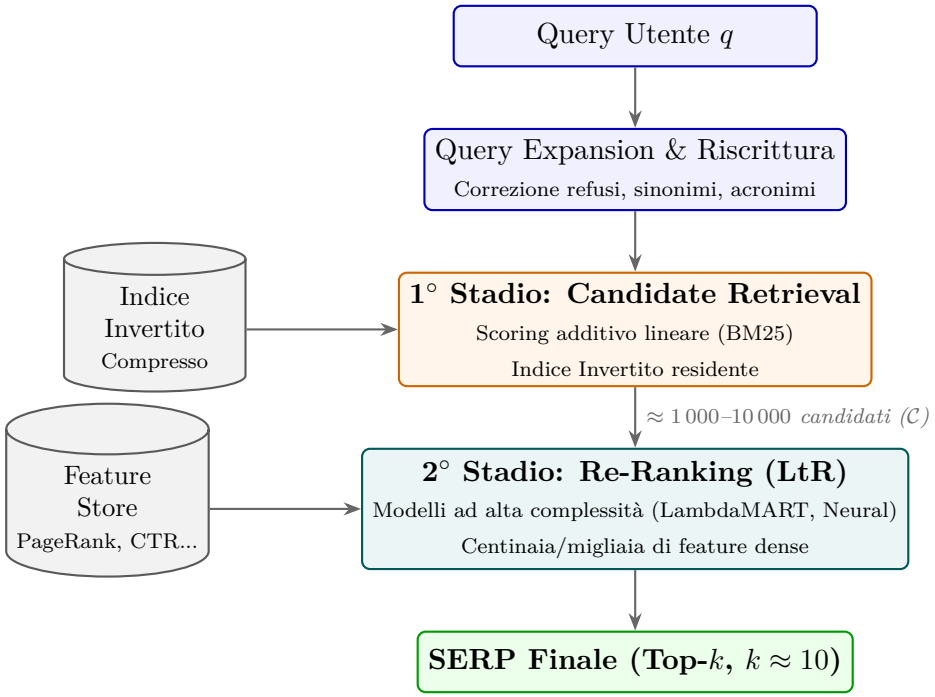


Figura 6.1: Architettura del paradigma di retrieval a due stadi: filtraggio massivo ad alta efficienza (primo stadio) e riordinamento semantico ad alta accuratezza (secondo stadio).

1. **Primo Stadio (Candidate Retrieval):** opera sull'intera collezione ($N \sim 10^9$–10^{10} documenti). Impiega funzioni di scoring additive veloci (tipicamente Okapi BM25) per estrarre rapidamente un sottoinsieme

ristretto di candidati promettenti:

$$\mathcal{C} \subset \mathcal{D}, \quad |\mathcal{C}| \approx 1\,000\text{–}10\,000 \text{ documenti.} \tag{6.1}$$

2. **Secondo Stadio (Re-Ranking / Heavy Scoring):** opera esclusivamente sull'insieme $\mathcal{C}$. Applica modelli di Machine Learning complessi (*Learning-to-Rank*, quali LambdaMART [4] o reti neurali basate su Transformer) valutando feature contestuali, grafiche, storiche e di autorità documentale per produrre la graduatoria finale Top-k ($k \approx 10$).

6.2 Modelli di Scoring: Prodotto Scalare e Connessione con il Dense Retrieval

L'elaborazione delle interrogazioni richiede una metrica algebrica solida per graduare i documenti in base alla loro aderenza al bisogno informativo. Questa sezione riesamina i limiti del modello booleano e del coefficiente di Jaccard, dimostrando come il prodotto scalare su vettori pesati costituisca la base unificante sia del retrieval sparso classico sia del moderno dense retrieval neurale.

6.2.1 Richiamo dei Limiti del Modello Booleano e di Jaccard

Nei sistemi delle prime generazioni, il reperimento dei documenti poggiava sul modello booleano esatto. Come formalizzato nella Sezione 4.2, tale approccio soffre della grave anomalia del *Feast or Famine*: una congiunzione stringente (`AND`) svuota l'insieme soluzione, mentre una disgiunzione (`OR`) restituisce centinaia di migliaia di risultati non graduati. Analogamente, l'applicazione ingenua del coefficiente insiemistico di Jaccard (analizzato in Sezione 4.3) fallisce poiché penalizza arbitrariamente i documenti lunghi e non discrimina tra parole comuni e parole altamente informative.

6.2.2 Il Prodotto Scalare come Misura Unificante di Rilevanza

Per superare tali limitazioni, lo Spazio Vettoriale (VSM) e i moderni schemi di ponderazione (formalizzati nel Capitolo 5) proiettano documenti e query in vettori pesati, trasformando la misura di pertinenza nel calcolo di un

prodotto scalare:

$$\text{Score}(q, d) = \mathbf{q} \cdot \mathbf{d} = \sum_{t \in \mathcal{V}} w(t, q) \cdot w(t, d). \tag{6.2}$$

Poiché per una query standard i termini non menzionati hanno peso $w(t, q) = 0$, la sommatoria (6.2) si restringe ai soli termini condivisi:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} w(t, q) \cdot w(t, d). \tag{6.3}$$

Questa formulazione evidenzia una profonda convergenza concettuale tra l'Information Retrieval lessicale classico e il moderno retrieval neurale:

- **Lexical Sparse Retrieval:** I vettori appartengono allo spazio $\mathbb{R}^{|\mathcal{V}|}$, dove ciascuna dimensione corrisponde a un lessema univoco del vocabolario ($|\mathcal{V}| \sim 10^7$). I vettori sono fortemente sparsi e lo score è calcolato tramite pesi statistici (TF-IDF, BM25).
- **Dense Neural Retrieval:** Modelli basati su Transformer (es. bi-encoder, Sentence-BERT) mappano testi e query in vettori continui densi di dimensionalità ridotta $\mathbf{v}_q, \mathbf{v}_d \in \mathbb{R}^k$ ($k \approx 768$ o $1\,536$). La rilevanza semantica è determinata dal prodotto scalare o similarità coseno:

$$\text{Score}_{\text{neural}}(q, d) = \mathbf{v}_q \cdot \mathbf{v}_d = \sum_{i=1}^{k} v_{q,i} \cdot v_{d,i}. \tag{6.4}$$

In entrambi i paradigmi, l'operazione fondamentale a runtime è l'aggregazione di prodotti scalari su collezioni su larga scala.

6.3 Strutture Dati ad Alte Prestazioni: Forward Index vs. Indice Invertito

Per gestire la sparsità estrema dello spazio termini–documenti, l'architettura del motore di ricerca deve abbandonare le matrici dense a favore di strutture di memorizzazione compressa. Questa sezione confronta le due modalità canoniche di organizzazione dei dati (Forward Index e Indice Invertito), motivando la centralità dell'indice invertito per la risoluzione rapida delle interrogazioni online.

6.3.1 La Matrice Termini–Documenti e il Problema della Sparsità

Consideriamo la matrice globale termini–documenti $M \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{D}|}$. I parametri dimensionali di un motore di ricerca web reale evidenziano l'impossibilità fisica di allocare e scansionare tale struttura in forma densa:

- Dimensione del lessico: $|\mathcal{V}| \sim 10^7$ termini unici.
- Dimensione della collezione: $|\mathcal{D}| \sim 10^9$–10^{10} documenti web.
- Numero totale di celle: $|\mathcal{V}| \times |\mathcal{D}| \approx 10^{16}$–$10^{17}$ posizioni.

Poiché un documento medio contiene tra 10^2 e 10^3 vocaboli distinti, la densità di elementi non nulli è infinitesima:

$$\frac{\text{elementi non nulli}}{|\mathcal{V}| \times |\mathcal{D}|} \ll 0.001\%. \tag{6.5}$$

Oltre il 99.99% delle celle contiene il valore zero. Memorizzare la matrice in formato denso comporterebbe la saturazione immediata della memoria secondaria e una complessità temporale di valutazione query proibitiva, pari a $\mathcal{O}(|\mathcal{V}| \cdot |\mathcal{D}|)$, costringendo la CPU a esaminare miliardi di documenti irrilevanti.

6.3.2 Rappresentazione tramite Vettori Sparsi: Forward Index vs. Indice Invertito

Per eliminare gli zeri, la matrice viene memorizzata come un insieme di vettori sparsi composti da coppie $(\text{Identificatore}, \text{Valore})$. L'orientamento di serializzazione determina due strutture complementari, sintetizzate nella Tabella 6.1:

- **Forward Index (Column-Wise / Document-Oriented):** memorizza per ciascun documento la lista dei termini che vi compaiono, ad esempio $d_1 \to [(t_2, w_2), (t_7, w_7)]$.
- **Indice Invertito (Row-Wise / Term-Oriented):** memorizza per ciascun termine la lista dei documenti in cui ricorre, ad esempio $t_1 \to [(d_2, w_1), (d_5, w_5)]$.

Proprietà	Forward Index	Indice Invertito
Orientamento	Per colonna (Document-Oriented).	Per riga (Term-Oriented).
Struttura logica	DocID $\rightarrow [(t_a, \text{tf}_a), \dots]$	$t \rightarrow [(\text{DocID}_i, \text{tf}_i), \dots]$
Complessità query	$\mathcal{O}(\lvert\mathcal{D}\rvert)$: richiede ispezione di tutti i documenti.	$\mathcal{O}\left(\sum_{t \in q} \lvert\mathcal{L}(t)\rvert\right)$: accede solo ai documenti candidati.
Ruolo nel motore	Memorizzazione metadati, feature store, snippet generation.	Cuore computazionale del First-Stage Retrieval (matching rapido).

Tabella 6.1: Confronto sistematico tra Forward Index e Indice Invertito.

6.4 Anatomia, Layout di Memoria e Compressione dell'Indice Invertito

Un indice invertito di livello industriale si articola in due componenti fisiche distinte: il **Dizionario** (o Lessico) e l'insieme delle **Posting List**.

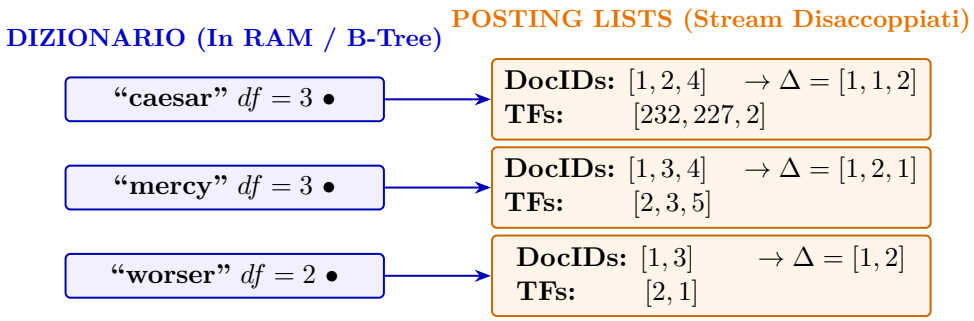


Figura 6.2: Anatomia di un indice invertito con disaccoppiamento fisico tra stream dei DocID (compressi tramite d-gaps) e stream delle frequenze tf.

6.4.1 Il Dizionario (Lexicon)

Il dizionario memorizza il vocabolario $\mathcal{V}$. Per ciascun termine t, registra la stringa lessicale, la Document Frequency df_t (utilizzata per ricavare l'IDF precalcolato) e il puntatore di memoria o l'offset su storage secondario che localizza la corrispondente posting list. Dato che $|\mathcal{V}|$ è dell'ordine di milioni di lessemi, il dizionario viene mantenuto interamente residente in **RAM**,

indicizzato tramite strutture di accesso rapido come B-Tree, Trie compressi o tabelle hash.

6.4.2 Le Posting List: Disaccoppiamento Fisico degli Stream

Concettualmente, la posting list associata al termine t è una sequenza di tuple:

$$\mathcal{L}(t) = \Big[\langle d_1, \mathrm{tf}_{t,d_1}\rangle, \langle d_2, \mathrm{tf}_{t,d_2}\rangle, \dots\Big]. \tag{6.6}$$

A livello implementativo, memorizzare array continui di strutture composte (es. struct C++ contenenti `uint32_t doc_id` e `uint32_t tf`) è altamente sub-ottimale. Per massimizzare i fattori di compressione numerica, i motori di ricerca operano il **disaccoppiamento fisico in stream paralleli**:

1. **Stream dei DocID (Monotono Crescente):**

   $$\mathbf{D}_t = [\mathrm{DocID}_1, \mathrm{DocID}_2, \dots, \mathrm{DocID}_m] \quad \text{con } \mathrm{DocID}_i < \mathrm{DocID}_{i+1}. \tag{6.7}$$

   Sfruttando la stretta monotonicità, i valori assoluti vengono convertiti nelle differenze tra identificatori contigui, dette d**-gaps** o delta:

   $$\Delta_1 = \mathrm{DocID}_1, \quad \Delta_i = \mathrm{DocID}_i - \mathrm{DocID}_{i-1} \quad (\forall i \geq 2). \tag{6.8}$$

   I delta generati sono interi piccoli (spesso $\Delta \leq 5$ per termini ad alta frequenza), comprimibili ad altissima densità tramite algoritmi dedicati (*Variable Byte*, *Elias-Fano*, *PForDelta*, *Binary Interpolative Coding* [3]).

2. **Stream delle Frequenze (Non Ordinato):**

   $$\mathbf{F}_t = [\mathrm{tf}(t, d_1), \mathrm{tf}(t, d_2), \dots, \mathrm{tf}(t, d_m)]. \tag{6.9}$$

   Le frequenze locali non godono di ordinamento; tuttavia, per la legge di Zipf, la loro distribuzione empirica è fortemente concentrata su valori minimi $(1, 2, 3)$. Esse vengono compresse simmetricamente tramite codici a lunghezza variabile senza richiedere la differenziazione delta.

6.4.3 Trade-Off Architetturale: Interi Compressi vs. Float Precalcolati

Una decisione progettuale critica consiste nello stabilire se memorizzare nell'indice il punteggio finale precalcolato $w(t, d) \in \mathbb{R}$ a virgola mobile oppure l'intero grezzo $\mathrm{tf}(t, d) \in \mathbb{N}$:

Parametro	Punteggio Float Precalcolato	Intero TF Compresso
Occupazione Spazio	32 o 64 bit per posting (`float`/`double`).	Intero compresso (spesso < 8 bit medi per posting).
Comprimibilità	Scarsa o nulla: mantissa float a entropia quasi-casuale.	Elevatissima: concentrazione statistica su delta piccoli.
Flessibilità Ranking	Nulla: vincola l'indice alla specifica formula usata a build-time.	Totale: consente di calcolare qualsiasi funzione (BM25, varianti) a runtime.
Throughput CPU	Nessun calcolo a runtime (lettura diretta).	Pochi cicli ALU per posting per calcolare la formula.

Tabella 6.2: Confronto di sistema tra memorizzazione float e interi compressi.

Risoluzione del Vincolo Memory-Bound

Come analizzato nel Capitolo 2, i sistemi di Information Retrieval su larga scala sono rigorosamente **memory-bound**: il vero fattore limitante delle prestazioni è la banda di trasferimento dalla memoria principale (DRAM) ai registri della CPU (≈ 80 ns di latenza), non il throughput dell'unità aritmetica (ALU, 0.5 ns). Ridurre la dimensione dei posting attraverso la compressione degli interi abbatte il volume di byte trasferiti sul bus di memoria; il modesto costo computazionale richiesto per valutare la formula a runtime (es. BM25) viene ampiamente compensato dal risparmio di banda.

6.5 Tipologie di Query, Indici Posizionali e Riscrittura del Testo

L'indice invertito essenziale a sole frequenze (DocID, tf) non è sufficiente per supportare ricerche complesse che richiedono la contiguità delle parole o la riscrittura semantica delle interrogazioni. Questa sezione approfondisce la struttura dell'indice posizionale per l'elaborazione delle *phrase queries* e illustra le tecniche di *query expansion* e normalizzazione a tempo di esecuzione.

6.5.1 Phrase Queries e la Necessità dell'Indice Posizionale

Nelle ricerche per frase esatta (*Phrase Queries*), la query è racchiusa tra virgolette (es. `"neural network architecture"`). L'utente impone che i termini compaiano nel medesimo documento, nello stesso ordine e in posizioni adiacenti:

$$\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1. \tag{6.10}$$

L'indice invertito standard a sole frequenze (DocID, tf) non permette di validare questo vincolo, avendo scartato la posizione delle parole. Per supportare le phrase queries, la struttura viene estesa a **Indice Posizionale**, memorizzando per ogni posting la lista degli offset ordinali:

$$t \longrightarrow \Big[\langle \text{DocID}_k, \text{tf}_k, [\text{pos}_1, \text{pos}_2, \ldots, \text{pos}_{\text{tf}}]\rangle\Big]. \tag{6.11}$$

Esempio 6.5.1 (Intersezione Posizionale per Phrase Match)**.** Si consideri la query `"neural network"`:

- Posting per `"neural"`: $\cdots \to \langle \text{Doc } 14, \text{tf} = 2, \text{pos} = [4, 21]\rangle \to \ldots$
- Posting per `"network"`: $\cdots \to \langle \text{Doc } 14, \text{tf} = 3, \text{pos} = [5, 89, 102]\rangle \to \ldots$

L'algoritmo esegue l'intersezione su DocID $= 14$ e confronta le liste posizionali: poiché $\text{pos}(\texttt{neural}) = 4$ e $\text{pos}(\texttt{network}) = 5$, la condizione $\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1$ è soddisfatta e il documento viene validato.

Overhead spaziale: L'inclusione delle posizioni incrementa l'impronta complessiva su disco dell'indice invertito di un fattore compreso tra $2\times$ e $3\times$.

6.5.2 Moduli di Query Expansion e il Caso Studio "Babbo Francesco"

I motori di ricerca industriali intercettano la query grezza dell'utente attraverso un modulo di **Query Expansion** e riscrittura, con l'obiettivo di correggere refusi, risolvere acronimi ed espandere sinonimi contestuali.

Tuttavia, espansioni lessicali non supervisionate basate su matrici di co-occorrenza o regole euristiche possono innescare gravi derive semantiche (*semantic drift*), come illustrato dal celebre caso riscontrato nei motori commerciali in lingua italiana:

1. **Query Utente:** L'utente inserisce la query a intento personale/familiare `"babbo francesco"`.

2. **Espansione Sinonimica:** Il modulo associa il termine regionale *babbo* al sinonimo *papà*.

3. **Normalizzazione e Troncamento Accenti:** La pipeline rimuove gli accenti trasformando *papà* in *papa* (omografo non accentato che designa il Pontefice).

4. **Query Riscritta all'Indice:** La disgiunzione risultante inviata al motore diventa `("babbo" OR "papà" OR "papa") AND "francesco"`.

5. **Scoring & Authority Ranking:** L'entità globale *"Papa Francesco"* possiede un'autorevolezza a priori (PageRank) e una co-occorrenza web sterminata rispetto a qualunque testo privato.

6. **Risultato Top-1 Restituito:** La SERP collassa restituendo in prima posizione la voce enciclopedica su *Papa Francesco (Jorge Mario Bergoglio)*, stravolgendo completamente l'intento informativo dell'utente.

6.5.3 Stemming vs. Query Expansion

È fondamentale chiarire la linea di demarcazione tra normalizzazione morfologica ed espansione semantica:

- **Stemming e Lemmatizzazione:** Operano riduzioni sintattiche all'interno della medesima radice etimologica (es. *running*, *runs* $\rightarrow$ *run*, come descritto in Sezione 3.3.3). Non possono collegare vocaboli con radici distinte aventi identico significato (es. *babbo* e *papà*).

- **Approccio Moderno:** I sistemi scalabili contemporanei riducono l'uso di stemming aggressivo in fase di indicizzazione (per preservare la specificità semantica dei termini nell'indice) e gestiscono le varianti flessive e sinonimiche a runtime tramite modelli linguistici neurali di espansione contestuale.

Dall'Indicizzazione al Query Processing

Costruite le strutture dati dell'indice e definite le modalità di rappresentazione delle interrogazioni, il motore deve eseguire il recupero dei documenti rilevanti in tempo reale ($\leq$ 50–100 ms). I modelli teorici e i paradigmi algoritmici per l'attraversamento delle posting list, l'aggregazione dei punteggi e l'ottimizzazione tramite salti rapidi (*Term-at-a-Time* e *Document-at-a-Time*) sono formalizzati e analizzati esaustivamente nel Capitolo 7.

Capitolo 7

Query Processing: Algoritmi di Scansione ed Efficienza di Salto

7.1 Anatomia Statistica dell'Indice Inverso: Dizionario e Posting Lists

L'indice inverso costituisce l'infrastruttura fondamentale su cui poggiano tutte le operazioni di interrogazione nei moderni motori di ricerca. Questa sezione ne esamina l'organizzazione strutturale tra dizionario dei termini e liste di occorrenza, l'invariante di ordinamento dei docID e le statistiche ausiliarie necessarie per abilitare i modelli di scoring pesato.

7.1.1 Definizione e Componenti Architetturali

Come introdotto nella Sezione 6.4, l'**Indice Inverso** è la struttura dati associativa centrale per l'Information Retrieval: mappa ciascun termine del lessico all'insieme ordinato delle sue occorrenze nell'intera collezione. Si compone di due elementi fisici distinti:

1. **Dizionario (Dictionary o Lexicon):** Memorizza l'elenco ordinato di tutti i termini distinti $\mathcal{V}$ della collezione assieme a metadati globali di frequenza e offset di puntamento alla memoria secondaria.

2. **Posting Lists (Liste delle Occorrenze):** Per ciascun termine $t \in \mathcal{V}$, una sequenza ordinata di record (detti *postings*) che identificano i documenti in cui t compare.

Esempio 7.1.1 (Costruzione dell'Indice Inverso Booleano)**.** Si consideri una collezione elementare di $N = 3$ documenti testuali:

- d_1: "*An inverted index is an index storing a mapping from terms to their occurrences in a set of documents.*"
- d_2: "*The inverted index data structure is a central component of a typical search engine indexing algorithm.*"
- d_3: "*The goal is to optimize the speed of the query: find the documents where a set of words occur.*"

Assegnati gli identificatori interi $d_1 \to 1$, $d_2 \to 2$, $d_3 \to 3$, le posting list per alcuni termini del vocabolario risultano:

$$\begin{aligned}
\mathcal{L}(\texttt{"a"}) &= [1,2,3] \quad \text{(stopword ad altissima frequenza, tocca tutti i documenti)},\\
\mathcal{L}(\texttt{"an"}) &= [1],\\
\mathcal{L}(\texttt{"index"}) &= [1,2],\\
\mathcal{L}(\texttt{"inverted"}) &= [1,2],\\
\mathcal{L}(\texttt{"query"}) &= [3].
\end{aligned}$$

7.1.2 L'Invariante Fondamentale di Ordinamento dei docID

All'interno di ciascuna posting list, i docID sono **rigorosamente ordinati in senso strettamente crescente**:

$$\forall i < j, \quad \text{docID}_i < \text{docID}_j. \tag{7.1}$$

Questa proprietà strutturale costituisce la precondizione algebrica indispensabile per eseguire l'unione (OR) e l'intersezione (AND) tra liste in tempo lineare o sublineare, evitando scansioni combinatorie esaustive.

7.1.3 Estensione per Modelli di Ranking: Statistiche di Dizionario e Posting

Nel recupero pesato (es. Okapi BM25 o TF-IDF), la semplice presenza binaria del termine non è sufficiente per calcolare la rilevanza. L'indice inverso memorizza statistiche strutturate su due livelli gerarchici:

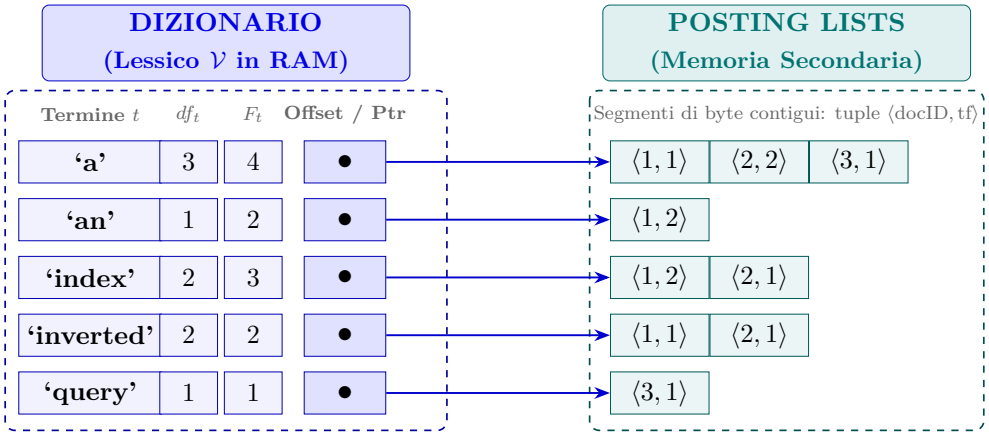


Figura 7.1: Organizzazione gerarchica dell'indice inverso per modelli pesati. A sinistra: il dizionario residente in RAM, ordinato lessicograficamente con statistiche aggregate (df_t, F_t) e campo offset/puntatore d'inizio lista. A destra: le posting list su memoria secondaria, strutturate come array contigui di byte (tuple $\langle \text{docID}, \text{tf} \rangle$) privi di puntatori di concatenazione interna.

1. **Statistiche a livello di Dizionario (per ciascun termine t):**
   - **Document Frequency (df_t):** il numero di documenti distinti contenenti almeno un'occorrenza di t. Corrisponde esattamente alla lunghezza (cardinalità) della posting list $|\mathcal{L}(t)|$.
   - **Collection Frequency (F_t):** il numero complessivo di occorrenze del termine t calcolato sull'intera collezione:
   $$F_t = \sum_{d \in \mathcal{L}(t)} \text{tf}_{t,d}. \tag{7.2}$$
   Ad esempio, per il termine `"a"` si ha $df_{\text{"a"}} = 3$ e $F_{\text{"a"}} = 1+2+1 = 4$. Vale sempre la relazione d'ordine fondamentale:
   $$F_t \geq df_t. \tag{7.3}$$
   L'uguaglianza $F_t = df_t$ si verifica se e solo se ciascun documento rilevante contiene il termine esattamente una volta ($\text{tf}_{t,d} = 1$).
   - **Puntatore / Offset d'Inizio (ptr_t):** l'indirizzo di memoria o l'offset in byte che indica il punto d'inizio della posting list del termine t all'interno del file compresso su memoria secondaria.
2. **Statistiche a livello di Posting List (per ciascun documento d):**
   - Viene memorizzata la **Term Frequency** locale $\text{tf}_{t,d}$, quantificando quante volte il termine compare nello specifico testo di d.

7.1.4 Trade-Off Architetturale: Perché NON Memorizzare lo Score Precalcolato?

Come approfondito nell'analisi di sistema della Sezione 6.4, ci si potrebbe domandare: *perché memorizzare la frequenza intera $tf_{t,d}$ anziché il punteggio finale già precalcolato (ad esempio lo score BM25 come float a 32 bit)?*

La prassi industriale esclude la memorizzazione di punteggi float precomputati per tre ragioni fisiche e progettuali:

1. **Comprimibilità dei Tipi di Dato:** La frequenza locale $\mathrm{tf}_{t,d}$ è un intero positivo strettamente delimitato, concentrato statisticamente su valori minimi $(1, 2, 3)$. Gli interi piccoli si prestano a rapporti di compressione lossless eccezionali (scendendo a pochi bit per posting). Al contrario, un numero float a virgola mobile possiede una mantissa ad alta entropia, pressoché incomprimibile.

2. **Località dell'IDF e Costo del Logaritmo:** Il calcolo dell'Inverse Document Frequency:

$$\mathrm{IDF}(t) = \log\left(\frac{N - df_t + 0.5}{df_t + 0.5}\right) \tag{7.4}$$

   dipende unicamente dalla frequenza di documento df_t e dalla taglia N della collezione: esso è **costante per l'intero termine** e completamente indipendente dal documento d. È computazionalmente vantaggioso calcolare l'IDF **una sola volta online** all'arrivo della query leggendo df_t dal dizionario (residente in RAM), anziché duplicarlo o fonderlo in ciascuno dei milioni di posting su memoria secondaria.

3. **Flessibilità e Modularità del Ranking:** Congelare lo score finale nell'indice vincolerebbe il sistema ai parametri specifici adottati a build-time (ad es. $k_1 = 1.2, b = 0.75$). Modificare i parametri o testare nuove varianti richiederebbe la reindicizzazione integrale del Web. Memorizzare dati grezzi (tf e lunghezze dei documenti) consente di variare liberamente la funzione di scoring a runtime.

7.2 Paradigmi di Scansione: TAAT vs. DAAT

Individuate le posting list associate ai termini di query $q = \{t_1, t_2, \ldots, t_m\}$, il motore deve scansionare i dati ed estrarre i primi k documenti con punteggio più elevato per popolare una coda con priorità (min-heap di dimensione fissa

k). Esistono due paradigmi algoritmici fondamentali: **Term-at-a-Time (TAAT)** e **Document-at-a-Time (DAAT)**.

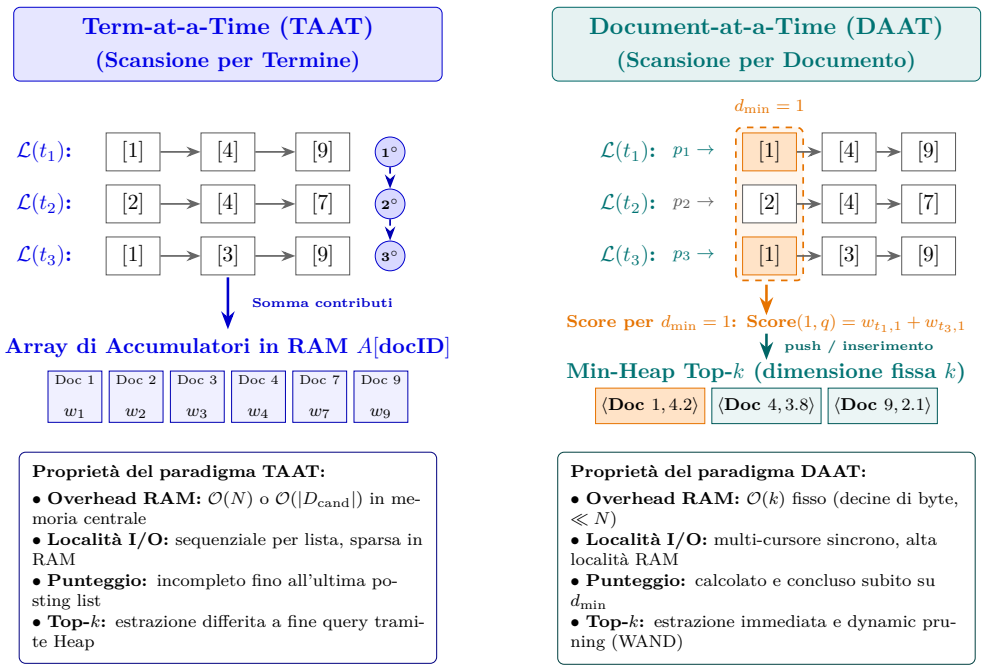


Figura 7.2: Confronto visivo tra i paradigmi di scansione: Term-at-a-Time (scansione orizzontale termine per termine con accumulo parziale $\mathcal{O}(N)$ in RAM) vs. Document-at-a-Time (scansione verticale sincrona per documento $d_{\min}$ con calcolo immediato dello score finale e memorizzazione ristretta al Min-Heap Top-k $\mathcal{O}(k)$).

7.2.1 La Strategia Term-at-a-Time (TAAT)

Nell'approccio TAAT, le posting list vengono elaborate in modo **sequenziale, una alla volta**:

1. Viene allocata una struttura di **accumulatori** $A[\text{DocID}] \in \mathbb{R}$ (tabella hash o array denso indicizzato per DocID).

2. Il motore apre la posting list del primo termine $\mathcal{L}(t_1)$ e itera su tutti i suoi posting, calcolando il punteggio parziale e incrementando l'accumulatore corrispondente.

3. Conclusa $\mathcal{L}(t_1)$, si passa a $\mathcal{L}(t_2)$ sommando i nuovi contributi, procedendo fino all'ultima lista $\mathcal{L}(t_m)$.

4. Al termine, si estraggono i top-k documenti scansionando la struttura degli accumulatori tramite un min-heap.

Algorithm 1: Term-at-a-Time (TAAT)

Dati: Query $q = \{t_1, t_2, \ldots, t_m\}$, Indice Invertito, intero k
Risultato: Min-Heap H contenente i Top-k documenti per punteggio

$A \leftarrow$ InizializzaTabellaAccumulatori() `// Mappa DocID → Score Parziale`
foreach *termine* $t_i \in q$ **do**
  $\mathcal{L}_i \leftarrow$ RecuperaPostingList(t_i)
  $\text{idf}_i \leftarrow$ CalcolaIDF(t_i)
  foreach $\langle d, tf \rangle \in \mathcal{L}_i$ **do**
    $s_{\text{parz}} \leftarrow$ CalcolaContributo(tf, idf_i)
    if $d \in A$ **then**
      $A[d] \leftarrow A[d] + s_{\text{parz}}$
    else
      $A[d] \leftarrow s_{\text{parz}}$
$H \leftarrow$ InizializzaMinHeap(capacità $= k$)
foreach *coppia* $(d, score) \in A$ **do**
  if $H.Dimensione() < k$ **then**
    H.Inserisci(d, score)
  else
    if $score > H.MinimoPunteggio()$ **then**
      H.EstraiMinimo()
      H.Inserisci(d, score)
return H

Analisi prestazionale di TAAT:

- **Vantaggi:** Accesso strettamente sequenziale alla memoria. Ciascuna posting list viene letta come un blocco contiguo, massimizzando l'efficacia del prefetching hardware della CPU e la località di cache (Capitolo 2).
- **Svantaggi:** Footprint di memoria volatile elevatissimo. La tabella degli accumulatori deve contenere l'unione dei documenti di tutte le posting list: con termini ad alta Document Frequency, A può coinvolgere decine di milioni di record. Inoltre, è impossibile effettuare il *dynamic early pruning*, poiché uno score parziale basso potrebbe essere incrementato arbitrariamente dalle liste successive.

7.2.2 La Strategia Document-at-a-Time (DAAT)

Nell'approccio DAAT, le posting list di tutti i termini di query vengono elaborate **simultaneamente e in parallelo**, operando come un merge multi-cursore:

1. Viene posizionato un puntatore p_i all'inizio di ciascuna lista $\mathcal{L}(t_i)$.

2. A ogni iterazione, il sistema determina il minimo identificatore corrente:

$$d_{\min} = \min_{i \in \{1,\dots,m\}} \{p_i.\text{CurrentDocID}()\}. \tag{7.5}$$

3. Vengono identificati tutti i cursori posizionati su $d_{\min}$, calcolando immediatamente il **punteggio finale consolidato** del documento:

$$S(q, d_{\min}) = \sum_{i : p_i.\text{DocID} = d_{\min}} \text{ScoreTerm}(p_i.\text{TF}, t_i). \tag{7.6}$$

4. Il punteggio definitivo viene confrontato all'istante con il Min-Heap di taglia k: se $S(q, d_{\min}) > H.\text{Minimo}()$, il documento entra nella Top-k scalzando il vecchio minimo.

5. Tutti i cursori attestati su $d_{\min}$ avanzano al prossimo elemento.

Algorithm 2: Document-at-a-Time (DAAT)

Dati: Query $q = \{t_1, t_2, \ldots, t_m\}$, Indice Invertito, intero k
Risultato: Min-Heap H contenente i Top-k documenti per punteggio

```
1  cursori ← InizializzaCursoriPerOgniLista(q)
2  H ← InizializzaMinHeap(capacità = k)
3  while EsistonoCursoriAttivi(cursori) do
4      d_min ← min{p.CurrentDocID() | p ∈ cursori ∧ p.ÈAttivo()}
5      score_tot ← 0.0
6      foreach p ∈ cursori do
7          if p.ÈAttivo() ∧ p.CurrentDocID() == d_min then
8              score_tot ←
                 score_tot + CalcolaContributo(p.CurrentTF(), p.Termine())
9              p.Avanza()
           // Il documento è completo:  inserimento immediato
              nell'heap
10     if H.Dimensione() < k then
11         H.Inserisci(d_min, score_tot)
12     else
13         if score_tot > H.MinimoPunteggio() then
14             H.EstraiMinimo()
15             H.Inserisci(d_min, score_tot)
16 return H
```

Analisi prestazionale di DAAT:

- **Vantaggi:** Memoria di lavoro trascurabile ($\mathcal{O}(k)$ per il Min-Heap e $\mathcal{O}(m)$ per lo stato dei cursori), senza alcuna tabella intermedia di accumulo. Supporto nativo all'**Early Pruning dinamico**: conoscendo la soglia minima corrente dell'heap $\theta = H$.MinimoPunteggio(), algoritmi avanzati come **WAND** (*Weak AND*) o **Block-Max WAND** possono stimare i limiti superiori (*upper bounds*) di punteggio residuo e saltare a blocchi intere sequenze di posting che matematicamente non possono superare θ, evitando decompressioni superflue.

- **Svantaggi:** Accesso alla memoria frammentato su più stream contigui, con frequenti salti tra puntatori che riducono l'efficacia del caching rispetto alla lettura lineare di TAAT.

7.3 Elaborazione Term-at-a-Time (TAAT) in Dettaglio

Nel paradigma TAAT la computazione procede esaurendo sequenzialmente una lista di occorrenza alla volta, accumulando incrementalmente i punteggi parziali dei documenti intercettati. Questa sezione analizza il flusso operativo di TAAT tramite un esempio tracciato passo-passo, confronta le strutture dati per la memorizzazione degli accumulatori e ne evidenzia i limiti prestazionali in termini di località di cache e assenza di salti.

7.3.1 Tracciamento dell'Esecuzione di TAAT

Si consideri una query composta dai termini `"information"` e `"retrieval"`. Le posting list associate ai due vocaboli sono:

$$\mathcal{L}(\texttt{"information"}) = [1, 5, 8, 11, 13, 20, 35, 40, 42],$$
$$\mathcal{L}(\texttt{"retrieval"}) = [1, 5, 6, 8, 11, 15, 17, 50, 60].$$

Fase 1: Scansione completa della lista `"information"`. Il puntatore scorre l'intera lista $\mathcal{L}(\texttt{"information"})$. Per ciascun identificatore incontrato, viene allocato o incrementato un accumulatore:

$$A[1] = 1, \quad A[5] = 1, \quad A[8] = 1, \quad A[11] = 1, \quad A[13] = 1,$$
$$A[20] = 1, \quad A[35] = 1, \quad A[40] = 1, \quad A[42] = 1.$$

Lo stato parziale della tabella degli accumulatori conta 9 record.

Fase 2: Scansione completa della lista `"retrieval"`. Il motore passa alla seconda lista e aggiorna progressivamente la tabella:

- docID 1: già presente $\implies A[1] \leftarrow 1 + 1 = 2$.
- docID 5: già presente $\implies A[5] \leftarrow 1 + 1 = 2$.
- docID 6: non presente $\implies$ viene creato l'accumulatore $A[6] \leftarrow 1$.
- docID 8: già presente $\implies A[8] \leftarrow 1 + 1 = 2$.
- docID 11: già presente $\implies A[11] \leftarrow 1 + 1 = 2$.
- docID 15, 17, 50, 60: non presenti $\implies$ vengono creati $A[15] = 1, A[17] = 1, A[50] = 1, A[60] = 1$.

Stato Finale degli Accumulatori e Risoluzione delle Query: Al termine della scansione:

- Documenti con punteggio 2: $\{1, 5, 8, 11\}$.
- Documenti con punteggio 1: $\{6, 13, 15, 17, 20, 35, 40, 42, 50, 60\}$.

A seconda della semantica richiesta:

- **Query Disgiuntiva (OR):** si restituiscono tutti i 14 documenti con $A[d] > 0$.
- **Query Congiuntiva (AND):** si filtrano unicamente gli accumulatori il cui punteggio eguaglia il numero di termini della query ($A[d] == 2$), restituendo $\{1, 5, 8, 11\}$.
- **Query con Negazione (`retrieval AND NOT information`):** durante la scansione della lista negata, gli accumulatori corrispondenti non vengono incrementati, ma azzerati o marcati con $-\infty$.
- **Ranking Pesato (BM25):** l'operazione non esegue un semplice incremento +1, ma accumula il contributo reale:

$$A[d] \leftarrow A[d] + \text{BM25_weight}(t, d). \tag{7.7}$$

7.3.2 Strutture Dati per gli Accumulatori: DAT vs. Hash Table

Per implementare l'insieme degli accumulatori A, si considerano due alternative ingegneristiche:

1. **Direct Access Table (DAT) / Array Diretto:** Si alloca un array contiguo di cardinalità pari al numero totale di documenti della collezione: $A[0 \dots N-1]$, inizializzato a zero. L'accesso e l'aggiornamento avvengono in tempo $O(1)$ immediato: $A[d] \leftarrow A[d] + w$. *Criticità insormontabile:* Se $N = 10^9$ documenti, allocare un array da 4 o 8 GB di memoria RAM per ciascun thread concorrente di query è fisicamente insostenibile.
2. **Hash Table:** Memorizza unicamente le coppie $\langle \text{docID}, \text{score} \rangle$ relative ai documenti effettivamente intercettati dalle liste della query. *Criticità:* Sebbene risparmi memoria teorica, la funzione di hashing disperde gli accessi casualmente nella RAM.

7.3.3 Vantaggi e Svantaggi Critici di TAAT

- **Pregi:** Semplicità implementativa assoluta. Ciascuna posting list viene letta linearmente tramite la semplice primitiva elementare next().
- **Difetti Strutturali Gravi:**
  1. **Inefficienza Catastrofica di Cache:** Come formalizzato nel Capitolo 2, l'aggiornamento continuo degli accumulatori (sia su hash table sia su array sparsi) provoca accessi casuali alla memoria fisica (*random memory jumps*). La CPU subisce continui **cache misses** a tutti i livelli (L1, L2, L3), degradando le prestazioni hardware.
  2. **Impossibilità di Salto (*No Skipping*):** L'algoritmo è costretto a percorrere integralmente ciascuna lista. Anche per query selettive (AND), ogni elemento deve essere obbligatoriamente visitato per verificare se l'accumulatore esista già.

7.4 DAAT per Query Disgiuntive (OR) e Congiuntive (AND Base)

A differenza di TAAT, il paradigma Document-at-a-Time valuta completamente un documento prima di passare al successivo, operando tramite l'avanzamento sincrono di cursori multipli. Questa sezione analizza il comportamento di DAAT su query disgiuntive (`OR`) basate sull'unione coordinata, e formalizza l'algoritmo elementare di intersezione per query congiuntive (`AND`), evidenziandone i margini di inefficienza nei passi unitari.

7.4.1 DAAT per Query Disgiuntive (OR): Unione Sincrona

Nelle query disgiuntive (OR), l'algoritmo DAAT opera facendo avanzare simultaneamente i cursori attestati sulle posting list, replicando l'algoritmo di unione a due puntatori del **MergeSort**:

1. A ogni iterazione si confrontano i docID puntati dai cursori: $d_1 = p_1.\text{docID}()$ e $d_2 = p_2.\text{docID}()$.
2. Sia $m = \min(d_1, d_2)$:

- Se $d_1 == d_2$: entrambi i puntatori convergono sullo stesso documento. Si emette m nei risultati (con il punteggio combinato) e **avanzano entrambi i cursori**.
- Se $d_1 < d_2$: il minimo è d_1. Si emette d_1 nei risultati e **avanza solo il puntatore** p_1.
- Se $d_2 < d_1$: il minimo è d_2. Si emette d_2 nei risultati e **avanza solo il puntatore** p_2.

Tracciamento Passo-Passo sull'Esempio di Lezione: Applicando l'unione sincronizzata sulle liste di `"information"` e `"retrieval"`:

- **Passo 1:** $p_1 \to 1$, $p_2 \to 1 \implies 1 == 1 \implies$ emette **1**. Avanzano entrambi ($p_1 \to 5$, $p_2 \to 5$).
- **Passo 2:** $p_1 \to 5$, $p_2 \to 5 \implies 5 == 5 \implies$ emette **5**. Avanzano entrambi ($p_1 \to 8$, $p_2 \to 6$).
- **Passo 3:** $p_1 \to 8$, $p_2 \to 6 \implies 6 < 8 \implies$ emette **6**. Avanza solo $p_2 \to 8$.
- **Passo 4:** $p_1 \to 8$, $p_2 \to 8 \implies 8 == 8 \implies$ emette **8**. Avanzano entrambi ($p_1 \to 11$, $p_2 \to 11$).
- **Passo 5:** $p_1 \to 11$, $p_2 \to 11 \implies 11 == 11 \implies$ emette **11**. Avanzano entrambi ($p_1 \to 13$, $p_2 \to 15$).
- **Passo 6:** $p_1 \to 13$, $p_2 \to 15 \implies 13 < 15 \implies$ emette **13**. Avanza solo $p_1 \to 20$.

Il processo prosegue linearmente emettendo l'unione rigorosamente ordinata:

$$\text{Results} = [1, 5, 6, 8, 11, 13, 15, 17, 20, 35, 40, 42, 50, 60].$$

Costo Computazionale delle Query OR: Il docente evidenzia la criticità intrinseca: **"OR query is expensive! We touch all the postings!"** Poiché ogni documento presente in anche una sola lista fa parte del risultato, è matematicamente impossibile scartare elementi. Il costo temporale è vincolato alla somma delle lunghezze:

$$T_{\text{OR}} = \Theta\left(\sum_{i=1}^{m} |\mathcal{L}(t_i)|\right). \tag{7.8}$$

7.4.2 DAAT per Query Congiuntive (AND): Algoritmo Base

Nelle query congiuntive (AND), un documento appartiene al risultato **se e solo se compare in tutte le posting list**:

$$\text{Results} = \mathcal{L}(t_1) \cap \mathcal{L}(t_2) \cap \cdots \cap \mathcal{L}(t_m). \tag{7.9}$$

L'algoritmo elementare a due puntatori scandisce le liste a passi unitari:

Listing 7.1: Algoritmo base di intersezione DAAT AND a due puntatori.

```
def intersect(p1, p2):
    i = 0
    j = 0
    r = []
    while i < len(p1) and j < len(p2):
        if p1[i] == p2[j]:
            r.append(p1[i])
            i += 1
            j += 1
            continue
        if p1[i] < p2[j]:
            i += 1
        else:
            j += 1
    return r
```

Limite di Inefficienza dell'Algoritmo Base: Nell'Algoritmo 7.1, ogni discrepanza ($p_1[i] \neq p_2[j]$) causa l'incremento di una sola posizione ($i \leftarrow i+1$ oppure $j \leftarrow j+1$). Se ad esempio $p_1[i] = 8$ e $p_2[j] = 100$, il puntatore i avanzerà faticosamente provando $9, 10, 11 \ldots$ fino a raggiungere 100. L'algoritmo tocca ogni singolo elemento fino all'esaurimento della lista più corta, disperdendo l'opportunità di compiere ampi salti in avanti.

7.5 La Primitiva di Salto 'nextGEQ' e Strutture di Skipping

Per superare la lentezza intrinseca della scansione sequenziale a passi unitari, l'accesso alle posting list viene potenziato introducendo primitive di salto condizionato. Questa sezione formalizza l'interfaccia a cursore basata sull'operatore nextGEQ, analizza le strategie di ricerca binaria e galloping search,

e presenta l'architettura industriale del partizionamento a blocchi compressi con puntatori di skip.

7.5.1 L'Interfaccia a Cursore su Posting List

Per consentire algoritmi di salto veloci, l'accesso a ciascuna posting list viene formalizzato tramite un'interfaccia a cursore dotata di 5 operazioni primitive:

Primitiva	**Semantica Operativa**
`first_t`()	Inizializza o reimposta il cursore sul primo docID della lista (indice 0).
`next_t`()	Avanza il puntatore di una singola posizione verso destra (passo unitario).
`nextGEQ_t`(d)	Sposta istantaneamente il cursore in avanti fino al **primo docID maggiore o uguale a** d: pos $= \min\{p \mid \text{list}[p] \geq d\}$. Se nessun docID soddisfa la condizione, posiziona oltre la fine (EOF).
`docId_t`()	Restituisce il valore numerico del docID attualmente puntato.
`position_t`()	Restituisce l'indice ordinale corrente del cursore all'interno dell'array $(0, 1, 2 \dots)$.

Tabella 7.1: Le 5 operazioni primitive dell'interfaccia a cursore su posting list.

Nota 7.5.1 (Utilità Architetturale di `position_t`()). Nei motori reali, le frequenze tf non sono interlacciate fisicamente con i docID, ma risiedono in uno stream parallelo disaccoppiato. Conoscere la posizione ordinale del cursore dei docID consente di accedere istantaneamente con costo $O(1)$ alla corrispondente frequenza locale nello stream parallelo.

7.5.2 Implementazione di 'nextGEQ': Galloping Search e Conflitto con la Compressione

Per implementare nextGEQ(d), una prima strategia algoritmica teorica consiste nell'applicare la **Ricerca Esponenziale** (*Exponential / Galloping Search*):

1. A partire dalla posizione corrente, si testano salti a distanze crescenti secondo potenze di due:

$$\text{distanza} \in \{2^0, 2^1, 2^2, 2^3, \dots\} = \{1, 2, 4, 8, 16, \dots\}.$$

2. Non appena si individua una potenza 2^i per cui list$[\text{pos} + 2^i] \geq d$, il target è circoscritto nell'intervallo ristretto $[\text{pos} + 2^{i-1}, \text{pos} + 2^i]$.

3. All'interno di tale segmento limitato si esegue una classica ricerca dicotomica standard.

4. Su t operazioni di salto lungo una lista di taglia n, il costo teorico è ottimale: $\Theta(t \log(n/t))$.

Perché la Ricerca Esponenziale Fallisce nei Sistemi Reali: Il docente spiega un principio cardine dell'ingegneria del software per IR:

- Le posting list occupano centinaia di gigabyte e **devono essere compresse** tramite differenziazione d-gaps e codifiche a lunghezza di bit variabile (VByte, Simple9, PForDelta).

- Gli algoritmi di compressione numerica producono flussi continui di bit che **non supportano l'accesso casuale** (*random access*): per conoscere l'elemento alla posizione 100, il processore è costretto a decomprimere sequenzialmente tutti i 99 elementi precedenti!

- Poiché la ricerca esponenziale richiede accessi casuali arbitrari, dover decomprimere dall'inizio a ogni salto vanifica qualunque beneficio algoritmico.

7.5.3 La Soluzione Industriale: Partizionamento a Blocchi e Skip Pointers

Per conciliare skipping rapido e compressione, le posting list vengono suddivise offline in **blocchi contigui indipendenti** di dimensione prefissata k (ad esempio $k = 64$ o $k = 128$ elementi):

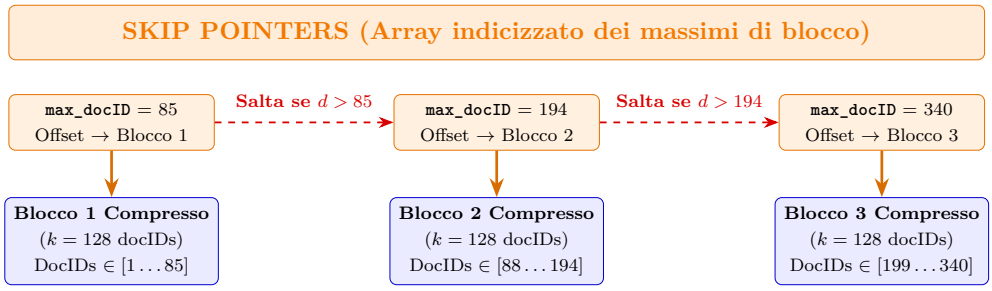


Figura 7.3: Architettura di skipping a blocchi: gli skip pointers consentono di bypassare la decompressione di interi blocchi non contenenti il target cercato.

- Ciascun blocco viene compresso e decompresso come **unità atomica indipendente**.
- In testa alla lista si memorizza un array di **Skip Pointers**, contenente per ogni blocco il valore massimo memorizzato ($\max_docID$) e l'indirizzo dell'offset di memoria.
- All'invocazione di nextGEQ(d), il sistema scansiona unicamente gli skip pointers: finché $\max_docID < d$, l'intero blocco viene **completamente saltato senza essere decompresso**.
- Non appena $\max_docID \geq d$, il bersaglio risiede necessariamente nel blocco corrente: solo allora il motore decomprime quel singolo blocco di k interi e vi esegue una ricerca lineare o dicotomica interna.
- Nei motori moderni, gli stessi skip pointers vengono codificati in strutture quasi-succinte come la rappresentazione di **Elias-Fano**, che supporta nativamente l'istruzione hardware ad altissima velocità di nextGEQ.

7.6 Algoritmi di Intersezione Avanzati con 'nextGEQ'

La disponibilità della primitiva di salto rapido trasforma radicalmente le prestazioni computazionali del query processing congiuntivo. Questa sezione analizza l'algoritmo di intersezione DAAT AND potenziato con nextGEQ, ne illustra il funzionamento pratico tramite un tracciamento su liste concrete, ed esamina l'approccio TAAT guidato dall'euristica *Shortest-List-First*.

7.6.1 Algoritmo DAAT AND Ottimizzato con 'nextGEQ'

Integrando la primitiva nextGEQ nell'algoritmo di intersezione a due puntatori, i passi unitari lenti vengono sostituiti da salti istantanei:

Listing 7.2: Intersezione DAAT AND ottimizzata con salti nextGEQ.

```
def intersect_nextGEQ(p1, p2):
    i = 0
    j = 0
    r = []
    while i < len(p1) and j < len(p2):
        if p1[i] == p2[j]:
```

```
            r.append(p1[i])
            i += 1
            j += 1
            continue
        if p1[i] < p2[j]:
            i = p1.nextGEQ(p2[j])
        else:
            j = p2.nextGEQ(p1[i])
    return r
```

7.6.2 Tracciamento Passo-Passo con Skipping sull'Esempio di Lezione

Siano p_1 (`"information"`) e p_2 (`"retrieval"`):

$$p_1 = [1, 5, 8, 11, 13, 20, 35, 40, 42],$$
$$p_2 = [1, 5, 6, 8, 11, 15, 17, 50, 60].$$

1. **Stato Iniziale:** Cursori su $i = 0, j = 0$ ($p_1[0] = p_2[0] = 1$). Match su **1**: si appende 1 a r. Si incrementa $i \leftarrow 1, j \leftarrow 1$.

2. **Passo 2:** $i = 1$ ($p_1[1] = 5$), $j = 1$ ($p_2[1] = 5$). $p_1[1] == p_2[1] \implies$ **Match su 5**. Inserisce 5 in r. Avanza $i \leftarrow 2$, $j \leftarrow 2$.

3. **Passo 3 (Primo Salto!):** $i = 2$ ($p_1[2] = 8$), $j = 2$ ($p_2[2] = 6$). Poiché $p_1[2] > p_2[2]$ ($8 > 6$), si invoca:

   $$j \leftarrow p_2.\text{nextGEQ}(p_1[i]) = p_2.\text{nextGEQ}(8).$$

   Il cursore j **salta istantaneamente** all'indice 3 di p_2 (valore 8), bypassando il controllo su 6!

4. **Passo 4:** $i = 2$ ($p_1[2] = 8$), $j = 3$ ($p_2[3] = 8$). $p_1[2] == p_2[3] \implies$ **Match su 8**. Inserisce 8 in r. Avanza $i \leftarrow 3$, $j \leftarrow 4$.

5. **Passo 5:** $i = 3$ ($p_1[3] = 11$), $j = 4$ ($p_2[4] = 11$). $p_1[3] == p_2[4] \implies$ **Match su 11**. Inserisce 11 in r. Avanza $i \leftarrow 4$, $j \leftarrow 5$.

6. **Passo 6 (Secondo Salto!):** $i = 4$ ($p_1[4] = 13$), $j = 5$ ($p_2[5] = 15$). Poiché $p_1[4] < p_2[5]$ ($13 < 15$), si invoca:

   $$i \leftarrow p_1.\text{nextGEQ}(p_2[j]) = p_1.\text{nextGEQ}(15).$$

   Nella lista p_1, dopo 13 il primo elemento ≥ 15 è **20** (indice 5). Il puntatore i salta direttamente a 20!

7. **Passo 7 (Terzo Salto!):** $i = 5$ ($p_1[5] = 20$), $j = 5$ ($p_2[5] = 15$). $20 > 15 \implies j \leftarrow p_2.\text{nextGEQ}(20)$, atterrando su **50** (indice 7 di p_2).

8. **Terminazione:** Le liste si esauriscono senza ulteriori match. Risultato finale: $[1, 5, 8, 11]$.

Confronto di Efficienza: AND vs. OR

La presenza della primitiva nextGEQ rovescia radicalmente i rapporti di forza computazionali:

"AND query is much more efficient than OR query because it skips a lot!"

Mentre la query OR è vincolata a scansionare tutti i posting di tutte le liste ($\sum |L_i|$), la query AND sfrutta i disallineamenti tra insiemi per saltare intere porzioni di posting list non contenenti elementi comuni.

7.6.3 Algoritmo TAAT AND Ottimizzato: L'Euristica Shortest-to-Longest

Il docente affronta un quesito cruciale: *È possibile sfruttare l'efficienza di nextGEQ anche nel paradigma Term-at-a-Time (TAAT)?*

La risposta è affermativa a patto di adottare l'euristica di **ordinamento preliminare per lunghezza crescente delle liste** (*from shortest to longest*):

1. Si consultano i valori df_{t_i} registrati nel dizionario.

2. Si ordinano le posting list della query in ordine rigorosamente crescente di cardinalità:
$$|\mathcal{L}_1| \leq |\mathcal{L}_2| \leq |\mathcal{L}_3| \leq \cdots \leq |\mathcal{L}_m|. \tag{7.10}$$

3. Si inizializza l'insieme soluzione temporaneo R con la lista più corta in assoluto:
$$R \leftarrow \mathcal{L}_1. \tag{7.11}$$

4. Per ciascuna lista successiva $\mathcal{L}_i$ ($i = 2, \ldots, m$), si filtrano gli elementi correnti di R: per ogni elemento $e \in R$, si esegue una ricerca mirata tramite salto:
$$j \leftarrow \mathcal{L}_i.\text{nextGEQ}(e).$$
Se $\mathcal{L}_i[j] == e$, il documento sopravvive nel nuovo insieme provvisorio R'; se $\mathcal{L}_i[j] > e$, e viene scartato definitivamente. Si aggiorna $R \leftarrow R'$.

Listing 7.3: Algoritmo TAAT AND ottimizzato con ordinamento Shortest-to-Longest e salti nextGEQ.

```
def intersect_nextGEQ_TAAT(term_lists):
    # term_lists pre-ordinata per lunghezza crescente: |L1| <= |L2| <= ...
    r = term_lists[0]
    for cur_list in term_lists[1:]:
        cur_r = []
        for e in r:
            j = cur_list.nextGEQ(e)
            if j < len(cur_list) and e == cur_list[j]:
                cur_r.append(e)
        r = cur_r
    return r
```

7.6.4 Dimostrazione della Monotonia Decrescente e Limite di TAAT AND

Teorema 7.6.1 (Monotonia Decrescente della Cardinalità dei Risultati Intermedi)**.** *La dimensione dell'insieme temporaneo dei candidati R è una sequenza debolmente monotona decrescente vincolata superiormente dalla cardinalità della lista più corta:*

$$|R_m| \leq |R_{m-1}| \leq \cdots \leq |R_1| \leq |\mathcal{L}_1| = \min_{t \in q} |\mathcal{L}(t)|. \tag{7.12}$$

Dimostrazione. Per definizione di intersezione insiemistica, $R_i = R_{i-1} \cap \mathcal{L}_i \subseteq R_{i-1}$, da cui discende $|R_i| \leq |R_{i-1}|$ per ogni passo $i \geq 2$. Poiché $R_1 = \mathcal{L}_1$, la dimensione massima è superiormente limitata da $|\mathcal{L}_1|$. □

Vantaggio Computazionale: Iniziando dalla lista più corta, il numero massimo di iterazioni del ciclo esterno è limitato dal minimo possibile ($|\mathcal{L}_1|$). Sulle liste successive $\mathcal{L}_2, \mathcal{L}_3 \ldots$ (che possono contenere milioni di documenti) **non viene effettuata alcuna scansione lineare**: ci si limita a eseguire al più $|R|$ chiamate nextGEQ, elidendo milioni di documenti irrilevanti a costo pressoché nullo.

Il Limite Strutturale di TAAT AND: L'unico limite intrinseco di questa tecnica risiede nella natura dell'insieme intermedio R: esso viene generato dinamicamente al volo (*online*) come un array temporaneo in RAM. Tale array non possiede indici di blocco o skip pointers: di conseguenza,

è possibile eseguire chiamate nextGEQ sulla lista della collezione $\mathcal{L}_i$ (che è statica e indicizzata), ma **non è possibile fare salti all'indietro sul risultato provvisorio** R. Ciononostante, l'euristica rende l'intersezione TAAT straordinariamente competitiva ed efficiente.

7.7 Rilevanza nei Motori Vettoriali e Sistemi Moderni (Vector Search)

Il dualismo tra TAAT e DAAT e le relative strategie di skipping non appartengono unicamente alla tradizione dell'IR classico, ma definiscono le architetture dei moderni sistemi di recupero neurale e geometrico:

- **Sparse Neural Retrieval (es. SPLADE [6]):** Quando modelli di deep learning predicono distribuzioni di pesi sparsi su vocabolari estesi, i motori riadattano indici invertiti processati con algoritmi DAAT potenziati da **WAND** (*Weak AND*) o **Block-Max WAND**, saltando dinamicamente i blocchi il cui limite superiore (*upper bound*) di punteggio non può superare la soglia minima corrente del Min-Heap.
- **Acceleratori Tensoriali e GPU:** In ambienti basati su GPU, il paradigma TAAT torna fortemente competitivo per via dell'altissimo throughput garantito da operazioni vettoriali contigue su blocchi di memoria regolari, dove la latenza dei salti complessi di DAAT risulterebbe penalizzante per il parallelismo SIMD.
- **Vector Search Engines Dedicati (es. Qdrant):** Motori di ricerca vettoriale moderni integrano indici invertiti di supporto per combinare filtri booleani e metadati con indici di prossimità geometrica (k-ANN su grafi HNSW), riutilizzando i pattern di scansione DAAT/TAAT per garantire un'efficienza congiunta nel primo stadio di recupero.

7.8 Quadro Sinottico Comparativo degli Algoritmi

A conclusione dell'analisi, la Tabella 7.2 riassume le proprietà architetturali, i vincoli di memoria e le capacità di salto dei paradigmi analizzati:

Algoritmo	Costo Computazionale	Occupazione RAM	Capacità di Salto
TAAT Base	$\Theta(\sum \lvert\mathcal{L}_i\rvert)$	Elevata ($\mathcal{O}(\lvert\bigcup \mathcal{L}_i\rvert)$)	**Nessuna:** scansione esaustiva obbligatoria.
DAAT OR	$\Theta(\sum \lvert\mathcal{L}_i\rvert)$	Minima ($\mathcal{O}(k)$ per Top-k)	**Nessuna:** tutti i posting vengono toccati.
DAAT AND Base	$\mathcal{O}(\sum \lvert\mathcal{L}_i\rvert)$	Minima ($\mathcal{O}(k)$ heap, $\mathcal{O}(m)$ cursori)	**Nessuna:** avanzamento a passi unitari +1.
DAAT AND nextGEQ	Sublineare: $\mathcal{O}\Big(\lvert\mathcal{L}_{\min}\rvert \log \frac{\lvert\mathcal{L}_{\max}\rvert}{\lvert\mathcal{L}_{\min}\rvert}\Big)$	Minima ($\mathcal{O}(k)$ heap, $\mathcal{O}(m)$ cursori)	**Eccellente:** salti bidirezionali sincroni su tutte le liste.
TAAT AND nextGEQ	Sublineare: $\mathcal{O}(\lvert\mathcal{L}_1\rvert + \sum_{i>1} c_i)$ con $c_i = \lvert R_{i-1}\rvert \log \frac{\lvert\mathcal{L}_i\rvert}{\lvert R_{i-1}\rvert}$	Moderata ($\mathcal{O}(\lvert R\rvert)$ array dinamico)	**Asimmetrica:** salti su $\mathcal{L}_i$, scansione su R.

Tabella 7.2: Quadro sinottico comparativo degli algoritmi di scansione e intersezione in Information Retrieval.

Capitolo 8

Elaborazione Efficiente di Query Top-k: WAND, MaxScore e Varianti a Blocchi

8.1 Architettura di Retrieval a Due Stadi e Motivazione del Top-k

L'elaborazione delle interrogazioni su collezioni web di scala planetaria impone di superare i limiti dell'approccio booleano esaustivo. Questa sezione analizza le asimmetrie di costo computazionale tra query congiuntive e disgiuntive, formalizza la gerarchia a imbuto dell'architettura di reperimento a due stadi e chiarisce le motivazioni comportamentali che rendono prioritario il calcolo efficiente dei soli migliori k documenti.

8.1.1 Limiti Computazionali delle Query Booleane Tradizionali

Nell'architettura standard di un sistema di Information Retrieval, a partire da un corpus documentale viene costruito offline l'indice invertito (*inverted index*), interrogato online durante la fase di query processing:

- **Query Congiuntive (`AND`):** Operano calcolando l'intersezione tra le posting list associate ai termini dell'interrogazione. Come dimostrato nella Sezione 7.6, l'algoritmo sfrutta intensivamente la primitiva nextGEQ(d) per compiere salti significativi, scavalcando blocchi estesi di documenti che non contengono contemporaneamente tutti i termini ricercati. La cardinalità dell'intersezione è limitata superiormente dalla

lista più corta, rendendo il processing rapido e computazionalmente scalabile.

- **Query Disgiuntive (`OR`):** Richiedono l'unione insiemistica delle posting list:
$$\mathcal{D}_{\text{cand}} = \bigcup_{t \in Q} \mathcal{L}(t). \tag{8.1}$$
Un approccio esaustivo è costretto ad accedere fisicamente, decomprimere e valutare il contributo di frequenza di ogni singolo posting presente in ciascuna lista. Per collezioni web composte da miliardi di documenti, le posting list dei termini più comuni contengono decine o centinaia di milioni di elementi. L'elaborazione esaustiva comporta costi I/O proibitivi e decine di milioni di accessi in memoria, determinando latenze dell'ordine dei secondi, incompatibili con i requisiti stringenti di reattività online ($< 50\,\text{ms}$).

8.1.2 Motivazione Comportamentale e il Primo Stadio di Retrieval

Come formalizzato nella Sezione 6.1 e illustrato nella Figura 6.1, i motori di ricerca su scala planetaria adottano un'**architettura di reperimento a due stadi** (*Two-Stage Retrieval*):

1. **Primo Stadio (Candidate Retrieval / Top-k):** estrae rapidamente un insieme compatto di candidati ($k \approx 1\,000$) valutati secondo una funzione di scoring additiva economica (Okapi BM25).

2. **Secondo Stadio (Re-ranking / Learning-to-Rank):** applica modelli di Machine Learning complessi e costosi sui soli candidati pre-filtrati per produrre i primi 10–20 risultati finali della SERP.

La ragione fondamentale per cui in ambito industriale è sufficiente estrarre i soli migliori $k \approx 1\,000$ documenti per il primo stadio risiede nell'analisi empirica del comportamento degli utenti sul web:

- L'attenzione dell'utente medio si concentra quasi esclusivamente sulla prima pagina dei risultati (i primi 10 documenti) o, al più, sulla seconda pagina (fino alla posizione 20).

- Qualora i primi 20 risultati non soddisfino il bisogno informativo, l'utente non scorre le centinaia di pagine successive della graduatoria, ma interrompe la sessione o riformula la query modificando i termini di ricerca.

Obiettivo Fondamentale

L'obiettivo algoritmico consiste nell'ottimizzare il **Primo Stadio**: estrarre i migliori k documenti per una query disgiuntiva (`OR`) garantendo un risultato matematicamente identico alla scansione esaustiva (*Exact Top-k*), ma evitando la decompressione e la valutazione dell'immensa maggioranza dei posting dell'indice.

8.2 Exact Top-k Retrieval e l'Algoritmo Esaustivo con Min-Heap

La ricerca dei migliori risultati richiede la gestione efficiente di una soglia dinamica di ammissione durante la scansione dei documenti. Questa sezione formalizza matematicamente il problema dell'Exact Top-k Retrieval, giustifica l'adozione strutturale di un Min-Heap di dimensione fissa k per il tracciamento della soglia τ, e ne simula l'esecuzione tramite una dettagliata traccia numerica passo-passo.

8.2.1 Definizione Formale del Problema Top-k

Definizione 8.2.1 (Exact Top-k Retrieval)**.** Siano dati una collezione documentale indicizzata $\mathcal{D}$, una query testuale $Q = \{t_1, t_2, \ldots, t_m\}$, un intero $k \geq 1$ e una funzione di scoring additiva $s(Q, d)$ definita come:

$$s(Q, d) = \sum_{t \in Q \cap d} s(t, d), \tag{8.2}$$

dove $s(t, d) > 0$ rappresenta il punteggio parziale assegnato al documento d per il termine t (ad esempio il peso parziale Okapi BM25 o TF-IDF). Il problema dell'**Exact Top-k Retrieval** richiede di individuare un sottoinsieme ordinato $R_k \subseteq \mathcal{D}$ tale che $|R_k| = \min(k, |\mathcal{D}|)$ e:

$$\forall d \in R_k,\ \forall d' \in \mathcal{D} \setminus R_k \implies s(Q, d) \geq s(Q, d'). \tag{8.3}$$

La specifica **Exact** stabilisce che il sottoinsieme R_k e i rispettivi punteggi devono coincidere rigorosamente con quelli ottenibili tramite la scansione esaustiva di tutti i documenti candidati, ammettendo **zero falsi negativi** (nessun documento che appartiene teoricamente al Top-k può essere omesso).

8.2.2 Gestione della Priorità: Il Ruolo del Min-Heap

Per estrarre i k valori massimi da un flusso non ordinato di punteggi, la struttura dati ottimale è una coda con priorità implementata tramite un

Min-Heap a capacità fissa k. A una prima analisi superficiale, l'impiego di un Min-Heap per individuare i massimi può apparire controintuitivo. La spiegazione risiede nella semantica della radice:

- La radice di un Min-Heap contiene, per definizione strutturale, l'elemento con il punteggio **minimo** tra i k elementi correntemente memorizzati.
- Tale valore minimo definisce la **soglia dinamica di ammissione** τ:

$$\tau = \min_{d \in \text{Heap}} s(Q, d). \tag{8.4}$$

- La soglia τ rappresenta il costo d'ingresso: qualunque nuovo documento candidato d' può entrare a far parte dei Top-k provvisori se e solo se il suo punteggio supera strettamente la soglia:

$$s(Q, d') > \tau. \tag{8.5}$$

- Se $s(Q, d') \leq \tau$, il documento non ha alcuna possibilità di qualificarsi e viene scartato immediatamente senza modificare l'heap.
- Se $s(Q, d') > \tau$, l'elemento alla radice (il peggiore tra i migliori finora osservati) viene estratto tramite extract-min(), il nuovo candidato viene inserito con insert$(d', s(Q, d'))$, e l'albero viene ripristinato in tempo logaritmico $\mathcal{O}(\log k)$. La soglia τ si aggiorna assumendo il valore della nuova radice: la soglia è una funzione **monotona non decrescente** del numero di candidati valutati con successo.

Proprietà 8.2.1 (Complessità del Min-Heap per Flusso di Dati)**.** *Dato un flusso sequenziale di n elementi, il costo computazionale per mantenere i k elementi massimi tramite Min-Heap di capacità k è limitato superiormente da $\mathcal{O}(n \log k)$. Lo spazio ausiliario in memoria centrale è strettamente $\mathcal{O}(k)$, indipendente dal numero totale di documenti presenti nella collezione o nella disgiunzione.*

Esempio 8.2.1 (Evoluzione della Soglia τ su Sequenza Numerica)**.** Si consideri il flusso di punteggi in arrivo $[2.5, 1.5, 3.0, 0.5]$ con $k = 3$. Si assuma che il Min-Heap contenga inizialmente i valori provvisori $\{2.1, 2.3, 3.1\}$.

1. La radice contiene il valore 2.1, da cui $\tau = 2.1$.
2. Arriva il punteggio 2.5: poiché $2.5 > 2.1$, la radice 2.1 viene rimossa e viene inserito 2.5. L'heap si riorganizza in $\{2.3, 2.5, 3.1\}$ e la nuova radice impone $\tau = 2.3$.

3. Arriva il punteggio 1.5: poiché $1.5 \leq \tau = 2.3$, il valore viene scartato all'istante senza alcuna operazione sull'albero.

4. Arriva il punteggio 3.0: poiché $3.0 > 2.3$, si estrae 2.3 e si inserisce 3.0. L'heap diviene $\{2.5, 3.0, 3.1\}$ e la soglia sale a $\tau = 2.5$.

5. Arriva il punteggio 0.5: poiché $0.5 \leq 2.5$, viene scartato.

Al termine, l'heap racchiude fedelmente i 3 punteggi massimi $\{2.5, 3.0, 3.1\}$.

8.2.3 Simulazione dell'Algoritmo Esaustivo DAAT con Min-Heap

L'esecuzione dell'Exact Top-k con approccio esaustivo segue lo schema di scansione sincrona Document-at-a-Time già formalizzato nell'**Algoritmo 2** (Capitolo 7). Per evidenziare empiricamente il sovraccarico computazionale di tale strategia, si consideri una query composta da $m = 4$ termini:

$$Q = \{\texttt{"rust"}, \texttt{"best"}, \texttt{"programming"}, \texttt{"language"}\}.$$

Le rispettive posting list contengono le coppie (docID, $s(t, d)$):

$$\begin{aligned}
\mathcal{L}(\texttt{"rust"}) &= [(15, 2.5), (16, 1.5), (25, 2.0), (45, 1.5), \dots] \\
\mathcal{L}(\texttt{"best"}) &= [(11, 0.3), (12, 0.1), (13, 0.1), (15, 0.2), (21, 1.0), \dots] \\
\mathcal{L}(\texttt{"programming"}) &= [(13, 0.5), (15, 1.0), (19, 1.0), (21, 1.0), \dots] \\
\mathcal{L}(\texttt{"language"}) &= [(10, 0.5), (13, 0.9), (25, 0.8), (29, 1.1), \dots]
\end{aligned}$$

Si imposti $k = 1$ (ricerca del Top-1) e si assuma che l'heap sia pre-popolato con un documento precedente $d = 8$ avente punteggio 2.1, da cui deriva la soglia iniziale $\tau = 2.1$.

Nel paradigma DAAT esaustivo, a ogni iterazione l'algoritmo calcola $d_{\min} = \min_i p_i.\text{docID}$, aggrega i contributi di tutte le liste che contengono $d_{\min}$, e confronta il punteggio ottenuto con la soglia τ:

1. **Candidato Doc 10:** Compare unicamente in $\mathcal{L}(\texttt{"language"})$. Punteggio: $s = 0.5$. Poiché $0.5 < 2.1$, viene scartato. L'heap rimane invariato ($\tau = 2.1$).

2. **Candidato Doc 11:** Compare unicamente in $\mathcal{L}(\texttt{"best"})$. Punteggio: $s = 0.3$. Scartato ($0.3 < 2.1$).

3. **Candidato Doc 12:** Compare unicamente in $\mathcal{L}(\texttt{"best"})$. Punteggio: $s = 0.1$. Scartato ($0.1 < 2.1$).

4. **Candidato Doc 13:** Compare simultaneamente in tre liste: $\mathcal{L}$("language") (0.9), $\mathcal{L}$("best") (0.1) e $\mathcal{L}$("programming") (0.5). Punteggio reale aggregato:
$$s(Q, 13) = 0.9 + 0.1 + 0.5 = 1.5.$$
Confronto: $1.5 < 2.1 \implies$ documento scartato. L'heap rimane invariato ($\tau = 2.1$).

5. **Candidato Doc 15:** Compare in tre liste: $\mathcal{L}$("rust") (2.5), $\mathcal{L}$("programming") (1.0) e $\mathcal{L}$("best") (0.2). Punteggio reale aggregato:
$$s(Q, 15) = 2.5 + 1.0 + 0.2 = 3.7.$$
Confronto: $3.7 > 2.1 \implies$ il documento entra nell'heap sostituendo Doc 8. La nuova soglia minima sale istantaneamente a $\tau = 3.7$.

DocID	Liste Coinvolte	Calcolo Punteggio Effettivo	Verifica Soglia (τ)	Stato Heap / Nuova τ
Doc 10	"language"	$s = 0.5$	$0.5 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 11	"best"	$s = 0.3$	$0.3 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 12	"best"	$s = 0.1$	$0.1 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 13	"language", "best", "prog"	$s = 0.9 + 0.1 + 0.5 = 1.5$	$1.5 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 15	"rust", "prog", "best"	$s = 2.5 + 1.0 + 0.2 = 3.7$	$3.7 > 2.1 \implies$ Inserito	Doc 15 ($\tau = 3.7$)

Tabella 8.1: Tracciamento della scansione esaustiva DAAT con Min-Heap per i primi 5 documenti candidati.

8.2.4 Limite Computazionale della Strategia Esaustiva

L'algoritmo esaustivo calcola correttamente il Top-k, ma soffre del difetto cardine delle interrogazioni disgiuntive: **accede e decomprime ogni singolo posting dell'indice**. Nella pratica reale, man mano che la scansione avanza, la soglia τ cresce rapidamente nei primissimi millisecondi di elaborazione. Di conseguenza, oltre il 95%–99% dei documenti successivamente esaminati possiede un punteggio di gran lunga inferiore a τ: la CPU esegue milioni di letture da memoria, decompressioni e somme scalari per documenti destinati inesorabilmente a essere scartati. Il **Dynamic Pruning** (potatura dinamica) nasce esattamente per superare questo collo di bottiglia: identificare ed escludere a priori blocchi estesi di documenti non competitivi senza mai toccarli né decomprimerli in memoria.

8.3 L'Algoritmo WAND (Weak AND) e il Dynamic Pruning

Per evitare la valutazione esaustiva di decine di milioni di documenti destinati a essere scartati, il dynamic pruning sfrutta stime maggioranti per scavalcare intere porzioni di indice. Questa sezione introduce l'algoritmo fondazionale WAND, ne definisce gli Upper Bound di termine, descrive la procedura di selezione del pivot ed esamina una simulazione completa dell'algoritmo.

8.3.1 Riferimento Bibliografico e Concetti Fondamentali

L'algoritmo **WAND** (*Weak AND*) è stato introdotto nel lavoro fondamentale di Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer e Jason Y. Zien [9] (CIKM 2003). WAND costituisce la prima architettura algoritmica di potatura dinamica in grado di garantire formalmente l'Exact Top-k su query `OR`, saltando la scansione e la decompressione della maggior parte dei posting delle liste.

Il pilastro teorico di WAND è l'introduzione dell'**Upper Bound di Termine (UB)**, definito come il massimo punteggio parziale che il termine può apportare a un qualsiasi documento all'interno della sua posting list:

Definizione 8.3.1 (Term Upper Bound $UB(t)$)**.** Dato un termine di vocabolario $t \in \mathcal{V}$ associato alla posting list $\mathcal{L}(t)$, il suo Upper Bound $UB(t)$ è calcolato offline durante l'indicizzazione come:

$$UB(t) = \max_{d \in \mathcal{L}(t)} s(t, d). \tag{8.6}$$

Costo in Spazio e Requisiti di Memorizzazione. La memorizzazione degli UB presenta proprietà estremamente favorevoli:

- Per ogni termine $t \in \mathcal{V}$ è sufficiente memorizzare un unico valore float a 32 bit indicante $UB(t)$.
- L'ingombro di memoria ausiliaria è proporzionale alla dimensione del lessico $|\mathcal{V}|$, non alla dimensione complessiva dell'indice inverso. Per un vocabolario tipico di alcuni milioni di termini, l'array degli UB occupa pochissimi megabyte e risiede stabilmente nella RAM centrale del broker di query.

- Al contrario, pre-calcolare e salvare i punteggi esatti all'interno di ciascun record della posting list su disco risulterebbe disastroso: i numeri a virgola mobile hanno entropia elevata e si comprimono male, compromettendo l'efficienza dei codec per interi (d-gaps).

8.3.2 Proprietà di Maggiorazione e Condizione di Scarto Sicuro

Teorema 8.3.1 (Condizione di Scarto Sicuro di WAND)**.** *Siano dati una query Q, un documento candidato $d \in \mathcal{D}$ e la soglia corrente del Min-Heap τ. Il punteggio reale $s(Q,d)$ è limitato superiormente dalla somma degli Upper Bound dei termini di query:*

$$s(Q,d) = \sum_{t \in Q \cap d} s(t,d) \leq \sum_{t \in Q \cap d} UB(t) \leq \sum_{t \in Q} UB(t). \tag{8.7}$$

Se un insieme di termini attivi $\mathcal{T} \subseteq Q$ soddisfa la condizione:

$$\sum_{t \in \mathcal{T}} UB(t) < \tau, \tag{8.8}$$

*allora nessun documento d che riceve contributi di scoring **esclusivamente** dai termini in $\mathcal{T}$ può superare la soglia d'ingresso dell'heap:*

$$s(Q,d) \leq \sum_{t \in \mathcal{T}} UB(t) < \tau. \tag{8.9}$$

*Tali documenti possono essere **scartati in modo sicuro** (safely skipped), garantendo l'assenza totale di falsi negativi nell'insieme Top-k finale.*

8.3.3 Modello di Costo Hardware: CPU Reasoning vs Pointer Movements

La progettazione di WAND riflette la fisica delle moderne architetture di calcolo, come formalizzato nel Capitolo 2:

- **Operazioni ad Alto Costo (Pointer Movements):** Lo scorrimento fisico di un cursore su una posting list compressa ('next()' o 'nextGEQ()') impone il caricamento di blocchi da memoria DRAM o disco, il fallimento dei predittori hardware per salti non contigui (*cache misses*), e l'esecuzione di cicli di decompressione software (PForDelta, Elias-Fano, SIMD).

- **Operazioni a Basso Costo (CPU Reasoning):** I calcoli eseguiti dalla CPU su registri interni e cache L1 (somme tra float, confronti scalari con τ, riordinamento di un piccolo array contenente i 3–5 puntatori della query) richiedono pochi cicli di clock ($<$ 1–5 ns).
- **Principio Guida:** Conviene sempre impiegare decine di istruzioni di calcolo nella CPU per verificare se un documento può essere scartato matematicamente, se ciò consente di evitare anche una sola lettura o decompressione di un blocco di posting list.

8.3.4 Meccanismo Operativo di WAND

Ad ogni iterazione, l'algoritmo opera secondo 4 fasi consequenziali:

1. **Ordinamento dei Puntatori per DocID Corrente:** L'array delle posting list attive viene ordinato in ordine crescente in base all'identificativo del documento puntato:

$$d_1 \leq d_2 \leq \cdots \leq d_m. \tag{8.10}$$

2. **Identificazione del Documento Pivot ($d_{\mathbf{pivot}}$):** Si scorrono le liste ordinate da sinistra a destra, accumulando i rispettivi $UB(t_i)$ fino a individuare il primo indice p (il *pivot*) per cui la somma cumulativa eguaglia o supera la soglia τ:

$$\sum_{i=1}^{p} UB(t_i) \geq \tau. \tag{8.11}$$

   Se la somma complessiva di tutti gli m termini di query risulta strettamente inferiore a τ ($\sum_{i=1}^{m} UB(t_i) < \tau$), nessun documento residuo può qualificarsi e l'algoritmo termina immediatamente.

3. **Proprietà dello Scarto Iniziale:** Per costruzione del pivot p, i documenti puntati dalle liste precedenti $1, \ldots, p-1$ accumulano una somma ottimistica strettamente inferiore a τ:

$$\sum_{i=1}^{p-1} UB(t_i) < \tau.$$

   Di conseguenza, nessun documento $d < d_p$ ha il potenziale matematico per entrare nell'heap.

4. **Azione sul Pivot Document ($d_{\mathbf{pivot}} = d_p$):**

- **Caso $d_1 == d_{\text{pivot}}$:** La prima lista punta esattamente al documento pivot. Ciò implica che $d_1 = d_2 = \cdots = d_p = d_{\text{pivot}}$: il documento compare in un numero sufficiente di liste la cui somma di UB supera τ. Il documento possiede un potenziale concreto: viene **valutato pienamente** (*fully evaluated*), calcolando la somma dei punteggi esatti per tutte le liste che lo contengono. Se $s(Q, d_{\text{pivot}}) > \tau$, entra nell'heap e τ viene incrementata. Infine, il puntatore della prima lista avanza al docID successivo.
- **Caso $d_1 < d_{\text{pivot}}$:** La prima lista punta a un documento d_1 strettamente inferiore al pivot. Dal Teorema 8.3.1, nessun documento inferiore a d_{pivot} può raggiungere la soglia τ. L'algoritmo invoca direttamente:

  $$p_1.\text{nextGEQ}(d_{\text{pivot}}), \tag{8.12}$$

  scavalcando in un singolo salto tutti i documenti compresi nell'intervallo $[d_1, d_{\text{pivot}} - 1]$ senza decomprimerli.

8.3.5 Tracciamento Dettagliato dell'Algoritmo WAND

Si consideri lo scenario operativo di riferimento con $m = 4$ termini di query:

- **Termini e rispettivi Upper Bound:**

$$\begin{aligned} UB(\texttt{"rust"}) &= 2.5 \\ UB(\texttt{"language"}) &= 1.1 \\ UB(\texttt{"programming"}) &= 0.6 \\ UB(\texttt{"best"}) &= 0.3 \end{aligned}$$

- **Soglia Iniziale del Min-Heap ($k = 1$):** $\tau = 2.1$.
- **Puntatori Correnti delle Liste:**
  - $\mathcal{L}(\texttt{"language"})$ punta a **10**
  - $\mathcal{L}(\texttt{"best"})$ punta a **11**
  - $\mathcal{L}(\texttt{"programming"})$ punta a **13**
  - $\mathcal{L}(\texttt{"rust"})$ punta a **15**

L'array ordinato per docID corrente risulta:

$$[\,\langle \texttt{"language"}, 10, 1.1\rangle,\ \langle \texttt{"best"}, 11, 0.3\rangle,\ \langle \texttt{"programming"}, 13, 0.6\rangle,\ \langle \texttt{"rust"}, 15, 2.5\rangle\,].$$

Fase 1: Determinazione del Pivot.

1. Si esamina la prima lista (`"language"`, $d_1 = 10$):

$$\text{Somma} = UB(\texttt{"language"}) = 1.1 < \tau = 2.1 \quad \text{(sotto la soglia)}.$$

2. Si aggiunge la seconda lista (`"best"`, $d_2 = 11$):

$$\text{Somma} = 1.1 + UB(\texttt{"best"}) = 1.4 < \tau = 2.1 \quad \text{(sotto la soglia)}.$$

3. Si aggiunge la terza lista (`"programming"`, $d_3 = 13$):

$$\text{Somma} = 1.4 + UB(\texttt{"programming"}) = 2.0 < \tau = 2.1.$$

   Osservazione critica: anche ipotizzando che un documento compaia simultaneamente in `"language"`, `"best"` e `"programming"` e ottenga il massimo punteggio teorico in ciascuna di esse, la sua somma cumulativa (2.0) è rigorosamente inferiore alla soglia $\tau = 2.1$!

4. Si aggiunge la quarta lista (`"rust"`, $d_4 = 15$):

$$\text{Somma} = 2.0 + UB(\texttt{"rust"}) = 2.0 + 2.5 = 4.5 \geq \tau = 2.1.$$

   La condizione è soddisfatta con $p = 4$. Il **documento pivot è** $d_{\mathbf{pivot}} = d_4 = 15$.

Fase 2: Valutazione del Documento 15.

- Poiché la prima lista puntava a $d_1 = 10 < 15$, tutti i documenti intermedi $(10, 11, 12, 13)$ vengono scartati all'istante senza calcolo del punteggio.
- I cursori avanzano con nextGEQ(15).
- Si procede alla valutazione completa di Doc 15:

$$\begin{aligned} s(\texttt{"rust"}, 15) &= 2.5 \\ s(\texttt{"programming"}, 15) &= 1.0 \\ s(\texttt{"best"}, 15) &= 0.2 \\ s(\texttt{"language"}, 15) &= 0.0 \quad \text{(termine non presente nel documento)} \end{aligned}$$

  Punteggio aggregato: $s(Q, 15) = 2.5 + 1.0 + 0.2 = 3.7$.

- Poiché $3.7 > 2.1$, Doc 15 viene inserito nel Min-Heap e la soglia di ammissione sale a $\tau = 3.7$.

Lo pseudocodice di WAND è formalizzato nell'Algoritmo 3.

Algorithm 3: Algoritmo WAND (Weak AND) Top-k Retrieval

Dati: Query $Q = \{t_1, \ldots, t_m\}$, Indice Invertito, array di Upper Bound $UB[t]$, intero k
Risultato: Min-Heap H contenente i Top-k documenti

```
1  H ← InizializzaMinHeap(capacità = k)
2  τ ← 0
3  Liste ← InizializzaCursori(Q)
4  while true do
5      Liste.SortByCurrentDocID()
6      acc ← 0;    pivot ← −1
7      for i ← 1 to |Liste| do
8          acc ← acc + UB[Liste[i].term]
9          if acc ≥ τ then
10             pivot ← i
11             break
12     if pivot == −1 then
13         break      // Nessun documento residuo può superare τ
14     d_pivot ← Liste[pivot].CurrentDocID()
15     if Liste[1].CurrentDocID() == d_pivot then
16         s_real ← 0
17         for i ← 1 to |Liste| do
18             if Liste[i].CurrentDocID() == d_pivot then
19                 s_real ← s_real + Liste[i].CurrentScore()
20                 Liste[i].Next()
21         if s_real > τ then
22             if |H| == k then
23                 H.ExtractMin()
24             H.Insert(⟨d_pivot, s_real⟩)
25             if |H| == k then
26                 τ ← H.FindMin().score
27     else
28         Liste[1].nextGEQ(d_pivot)          // Salto sicuro oltre i
               documenti non competitivi
29 return H
```

8.4 Analisi Critica e Casi Limite nel Dynamic Pruning

L'analisi del comportamento di WAND solleva quattro questioni teoriche e ingegneristiche fondamentali, che ne chiariscono i limiti operativi e le garanzie algebriche:

8.4.1 Semantica Disgiuntiva e Trattamento dei Termini Assenti

Nella simulazione di WAND, il documento pivot $d_{\text{pivot}} = 15$ viene valutato e inserito nell'heap pur non contenendo il termine `"language"`. Questa circostanza non costituisce un'anomalia logica:

- Nelle query disgiuntive (`OR`), la semantica di retrieval stabilisce che un documento è rilevante se la combinazione lineare dei pesi dei termini effettivamente presenti supera la soglia di ammissione τ.
- A differenza della semantica congiuntiva (`AND`) — in cui l'assenza di un termine impone l'esclusione immediata del documento —, nella disgiunzione il contributo del termine non presente è semplicemente nullo ($s(\texttt{"language"}, 15) = 0$). L'inclusione è legittimata dal fatto che la somma parziale dei termini rimanenti ($2.5 + 1.0 + 0.2 = 3.7$) supera comunque la soglia $\tau = 2.1$.

8.4.2 Analisi di Convenienza: WAND su Query Congiuntive (`AND`)

Sebbene la maggiorazione algebrica basata su Upper Bound rimanga formalmente corretta anche per l'operatore `AND`, l'applicazione di WAND a query congiuntive risulta nella pratica inefficiente e superflua:

1. Nelle query `AND`, l'algoritmo di salto deterministico basato sull'operatore nextGEQ sul massimo dei docID correnti (analizzato nella Sezione 7.6) è già asintoticamente sublineare e non richiede il calcolo di somme ottimistiche tra numeri float.
2. Nelle query congiuntive la cardinalità dell'intersezione è fortemente ridotta: pochissimi documenti soddisfano la condizione di presenza simultanea di tutti i termini. L'heap riceve quindi un numero esiguo di candidati, facendo crescere la soglia τ molto lentamente e vanificando il potenziale di potatura dinamica. Il guadagno computazionale

esponenziale di WAND si realizza specificamente sulle query `OR`, dove l'approccio esaustivo comporterebbe la scansione di milioni di record.

8.4.3 Trade-off Computazionale sui Salti Intermedi Anticipati

Durante la scansione, non appena la CPU rileva che il contributo cumulativo delle prime liste non supera τ, si potrebbe ipotizzare di eseguire salti intermedi anticipati (nextGEQ) anche sulle liste successive prima di raggiungere il pivot. Tuttavia, l'implementazione pratica deve bilanciare un compromesso architetturale:

- Ogni invocazione di avanzamento altera la posizione del cursore e impone il riordinamento dell'array dei puntatori per preservare l'invariante $d_1 \leq d_2 \leq \cdots \leq d_m$.
- Se la CPU esegue salti intermedi troppo aggressivi, il costo dei cicli spesi nei confronti logici e nel continuo riordinamento in cache/registri può eccedere il tempo risparmiato sull'indice compresso.

8.4.4 Dimostrazione Algebrica dello Scarto a Priori (Doc 12)

Si consideri lo stato dei puntatori all'inizio della scansione:

- Il cursore di $\mathcal{L}(\texttt{"programming"})$ è attestato su Doc 13 $\implies$ Doc 12 non può comparire in $\mathcal{L}(\texttt{"programming"})$.
- Il cursore di $\mathcal{L}(\texttt{"rust"})$ è attestato su Doc 15 $\implies$ Doc 12 non può comparire in $\mathcal{L}(\texttt{"rust"})$.

Doc 12 potrebbe comparire unicamente nelle liste con cursori ≤ 12, ossia $\mathcal{L}(\texttt{"language"})$ e $\mathcal{L}(\texttt{"best"})$. Il massimo punteggio teorico conseguibile da Doc 12 è maggiorato dalla somma dei rispettivi Upper Bound:

$$s(Q, 12) \leq UB(\texttt{"language"}) + UB(\texttt{"best"}) = 1.1 + 0.3 = 1.4. \tag{8.13}$$

Poiché $1.4 < \tau = 2.1$, è una rigorosa impossibilità algebrica che Doc 12 entri nel Top-k. Qualsiasi operazione di lettura, decompressione o calcolo per Doc 12 costituirebbe un'inutile dispersione di cicli macchina: lo scarto a priori è matematicamente sicuro.

Nota 8.4.1 (Metodologia di Studio e Tracciamento Manuale)**.** Per padroneggiare a fondo la meccanica di WAND e MaxScore, è fortemente consigliato esercitarsi nel tracciamento manuale passo-passo su piccoli indici invertiti giocattolo a 3 o 4 termini, simulando con rigore: l'ordinamento dinamico dei puntatori, la selezione del pivot tramite accumulo progressivo degli UB, l'esecuzione dei salti con nextGEQ, e l'aggiornamento monotono della soglia τ nell'heap.

8.5 L'Algoritmo MaxScore: Partizionamento in Liste Essenziali e Non-Essenziali

Sebbene WAND riduca drasticamente il numero di documenti valutati, l'ordinamento dinamico dei puntatori a ogni iterazione rappresenta un significativo collo di bottiglia CPU su query con molti termini. Questa sezione presenta l'algoritmo MaxScore, illustrandone la ripartizione dinamica tra liste essenziali e non-essenziali e sviluppandone il tracciamento dettagliato.

8.5.1 Riferimento Bibliografico e Principio di Funzionamento

L'algoritmo **MaxScore** è stato introdotto da Howard Turtle e James Flood [10] nell'articolo *Query evaluation: Strategies and optimizations* (Information Processing & Management, 1995). Come WAND, MaxScore calcola il risultato Top-k esatto su query disgiuntive; tuttavia, adotta una diversa filosofia architetturale che **elimina il collo di bottiglia del riordinamento dinamico dei puntatori** a ogni singolo passo.

L'idea fondamentale di MaxScore consiste nel suddividere dinamicamente le posting list della query in due insiemi disgiunti in funzione della soglia τ:

1. **Liste Non-Essenziali (*Non-Essential Lists*):** L'insieme delle posting list con gli UB più bassi la cui somma cumulativa è strettamente inferiore alla soglia τ:

$$\sum_{t \in \text{Non-Essential}} UB(t) < \tau. \tag{8.14}$$

2. **Liste Essenziali (*Essential Lists*):** Tutte le rimanenti posting list della query (quelle con gli UB più elevati).

Teorema 8.5.1 (Invariante Fondamentale di MaxScore)**.** *Un documento presente **esclusivamente** all'interno delle liste non-essenziali non potrà mai*

entrare nel Top-k. Infatti, anche nell'ipotesi massimale in cui tale documento compaia in tutte le liste non-essenziali e ottenga in ciascuna il massimo punteggio parziale, il suo score totale soddisfa:

$$s(Q,d) \leq \sum_{t \in \textit{Non-Essential}} UB(t) < \tau. \tag{8.15}$$

Conseguenza Operativa Immediata: *Qualsiasi documento appartenente al Top-k* ***deve necessariamente comparire in almeno una lista essenziale****.*

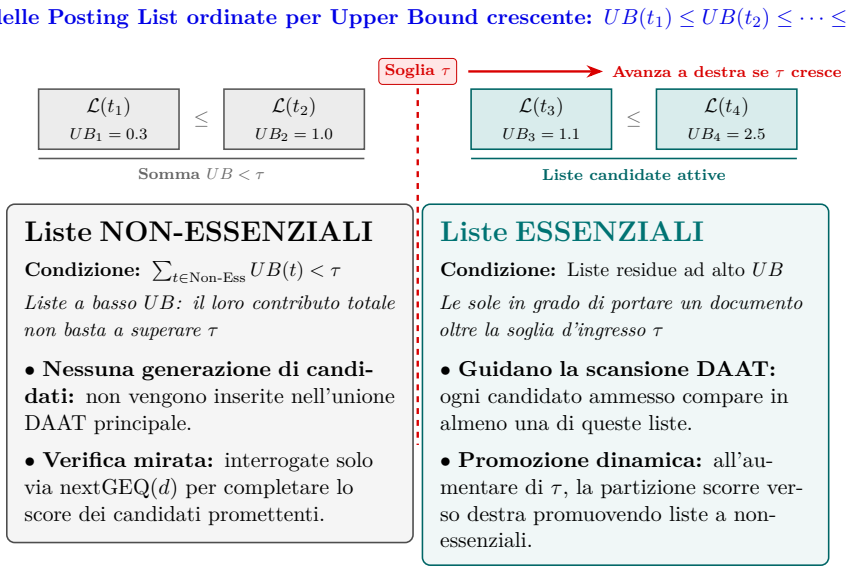


Figura 8.1: Partizionamento dinamico delle posting list nell'algoritmo MaxScore.

8.5.2 Differenze Strutturali rispetto a WAND

1. **Ordinamento Statico Iniziale:** In MaxScore le posting list vengono ordinate per UB crescente **una sola volta** all'avvio dell'interrogazione:

   $$UB(t_1) \leq UB(t_2) \leq \cdots \leq UB(t_m). \tag{8.16}$$

   Durante l'esecuzione dell'algoritmo non è richiesto alcun riordinamento dinamico dei puntatori.

2. **Scansione DAAT Selettiva:** La scansione DAAT OR viene eseguita **esclusivamente sull'insieme delle liste essenziali**. Le liste non-essenziali sono passive: non partecipano all'avanzamento dei puntatori primari.

3. **Promozione Dinamica delle Liste:** Man mano che l'heap si popola e la soglia τ cresce, la sommatoria degli UB sul prefisso delle liste può inglobare nuovi termini senza eccedere τ. La frontiera di demarcazione si sposta verso destra: liste originariamente essenziali vengono **promosse a non-essenziali**, riducendo progressivamente il numero di liste attive da scansionare.

8.5.3 Valutazione Selettiva del Documento Candidato

L'algoritmo esamina un candidato d generato dall'unione delle sole liste essenziali ($d = \min_{t \in \text{Ess}} p_t.\text{CurrentDocID}()$):

1. Calcola il punteggio parziale esatto generato dalle liste essenziali: $s_{\text{ess}}(Q, d)$.
2. Calcola la **stima ottimistica massima** che il documento d potrebbe raggiungere sommando il contributo potenziale massimo di tutte le liste non-essenziali:

$$\text{Stima}(d) = s_{\text{ess}}(Q, d) + \sum_{t \in \text{Non-Essential}} UB(t). \tag{8.17}$$

3. **Controllo della Stima:**
   - Se $\text{Stima}(d) \leq \tau$: il documento non ha alcuna possibilità di entrare nel Top-k. Viene **scartato immediatamente senza toccare né decomprimere alcuna lista non-essenziale**.
   - Se $\text{Stima}(d) > \tau$: il candidato merita la valutazione. Si interrogano le liste non-essenziali invocando $p_t.\text{nextGEQ}(d)$. Se il documento è presente, si aggiunge il punteggio parziale effettivo. Qualora il punteggio finale superi τ, il documento entra nell'heap e τ viene incrementata.

8.5.4 Tracciamento Dettagliato dell'Algoritmo MaxScore

Si consideri il medesimo scenario di simulazione con i quattro termini ordinati per Upper Bound:

- **Ordinamento Statico per UB Crescente:**
  1. `"best"`: $UB = 0.3$
  2. `"programming"`: $UB = 1.0$

3. `"language"`: $UB = 1.1$
4. `"rust"`: $UB = 2.5$

- **Soglia Iniziale del Min-Heap:** $\tau = 2.1$.

Partizionamento Iniziale:

- Lista `"best"`: $UB = 0.3 < 2.1$.
- Aggiungiamo `"programming"`: $0.3 + 1.0 = 1.3 < 2.1$.
- Se aggiungessimo `"language"` (1.1), la somma cumulativa diverrebbe $1.3 + 1.1 = 2.4 \geq 2.1$.
- Pertanto la partizione iniziale risulta:

$$\text{Non-Essential} = \{\texttt{"best"}, \texttt{"programming"}\} \quad (\textstyle\sum UB = 1.3),$$
$$\text{Essential} = \{\texttt{"language"}, \texttt{"rust"}\}.$$

Scansione Guidata dalle Liste Essenziali:

1. **Candidato Doc 10 (proposto da $\mathcal{L}$(`"language"`), in quanto $10 <$ 15 di `"rust"`):**
   - Punteggio parziale reale in `"language"`: $s(\texttt{"language"}, 10) = 0.5$.
   - Calcolo della stima ottimistica:
   $$\text{Stima}(10) = 0.5 + \sum_{t \in \text{Non-Ess}} UB(t) = 0.5 + 1.3 = 1.8.$$
   - Confronto: $1.8 < \tau = 2.1 \implies$ **Doc 10 viene scartato all'istante senza accedere a `"best"` o `"programming"`!**
2. **Candidato Doc 13 (proposto da $\mathcal{L}$(`"language"`), con $13 < 15$):**
   - Punteggio parziale reale in `"language"`: $s(\texttt{"language"}, 13) = 0.9$.
   - Calcolo della stima ottimistica:
   $$\text{Stima}(13) = 0.9 + 1.3 = 2.2.$$
   - Confronto: $2.2 > \tau = 2.1 \implies$ Doc 13 è promettente. Si interrogano le liste non-essenziali con nextGEQ(13):
     - In $\mathcal{L}$(`"programming"`), Doc 13 è presente con score parziale 0.5.

  - In $\mathcal{L}(\texttt{"best"})$, Doc 13 è presente con score parziale 0.1.
- Punteggio reale complessivo: $s(Q, 13) = 0.9 + 0.5 + 0.1 = 1.5$.
- Confronto finale: $1.5 < \tau = 2.1 \implies$ Doc 13 non entra nell'heap e viene scartato.
- *Nota sul Doc 12:* Si osservi che Doc 12 è stato scavalcato a monte senza essere mai preso in considerazione, poiché non compariva in alcuna lista essenziale.

3. **Candidato Doc 15 (proposto da $\mathcal{L}(\texttt{"rust"})$):**
   - Punteggio parziale reale in `"rust"`: $s(\texttt{"rust"}, 15) = 2.5$.
   - Calcolo della stima ottimistica:

$$\text{Stima}(15) = 2.5 + 1.3 = 3.8.$$

   - Confronto: $3.8 > \tau = 2.1 \implies$ Doc 15 merita la valutazione. Si interrogano le liste non-essenziali:
     - In $\mathcal{L}(\texttt{"programming"})$, presente con score 1.0.
     - In $\mathcal{L}(\texttt{"best"})$, presente con score 0.2.
   - Punteggio reale complessivo: $s(Q, 15) = 2.5 + 1.0 + 0.2 = 3.7$.
   - Confronto finale: $3.7 > 2.1 \implies$ **Doc 15 entra nell'heap** e la soglia minima sale a $\tau = 3.7$.

Aggiornamento della Partizione dopo l'Aumento della Soglia ($\tau = 3.7$): Con la nuova soglia $\tau = 3.7$, la somma di `"best"` (0.3) + `"programming"` (1.0) + `"language"` (1.1) è pari a:

$$0.3 + 1.0 + 1.1 = 2.4 < 3.7.$$

Anche la lista `"language"` viene promossa a non-essenziale:

- Non-Essential $= \{\texttt{"best"}, \texttt{"programming"}, \texttt{"language"}\}$ $(\sum UB = 2.4 < 3.7)$;
- Essential $= \{\texttt{"rust"}\}$.

Per l'intero prosieguo dell'elaborazione, l'unica lista attiva che genera candidati rimane $\mathcal{L}(\texttt{"rust"})$, riducendo drasticamente il costo computazionale.

Lo pseudocodice completo di MaxScore è formalizzato nell'Algoritmo 4.

Algorithm 4: Algoritmo MaxScore Top-k Retrieval

Dati: Query $Q = \{t_1, \ldots, t_m\}$, Indice Invertito, array $UB[t]$, intero k
Risultato: Min-Heap H contenente i Top-k documenti

```
1  H ← InizializzaMinHeap(capacità = k)
2  τ ← 0
3  Liste ← OrdinaPerUBCrescente(Q)        // Ordinamento statico una sola volta
4  nonEssEnd ← CalcolaFrontieraNonEssenziali(Liste, τ)
5  while ∃i > nonEssEnd tale che Liste[i].ÈAttivo() do
6      d ← min{Liste[i].CurrentDocID() | i > nonEssEnd ∧ Liste[i].ÈAttivo()}
7      s_partial ← 0
8      for i ← nonEssEnd + 1 to |Liste| do
9          if Liste[i].ÈAttivo() ∧ Liste[i].CurrentDocID() == d then
10             s_partial ← s_partial + Liste[i].CurrentScore()
11             Liste[i].Next()
12     stima ← s_partial + Σ_{j=1}^{nonEssEnd} UB[Liste[j].term]
13     if stima > τ then
14         for j ← 1 to nonEssEnd do
15             Liste[j].nextGEQ(d)
16             if Liste[j].ÈAttivo() ∧ Liste[j].CurrentDocID() == d then
17                 s_partial ← s_partial + Liste[j].CurrentScore()
18         if s_partial > τ then
19             if |H| == k then
20                 H.ExtractMin()
21             H.Insert(⟨d, s_partial⟩)
22             if |H| == k then
23                 τ ← H.FindMin().score
24                 nonEssEnd ← CalcolaFrontieraNonEssenziali(Liste, τ)
25 return H
```

8.6 Valutazione Sperimentale e Varianti a Blocchi: BMW e VBMW

L'adozione di un unico Upper Bound globale per un'intera posting list soffre della presenza di valori anomali, che limitano la frequenza dei salti. Questa

sezione analizza le evidenze empiriche del benchmark standard Gov2, introduce la segmentazione a blocchi di dimensione fissa (Block-Max WAND, BMW) e formalizza la variante a dimensione variabile (Variable Block-Max WAND, VBMW) per minimizzare l'errore di sovrastima.

8.6.1 Valutazione Sperimentale Comparativa (Collezione Gov2 - TREC 05)

L'efficacia pratica degli algoritmi di Dynamic Pruning è documentata dai benchmark accademici su larga scala. La Tabella 8.2 riporta i tempi medi di risposta per query (in millisecondi) misurati sulla collezione standard **Gov2** (25 milioni di pagine web, benchmark TREC 05), al variare della lunghezza della query (da 2 a oltre 6 termini).

Algoritmo	2 termini	3 termini	4 termini	5 termini	6+ termini	Media (*Avg*)	Spazio Extra
RankedOR (Esaustivo)	23.6 ms	76.5 ms	147.9 ms	235.4 ms	418.7 ms	106.7 ms	0.00%
WAND	5.1 ms	5.9 ms	7.0 ms	8.8 ms	17.8 ms	7.0 ms	0.22%
MaxScore	4.7 ms	6.0 ms	7.1 ms	9.2 ms	14.2 ms	6.6 ms	0.22%
BMW$_{40}$	1.2 ms	2.9 ms	4.3 ms	6.7 ms	14.8 ms	3.6 ms	1.55%
VBMW$_{40}$	**0.8** ms	**1.7** ms	**2.4** ms	**3.8** ms	**8.1** ms	**2.1** ms	1.55%

Tabella 8.2: Tempi medi di query processing (Top-10) su collezione Gov2 (TREC 05) al variare del numero di termini. Fonte: Ding & Suel [11], Mallia et al. [12].

Considerazioni Critiche sui Risultati:

1. **Crescita Non Lineare di RankedOR:** Nella scansione esaustiva, il tempo di risposta scala drammaticamente all'aumentare dei termini (418.7 ms per query con 6+ termini), poiché l'unione accumula decine di milioni di identificatori da verificare.

2. **Confronto Diretto tra WAND e MaxScore:** Entrambi gli algoritmi riducono la latenza media da oltre 100 ms a circa 6.6–7.0 ms (un'accelerazione di oltre 15×). WAND risulta lievemente più rapido su query corte (2 termini), mentre MaxScore prevale sulle query lunghe (6+ termini): su insiemi ampi di liste, l'assenza di riordinamento dinamico dei puntatori in MaxScore compensa ampiamente qualsiasi vantaggio di salto di WAND.

3. **Analisi del Costo Spaziale (**0.22% **vs** 1.55%**):** Il dato 0.22% indica l'ingombro percentuale dei metadati degli Upper Bound rispetto alla dimensione complessiva dell'indice compresso su disco. Trattandosi di

un singolo valore float per ciascun termine del dizionario, il sovraccarico di memoria per WAND e MaxScore è del tutto trascurabile. Nelle varianti a blocchi (1.55%), l'indice memorizza un float aggiuntivo per ogni blocco compresso; tale modesto incremento di spazio è ripagato da un ulteriore dimezzamento della latenza media (2.1 ms).

8.6.2 Il Limite dell'Upper Bound Globale e l'Approccio Block-Max WAND (BMW)

Il limite intrinseco di WAND e MaxScore consiste nell'assegnare **un unico Upper Bound globale a un'intera posting list**. Se una lista contiene 10 milioni di posting e un singolo documento isolato presenta un picco eccezionale di frequenza (ad esempio un outlier con score parziale 8.0), l'Upper Bound globale dell'intera lista sarà forzato a $UB = 8.0$. Di conseguenza, anche se il restante 99.99% dei documenti possiede punteggi inferiori a 1.0, la stima ottimistica della query resterà artificialmente maggiorata lungo tutti i 10 milioni di elementi, impedendo alla CPU di compiere salti significativi.

Per risolvere questo problema, Ding e Suel [11] hanno introdotto **Block-Max WAND (BMW)**:

- La posting list viene partizionata in **blocchi logici contigui di dimensione fissa** (tipicamente $B = 40, 64$ o 128 documenti).
- Per ciascun blocco b, l'indice memorizza il massimo identificatore contenuto nel blocco (lastDocID_b) e un **Upper Bound locale di blocco**:
$$UB_{\text{blocco}}(b) = \max_{d \in b} s(t, d). \tag{8.18}$$
- Durante il calcolo, la CPU stima il potenziale del documento corrente utilizzando l'UB del blocco locale a cui appartiene, anziché il massimo globale dell'intera lista. Se l'UB locale sommato agli altri contributi non supera τ, l'**intero blocco di 40 o 64 documenti viene saltato e scartato in un unico passaggio**, senza accedere ai posting compressi.

8.6.3 Variable Block-Max WAND (VBMW) e Minimizzazione dell'Area di Sovrastima

Sebbene BMW migliori significativamente le prestazioni, l'adozione di blocchi a dimensione fissa introduce un'inefficienza strutturale dovuta alla presenza degli outlier: se un blocco fisso di 40 elementi racchiude 39 documenti con

punteggio 1.0 e un solo outlier con punteggio 8.0, l'intero blocco riceve un UB locale pari a 8.0, creando un ampio **errore di sovrastima**.

Mallia et al. [12] hanno proposto **Variable Block-Max WAND (VBMW)**, in cui la suddivisione della posting list non avviene a intervalli regolari, bensì **adattando dinamicamente la dimensione dei blocchi** alla forma della distribuzione dei punteggi. La Figura 8.2 illustra il confronto geometrico tra le due filosofie.

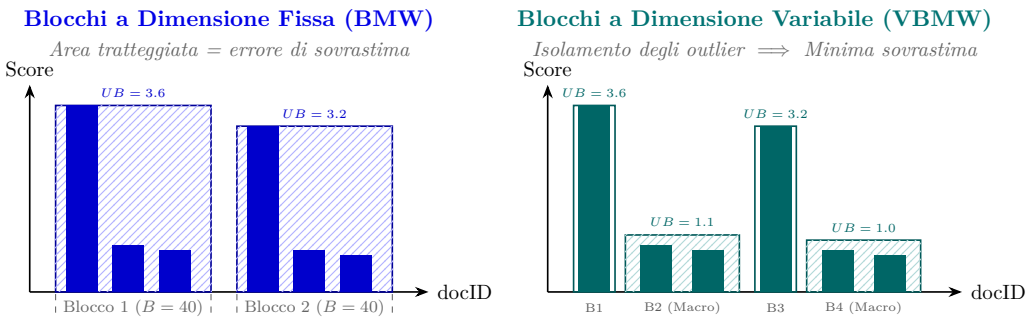


Figura 8.2: Confronto geometrico tra partizionamento a blocchi di dimensione fissa (BMW) e a dimensione variabile (VBMW). L'area tratteggiata rappresenta l'errore di sovrastima: VBMW la riduce al minimo comprimendo gli intervalli omogenei e isolando i picchi.

Il partizionamento a dimensione variabile viene determinato offline tramite un algoritmo di programmazione dinamica:

- Sequenze contigue di documenti con punteggi uniformemente bassi vengono raggruppate in macro-blocchi estesi ($B > 100$) aventi un UB locale molto basso ($UB \approx 1.0$). Poiché l'UB del blocco è modesto, il macro-blocco viene quasi sempre saltato in blocco durante la query.
- Gli outlier e i documenti ad altissima rilevanza vengono confinati in micro-blocchi dedicati composti da pochissimi elementi.
- **Impatto Pratico:** L'area di sovrastima viene minimizzata, consentendo a VBMW di raggiungere una latenza media di appena 2.1 **ms** su collezione Gov2, con un fattore di velocizzazione fino a 50× rispetto alla scansione esaustiva di base.

8.7 Sistemi Industriali: L'Ecosistema Apache Lucene ed Elasticsearch

I modelli teorici di potatura dinamica trovano riscontro diretto nell'architettura dei motori di ricerca distribuiti di produzione. Questa sezione analizza il ruolo di Apache Lucene ed Elasticsearch, ne descrive l'evoluzione decennale fino all'adozione del paradigma *Block-Max MaxScore*, e riflette sui complessi requisiti architetturali e di retrocompatibilità che separano la ricerca accademica dal software industriale.

8.7.1 L'Architettura di Apache Lucene

Le strutture dati e gli algoritmi formalizzati in questo capitolo non sono mere astrazioni accademiche, ma costituiscono il motore pulsante dell'infrastruttura di ricerca open-source più diffusa al mondo:

- **Apache Lucene:** Libreria software core scritta in Java (originariamente concepita da Doug Cutting), responsabile della gestione a basso livello degli indici invertiti, dei codec di compressione su segmenti immutabili e dell'elaborazione di query complesse con scoring BM25.
- **Elasticsearch e Apache Solr:** Motori di ricerca e analisi distribuiti di livello enterprise costruiti al di sopra di Lucene, che offrono scalabilità orizzontale tramite partizionamento (*sharding*), replica e API RESTful.

8.7.2 L'Evoluzione Algoritmica in Lucene e la Sintesi di Adrien Grand (2023)

Nel dicembre 2023, Adrien Grand (Principal Software Engineer presso Elastic e tra i principali architetti di Lucene) ha pubblicato un'approfondita analisi tecnica intitolata *"MAXSCORE & block-max MAXSCORE: More skipping with block-max MAXSCORE"* [13]. L'articolo ripercorre la traiettoria ingegneristica compiuta da Lucene in oltre un decennio, riassumibile in **quattro fasi evolutive**:

1. **Fase 1 (Valutazione Esaustiva):** Scansione DAAT standard di tutti i posting con Min-Heap; tempo di risposta dominato dalla decompressione integrale delle liste.
2. **Fase 2 (Adozione di WAND):** Integrazione del Dynamic Pruning basato su WAND (classe `TopScoreDocCollector`) con skipping basato su Upper Bound globali, introdotta in Lucene 8.0.

3. **Fase 3 (Transizione a MaxScore):** Sostituzione di WAND con MaxScore per eliminare il costo del riordinamento continuo dei puntatori e massimizzare il throughput su query contenenti molti termini disgiuntivi.

4. **Fase 4 (Integrazione di Block-Max MaxScore):** Combinazione dello schema di partizionamento essenziale/non-essenziale di MaxScore con i metadati di blocco locali (ispirati a Block-Max WAND). I blocchi dell'indice su disco integrano direttamente gli UB locali, consentendo a Lucene di compiere salti a due livelli (*two-level skipping*) su indici industriali distribuiti.

8.7.3 Il Divario tra Teoria Accademica e Produzione Industriale

Un aspetto di grande valore metodologico riguarda il **divario temporale di circa 13 anni** intercorso tra la pubblicazione scientifica delle varianti a blocchi (Ding & Suel, 2011 [11]) e la loro piena adozione stabile nei formati di indice di Apache Lucene:

- **Riprogettazione dei Codec di Indice:** Nei sistemi di produzione che gestiscono petabyte di dati, l'aggiunta di metadati di blocco non può essere realizzata banalmente. Richiede la completa riscrittura dei codec di serializzazione su disco (*Lucene Postings Format*), garantendo al contempo:
  - Retrocompatibilità binaria assoluta con gli indici generati dalle versioni software precedenti;
  - Supporto efficiente per operazioni concorrenti di fusione dei segmenti (*segment merging*);
  - Conservazione del budget I/O e assenza di frammentazione della memoria secondaria.
- **Robustezza su Workload Eterogenei:** Nei benchmark accademici, gli algoritmi vengono valutati su collezioni statiche e query disgiuntive pure. In un motore di produzione reale come Elasticsearch, la stessa pipeline di esecuzione deve gestire simultaneamente:
  - Filtri booleani strutturati arbitrariamente nidificati (`must`, `should`, `filter`);
  - Match di prossimità sintattica e query a frase (`match_phrase`);

– Calcolo parallelo di aggregazioni analitiche (sfaccettature, conteggi, metriche);
– Aggiornamenti e inserimenti di nuovi documenti in tempo reale (*near-real-time indexing*).

Un'ottimizzazione algoritmica può essere integrata nel core di Lucene solo se **non introduce alcuna regressione prestazionale** su nessuno degli altri scenari operativi.

8.8 Quadro Sinottico Comparativo degli Algoritmi Top-k

A conclusione della trattazione sul query processing top-k, la Tabella 8.3 sintetizza le proprietà algebriche, computazionali e architetturali di tutti gli approcci trattati nel capitolo, offrendo un confronto sinottico tra le strategie esaustive e i metodi di dynamic pruning.

Algoritmo	**Strategia di Scansione**	**Ordinamento Liste**	**Granularità di Pruning**	**Spazio Ausiliario**	**Latenza Media**
DAAT Esaustivo	Unione sincrona di tutte le liste	Heap dei puntatori o scansione lineare	Nessuna (visita tutti i posting)	$\mathcal{O}(k)$ (solo MinHeap)	106.7 ms (Baseline)
WAND	Scansione guidata dal Pivot Document	Dinamico a ogni passo ($d_1 \leq \cdots \leq d_m$)	Documento singolo (UB globale di termine)	$\mathcal{O}(\lvert\mathcal{V}\rvert)$ float in RAM	$\approx$ 7.0 ms ($\approx 15\times$)
MaxScore	Partizione in liste Essenziali e Non-Essenziali	Statico una sola volta per UB crescente	Documento singolo (scansione solo su essenziali)	$\mathcal{O}(\lvert\mathcal{V}\rvert)$ float in RAM	$\approx$ 6.6 ms ($\approx 16\times$)
BMW	WAND su blocchi a dimensione fissa	Dinamico sui blocchi correnti	Blocco fisso ($B = 40, 64$ posting)	+1.55% su disco (UB per blocco)	$\approx$ 3.6 ms ($\approx 30\times$)
VBMW	WAND su blocchi a dimensione variabile	Dinamico adattivo	Macro/Micro blocchi ottimizzati da DP	+1.55% su disco (UB per blocco)	$\approx$ **2.1** ms ($\approx$ **50**$\times$)

Tabella 8.3: Quadro sinottico comparativo degli algoritmi di Top-k Retrieval su query disgiuntive.

Bibliografia

[1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.

[3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.

[4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

[5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.

[6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.

[7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.

[8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.

[9] Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Y. Zien. Efficient query evaluation using a two-level retrieval process. In *Proceedings of the 12th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 426–434, 2003.

[10] Howard Turtle and James Flood. Query evaluation: Strategies and optimizations. *Information Processing & Management*, 31(6):831–850, 1995.

[11] Shuai Ding and Torsten Suel. Faster top-k document retrieval using block-max indexes. In *Proceedings of the 4th ACM International Conference on Web Search and Web Data Mining (WSDM)*, pages 193–202, 2011.

[12] Antonio Mallia, Giuseppe Ottaviano, Elia Porciani, Nicola Tonellotto, and Rossano Venturini. Faster block-max WAND with variable-sized blocks. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 625–634, 2017.

[13] Adrien Grand. MAXSCORE & block-max MAXSCORE: More skipping with block-max MAXSCORE. Elasticsearch Labs Technical Blog, 2023.